

Національний університет «Львівська політехніка»
Кафедра програмного забезпечення

КУРСОВА РОБОТА
з дисципліни «Бази даних»
на тему:
**«Інформаційна система для
автоматизації роботи служби таксі»**

Виконав:
студент спеціальності 121
«Інженерія програмного забезпечення»
групи ПЗ-33
Гамела Б. А.

Керівник:
доцент кафедри програмного забезпечення,
Білоіваненко М. В.

Оцінка:
Національна шкала _____
Кількість балів _____ Оцінка ECTS _____

Члени комісії
БІЛОІВАНЕНКО

_____ Максим
(підпис)

Львів – 2026 рік

Зміст

Зміст.....	2
1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАВДАННЯ.....	4
1.1. Опис предметної області (актуальність теми, визначення важливих понять, сфера застосування системи, огляд існуючих аналогічних систем).....	4
1.2. Вимоги до обробки даних (характеристика розглянутих бізнес процесів, способів накопичення та аналізу даних, форми введення та виведення даних).....	6
1.3. Постановка завдання (призначення інформаційної системи, ролі потенційних користувачів, перелік функцій).....	8
2. РОЗРОБКА МОДЕЛЕЙ ДАНИХ.....	10
2.1. Концептуальне проектування (взаємозв'язки та характеристики понять, що описані в базі даних).....	10
2.2. Вимоги до системи накопичення даних (оцінка обсягів даних, частота додавання, очікувані запити та вибірки, обмеження доступу тощо).....	11
2.3. Логічне проектування бази даних (домени, ключі, таблиці, обмеження).....	13
2.4. Реалізація процедур бізнес-логіки (тригери, процедури, представлення).....	19
3. РОЗРОБКА ПРОГРАМНОГО КОДУ	21
3.1. Обґрунтування обраної архітектури (тип, мова програмування, система баз даних, протоколи обміну)	21
3.2. Структурна модель інформаційної системи	22
3.3. Призначення модулів та компонентів системи.....	23
3.4. Опис реалізації функцій системи	25
3.5. Опис реалізації елементів інтерфейсу	26
4. ОПРАЦЮВАННЯ ДАНИХ.....	29
4.1. Характеристика первинного джерела даних (структура, обсяг, оцінка об'єктивності даних).....	29
4.2. Алгоритм трансформації даних при завантаженні.....	29
4.3. Алгоритм генерації тестових даних.....	30
4.4. Опис додаткових методів обробки даних.....	32
5. ФУНКЦІОНАЛЬНІ МОЖЛИВОСТІ СИСТЕМИ.....	36
5.1. Інтерфейси управління даними	36
5.2. Панель показників бізнес-процесу.....	40

5.3. Засоби аналітичного представлення даних	42
5.4. Опис додаткових реалізованих функцій.....	42
ВИСНОВКИ.....	44
СПИСОК ЛІТЕРАТУРИ.....	46
ДОДАТКИ.....	47
Додаток А. Скрипт створення бази даних.....	47
Додаток Б. Скрипт завантаження первинних даних	53
Додаток В. Програмний код реалізації бізнес-логіки	59
Додаток Г. Зразки сторінок графічного інтерфейсу.....	78

1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАВДАННЯ

1.1. Опис предметної області (актуальність теми, визначення важливих понять, сфера застосування системи, огляд існуючих аналогічних систем)

Сучасний ринок онлайн-освіти переживає стрімке зростання. За даними аналітичних агенцій, обсяг глобального ринку електронного навчання у 2025 році перевищив 360 мільярдів доларів США, а щорічний темп зростання складає понад 7%. Причинами такої динаміки є зростання попиту на гнучкі формати навчання, можливість здобувати знання незалежно від географічного розташування, а також широке поширення мобільних пристроїв та високошвидкісного доступу до мережі Інтернет.

Навчальна онлайн-платформа – це програмний комплекс, що забезпечує створення, публікацію та продаж навчальних курсів, а також надає інструменти для проходження навчальних матеріалів, відстеження прогресу та оцінювання якості курсів. Такі платформи є посередником між двома основними категоріями учасників: викладачами, які створюють навчальний контент, та студентами, які його споживають. Додатково присутня роль адміністратора, що відповідає за фінансовий моніторинг та модерацію користувачів.

Основними поняттями предметної області є:

- **курс (клієнт)** – структурована одиниця навчального контенту, що має назву, опис, ціну, рівень складності та складається з упорядкованої послідовності уроків. Курс створюється викладачем і стає доступним студентам після публікації;
- **урок** – окремий елемент навчального курсу, що містить текстовий навчальний матеріал. Уроки мають визначений порядок в межах курсу;

- запис на курс (реєстрація) – факт приєднання студента до курсу, який фіксує дату початку навчання та відсоток пройденого матеріалу;
- прогрес навчання – автоматично обчислюваний показник, що відображає частку уроків, позначених студентом як завершені;
- відгук – оцінка (від 1 до 5) та текстовий коментар, який студент може залишити після запису на курс. Один студент може залишити лише один відгук на один курс;
- оплата – фінансова транзакція, що фіксується при записі студента на курс. Включає суму, статус (завершена або повернена) та дату;
- виплата – запит викладача на отримання зароблених коштів. Проходить через стадії: очікує розгляду, схвалена, виплачена або відхилена;
- категорія – тематична класифікація курсів. Один курс може належати до кількох категорій одночасно.

Серед існуючих аналогічних систем найвідомішими є Udemy, Coursera та Skillshare. Udemy є найбільш наближеною до розроблюваної системи за моделлю: будь-який викладач може створити курс та продавати його, платформа виступає посередником. Coursera працює переважно з університетами та пропонує акредитовані програми, що виходить за межі даної роботи. Skillshare використовує підписну модель замість поодиноких оплат. Розроблювана система є спрощеним аналогом Udemy, адаптованим для демонстрації принципів проектування реляційних баз даних та реалізації серверної бізнес-логіки.

1.2. Вимоги до обробки даних (характеристика розглянутих бізнес процесів, способів накопичення та аналізу даних, форми введення та виведення даних)

Інформаційна система сервісу таксі «Moment Taxi» повинна забезпечувати ефективне накопичення, зберігання та обробку даних, необхідних для підтримки основних бізнес-процесів.

Основними бізнес-процесами, що підтримуються системою, є:

- **Продаж навчальних курсів.** Викладач створює курс у стані чернетки, заповнює його метадані (назва, опис, ціна, рівень складності, категорії), додає уроки з навчальним матеріалом та публікує курс. Студент переглядає каталог опублікованих курсів із можливістю пошуку за ключовими словами, фільтрації за категоріями, ціною, рівнем складності та рейтингом, а також сортування за різними критеріями. Після вибору курсу студент здійснює оплату, що створює запис про реєстрацію та фіксує фінансову транзакцію. Адміністратор має можливість оформити повернення коштів, що скасовує реєстрацію та каскадно видаляє пов'язаний прогрес і відгук.
- **Процес проходження навчальних матеріалів.** Зареєстрований студент послідовно переглядає уроки курсу та позначає їх як завершені. Система автоматично перераховує відсоток прогресу на рівні бази даних за допомогою тригера. Після запису на курс студент може залишити відгук із числовою оцінкою та текстовим коментарем. Середня оцінка курсу обчислюється динамічно при кожному запиті. Система аналізує шаблони реєстрацій студентів та пропонує персоналізовані рекомендації на основі алгоритму асоціативних правил.
- **Виплата винагороди викладачам.** Кошти від оплат студентів зараховуються на баланс відповідного викладача автоматично за допомогою тригера бази даних. Викладач може переглянути свій поточний баланс та подати запит на виплату. Адміністратор розглядає запити, змінюючи їх статус (схвалити, відхилити, позначити виплаченим). Зміна статусу виплати також

автоматично перераховує баланс викладача за допомогою відповідного тригера.

Накопичення даних у системі відбувається трьома способами. По-перше, безпосередньо через дії користувачів в інтерфейсі: викладач створює курси та уроки, студент записується на курс, позначає уроки як завершені, залишає відгуки, адміністратор змінює статуси оплат і виплат. По-друге, автоматично на рівні бази даних – тригери перераховують похідні показники (відсоток прогресу навчання, баланс викладача) при кожній зміні відповідних базових записів, забезпечуючи узгодженість агрегованих значень без участі прикладного коду. По-третє, шляхом початкового завантаження – спеціалізована програма-завантажувач зчитує дані про курси із зовнішнього CSV-файлу та генерує пов'язані записи (користувачі, реєстрації, оплати, відгуки) за визначеними правилами.

Аналіз накопичених даних реалізовано на кількох рівнях. Для викладачів передбачена аналітична панель окремого курсу, що містить показники кількості записаних студентів, загального доходу, середнього рейтингу, відсотку завершення курсу та лінійний графік динаміки реєстрацій за обраний період. Для адміністратора — фінансова панель із агрегованими показниками валового доходу, повернених коштів та виплат викладачам, доповнена графіком динаміки доходів і виплат за обраний часовий діапазон. Крім того, система застосовує алгоритм асоціативних правил для аналізу шаблонів реєстрацій та формування персоналізованих рекомендацій курсів для кожного студента.

Введення даних у систему здійснюється через графічний інтерфейс користувача у вигляді форм (напр., форма реєстрації та авторизації користувачів, форма створення та редагування курсу тощо). Виведення інформації реалізується у вигляді табличних подань, списків, аналітичних панелей та списку рекомендованих курсів із поясненням алгоритму підбору.

1.3. Постановка завдання (призначення інформаційної системи, ролі потенційних користувачів, перелік функцій)

Метою курсової роботи є проектування та реалізація інформаційної системи для управління навчальною онлайн-платформою. Система призначена для автоматизації процесів створення та продажу навчальних курсів, відслідковування процесу навчання студентів та моніторингу їх успішності.

Система реалізує трирівневу клієнт-серверну архітектуру: клієнтський вебзастосунок взаємодіє із сервером прикладної логіки через REST API, а сервер зберігає дані у реляційній системі управління базами даних (СУБД) PostgreSQL.

У системі передбачено три ролі користувачів:

- Студент – кінцевий споживач навчального контенту. Має доступ до каталогу курсів із пошуком і фільтрацією, може записуватися на курси, проходити уроки, відстежувати свій прогрес, залишати відгуки та переглядати персоналізовані рекомендації.
- Викладач – автор навчальних курсів. Має доступ до інструментів створення та редагування курсів і уроків, публікації курсів, перегляду аналітики власних курсів (кількість студентів, дохід, рейтинг, динаміка реєстрацій), перегляду відгуків студентів, а також управління виплатами (перегляд балансу, подання запитів на виплату).
- Адміністратор (Admin) – модератор платформи. Має доступ до фінансової панелі з агрегованими показниками, управління оплатами (перегляд, оформлення повернень), управління виплатами викладачам (схвалення, відхилення, позначення виплаченими), а також управління користувачами (перегляд, призупинення та відновлення облікових записів).

Основними функціями інформаційної системи є:

- створення та редагування навчальних курсів та уроків;
- пошук та фільтрація курсів за категоріями, ціною, рівнем складності, рейтингом та ключовими словами;

- запис студента на курс із фіксацією оплати;
- процес проходження курсу: перегляд уроків та автоматичне відстеження прогресу;
- можливість залишити оцінку та текстовий відгук на курс;
- фіксація оплат та управління виплатами викладачам;
- аналітичні панелі для викладачів та адміністраторів із графічним представленням динаміки показників;
- управління користувачами (призупинення та відновлення облікових записів).

Додаткова функція: реалізація алгоритму підбору рекомендованих курсів для студентів на основі методу асоціативних правил (association rule mining) із використанням метрик підтримки (support), впевненості (confidence) та підйому (lift).

2. РОЗРОБКА МОДЕЛЕЙ ДАНИХ

2.1. Концептуальне проектування (взаємозв'язки та характеристики понять, що описані в базі даних)

На етапі концептуального проектування було визначено основні сутності предметної області та зв'язки між ними. Результатом є діаграма «сутність – зв'язок» (ER-діаграма) (див. рис. 2.1.).

Предметна область системи охоплює такі взаємопов'язані сутності:

- Користувач (User) – центральна сутність, що описує будь-якого зареєстрованого учасника системи. Має атрибути імені, прізвища, електронної пошти, ролі (студент, викладач або адміністратор) та статусу облікового запису (активний або призупинений);
- Профіль викладача (TeacherProfile) – розширення користувача з роллю «викладач». Містить біографію та поточний баланс нарахованих коштів. Зв'язок 1:1 із користувачем;
- Курс (Course) – навчальний курс, створений викладачем. Має назву, опис, ціну, рівень складності, статус публікації та дату створення. Зв'язок N:1 із профілем викладача (автор);
- Категорія (Category) – тематична класифікація. Зв'язок M:N із курсами через проміжну таблицю;
- Урок (Lesson) – окремий елемент курсу з назвою, текстовим вмістом та порядковим номером. Зв'язок N:1 із курсом;
- Запис на курс (Enrollment) – реєстрація студента на курс. Фіксує дату та відсоток прогресу. Зв'язок N:1 із користувачем (студентом) та N:1 із курсом;
- Прогрес уроку (LessonProgress) – факт завершення конкретного уроку в межах конкретного запису. Зв'язок N:1 із записом та N:1 із уроком. Має композитний первинний ключ;

- Відгук (Review) – оцінка та коментар до курсу. Первинним ключем є ідентифікатор запису на курс, що структурно гарантує обмеження «один відгук на один запис». Зв'язок 1:1 із записом;
- Оплата (Payment) – фінансова транзакція при записі на курс. Зв'язок N:1 із користувачем та N:1 із курсом;
- Виплата (Payout) – запит викладача на виведення коштів. Зв'язок N:1 із профілем викладача.

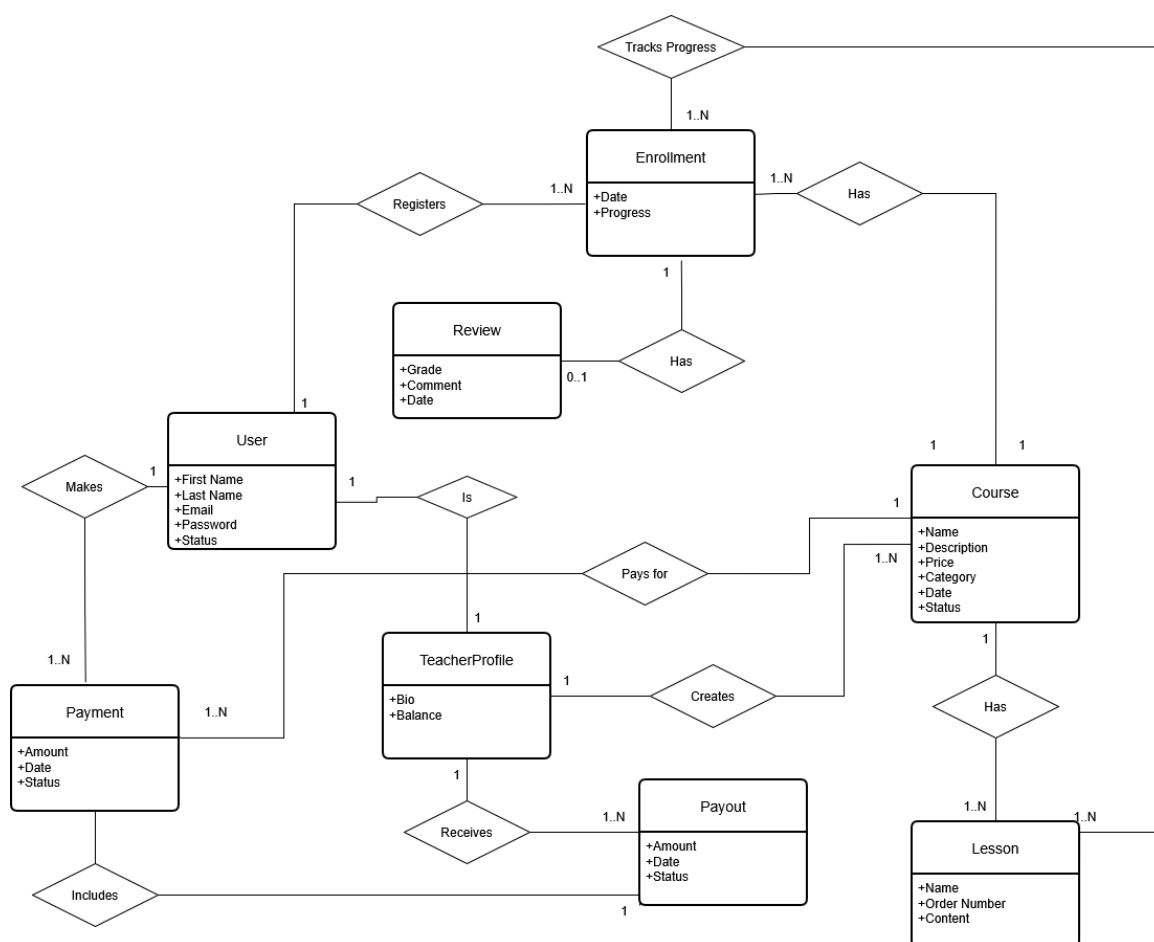


Рис 2.1. Діаграма «сутність – зв'язок» інформаційної системи

2.2. Вимоги до системи накопичення даних (оцінка обсягів даних, частота додавання, очікувані запити та вибірки, обмеження доступу тощо)

Система розрахована на роботу з обсягом даних навчальної онлайн-платформи середнього розміру. Для тестування та демонстрації використовується

набір із понад 3 600 реальних курсів (з відкритого набору даних Udemu Courses на платформі Kaggle ДЖЕРЕЛО), доповнений згенерованими записами. Загальний обсяг тестових даних перевищує 50 000 записів:

- 2500 користувачів (викдалачів та студентів);
- 27000 уроків;
- 18000 записів на курси;
- 10000 записів прогресу уроків;
- 14000 відгуків;
- 18000 оплат;
- 1500 виплат;

Транзакційні записи (оплати, записи на курси, прогрес) створюються при кожній дії користувача. Довідкові дані (категорії) змінюються рідко. Курси та уроки створюються та редагуються викладачами на регулярній основі.

Основні типи запитів, що формуються інтерфейсом:

- Каталог курсів – вибірка опублікованих курсів із фільтрацією за категоріями, ціною, рівнем складності та рейтингом, повнотекстовим пошуком за назвою, сортуванням за різними критеріями та пагінацією;
- Аналітика курсу – агрегація кількості записів, загального доходу, середньої оцінки та відсотку завершення по конкретному курсу, а також часовий ряд реєстрацій за обраний період;
- Фінансова панель – агрегація оплат і виплат за довільний часовий діапазон з обчисленням підсумкових показників;
- Рекомендації – складний аналітичний запит із тристороннім самооб'єднанням таблиці записів для виявлення шаблонів спільних реєстрацій.

Доступ до даних обмежується на рівні серверного API відповідно до ролі автентифікованого користувача:

- Студент має доступ лише до власних записів, прогресу та відгуків, а також до каталогу опублікованих курсів.
- Викладач має доступ лише до власних курсів, уроків, аналітики та виплат.
- Адміністратор має доступ до фінансових даних усіх користувачів та управління їх статусом.

Неавтентифіковані запити отримують відповідь із кодом 401 (Unauthorized), а запити з недостатніми правами – 403 (Forbidden).

2.3. Логічне проектування бази даних (домени, ключі, таблиці, обмеження)

Схема бази даних складається з 11 таблиць. Усі назви таблиць та стовпців використовують формат `snake_case`, що забезпечується бібліотекою `EFCore.NamingConventions`. Нижче подано опис кожної таблиці.

`users` – таблиця облікових записів усіх користувачів системи. Роль та статус зберігаються як текстові стовпці з обмеженнями `CHECK` на допустимі значення, що спрощує еволюцію схеми порівняно з типами `ENUM`. Електронна пошта має обмеження `UNIQUE` для запобігання дублікатів.

Атрибути:

- `id` (integer, PK, автоінкремент) – унікальний ідентифікатор користувача;
- `first_name` (varchar(100), NOT NULL) – ім'я користувача;
- `last_name` (varchar(100), NOT NULL) – прізвище користувача;
- `email` (varchar(255), NOT NULL, UNIQUE) – адреса електронної пошти, використовується для входу в систему;
- `password_hash` (text, NOT NULL) – хеш пароля, створений алгоритмом PBKDF2;

- role (varchar(20), NOT NULL, CHECK IN ('Admin', 'Teacher', 'Student')) – роль користувача в системі, визначає доступні функції;
- status (varchar(20), NOT NULL, DEFAULT 'Active', CHECK IN ('Active', 'Suspended')) – статус облікового запису, призупинені користувачі не можуть увійти в систему.

teacher_profiles – розширення для користувачів із роллю «Викладач». Зв'язок 1:1 із таблицею users через первинний ключ user_id, що одночасно є зовнішнім ключем. При видаленні користувача профіль видаляється каскадно (ON DELETE CASCADE). Поле balance є похідним значенням, що підтримується тригерами.

Атрибути:

- user_id (integer, PK, FK – users(id), ON DELETE CASCADE) – ідентифікатор користувача-викладача;
- bio (varchar(2000)) – текстова біографія викладача, відображається на публічному профілі;
- balance (numeric(10,2), DEFAULT 0) – поточний баланс нарахованих коштів, автоматично перераховується тригерами при зміні оплат або виплат;

teacher_profiles – розширення для користувачів із роллю «Викладач». Зв'язок 1:1 із таблицею users через первинний ключ user_id, що одночасно є зовнішнім ключем. При видаленні користувача профіль видаляється каскадно (ON DELETE CASCADE). Поле balance є похідним значенням, що підтримується тригерами.

Атрибути:

- user_id (integer, PK, FK – users(id), ON DELETE CASCADE) – ідентифікатор користувача-викладача;
- bio (varchar(2000)) – текстова біографія викладача, відображається на публічному профілі;
- balance (numeric(10,2), DEFAULT 0) – поточний баланс нарахованих коштів, автоматично перераховується тригерами при зміні оплат або виплат;

categories – довідник тематичних категорій курсів.

Атрибути:

- id (integer, PK, автоінкремент) – унікальний ідентифікатор категорії;
- name (varchar(60), NOT NULL, UNIQUE) – назва категорії, наприклад “Web Development” або “Business Finance”;

courses – навчальні курси, створені викладачами. Зовнішній ключ author_id посиляється на teacher_profiles(user_id), а не на users(id), що структурно гарантує: автором курсу може бути лише користувач із профілем викладача. Видалення автора заборонено при наявності курсів (ON DELETE RESTRICT). Ціна обмежена знизу нулем. Статус та рівень складності контролюються обмеженнями CHECK.

Атрибути:

- id (integer, PK, автоінкремент) – унікальний ідентифікатор курсу;
- author_id (integer, FK – teacher_profiles(user_id), ON DELETE RESTRICT) – ідентифікатор викладача-автора курсу;
- title (varchar(200), NOT NULL) – назва курсу, відображається в каталозі та на сторінці деталей;
- description (text, NOT NULL) – розгорнутий опис курсу;
- price (numeric(10,2), CHECK >= 0) – вартість курсу в доларах, 0 для безкоштовних курсів;
- level (varchar(20), NOT NULL, CHECK IN ('Beginner', 'Intermediate', 'Advanced')) – рівень складності курсу;
- status (varchar(20), NOT NULL, CHECK IN ('Draft', 'Published')) – стан публікації: чернетка або опублікований;
- created_at (timestamp, DEFAULT now()) – дата та час створення курсу.

course_categories – проміжна таблиця для зв'язку M:N між курсами та категоріями. Композитний первинний ключ унеможливорює повторне призначення категорії до курсу.

Атрибути:

- courses_id (integer, PK (частина), FK → courses(id)) – ідентифікатор курсу;
- categories_id (integer, PK (частина), FK → categories(id)) – ідентифікатор категорії;

lessons – уроки в межах курсу. Комбінація (course_id, order_number) має обмеження UNIQUE для гарантування унікальності порядку уроків у межах курсу. При видаленні курсу уроки видаляються каскадно (ON DELETE CASCADE).

Атрибути:

- id (integer, PK, автоінкремент) – унікальний ідентифікатор уроку;
- course_id (integer, FK – courses(id), ON DELETE CASCADE) – ідентифікатор курсу, до якого належить урок;
- title (varchar(200), NOT NULL) – назва уроку;
- content (text, NOT NULL) – текстовий навчальний матеріал уроку;

lesson_progress – відстеження завершення окремих уроків. Композитний первинний ключ (enrollment_id, lesson_id) гарантує, що кожен урок може бути позначений як завершений лише один раз у межах одного запису. При видаленні запису прогрес видаляється каскадно; видалення уроку при наявності записів прогресу заборонено.

Атрибути:

- enrollment_id (integer, PK (частина), FK – enrollments(id), ON DELETE CASCADE) – ідентифікатор запису на курс;
- lesson_id (integer, PK (частина), FK – lessons(id), ON DELETE RESTRICT) – ідентифікатор завершеного уроку;

- `completed_at` (timestamp, DEFAULT now()) – дата та час позначення уроку як завершеного;

`reviews` – відгуки студентів на курси. Первинний ключ `enrollment_id` одночасно є зовнішнім ключем до таблиці `enrollments`, що забезпечує два структурних обмеження: один відгук на один запис та неможливість залишити відгук без попереднього запису на курс. При видаленні запису відгук видаляється каскадно.

Атрибути:

- `enrollment_id` (integer, PK, FK – `enrollments(id)`, ON DELETE CASCADE) – ідентифікатор запису на курс, що одночасно є первинним ключем відгуку;
- `grade` (integer, CHECK BETWEEN 1 AND 5) – числова оцінка курсу від 1 до 5;
- `created_at` (timestamp, DEFAULT now()) – дата та час створення відгуку;

`payments` – фінансові транзакції при записі на курс. Видалення пов'язаного студента або курсу заборонено (ON DELETE RESTRICT). Сума обмежена знизу нулем, статус – обмеженням CHECK.

Атрибути:

- `id` (integer, PK, автоінкремент) – унікальний ідентифікатор оплати;
- `student_id` (integer, FK – `users(id)`, ON DELETE RESTRICT) – ідентифікатор студента, що здійснив оплату;
- `course_id` (integer, FK – `courses(id)`, ON DELETE RESTRICT) – ідентифікатор оплаченого курсу;
- `amount` (numeric(10,2), CHECK >= 0) – сума оплати в доларах;
- `status` (varchar(20), NOT NULL, CHECK IN ('Completed', 'Refunded')) – статус транзакції: завершена або повернена;
- `created_at` (timestamp, DEFAULT now()) – дата та час здійснення оплати;

payouts – запити викладачів на виплату коштів. Статус відображає автомат станів: Pending → Approved → Paid або Pending → Rejected. Видалення профілю викладача заборонено при наявності виплат (ON DELETE RESTRICT)..

Атрибути:

- id (integer, PK, автоінкремент) – унікальний ідентифікатор виплати;
- teacher_id (integer, FK – teacher_profiles(user_id), ON DELETE RESTRICT) – ідентифікатор викладача, що подав запит;
- amount (numeric(10,2), CHECK >= 0) – запитувана сума виплати в доларах;
- status (varchar(20), NOT NULL, CHECK IN ('Pending', 'Approved', 'Rejected', 'Paid')) – поточний статус запиту на виплату;
- requested_at (timestamp, DEFAULT now()) – дата та час подання запиту;

Спроектowana схема відповідає третій нормальній формі (3НФ):

- 1НФ – кожен стовпець містить атомарні значення; зв'язок М:Н між курсами та категоріями реалізовано через окрему проміжну таблицю course_categories, а не через масив значень;
- 2НФ – усі неключові атрибути повністю функціонально залежать від первинного ключа. У таблицях із композитним ключем (lesson_progress, course_categories) немає часткових залежностей;
- 3НФ – відсутні транзитивні залежності. Зокрема, поле balance у таблиці teacher_profiles та поле progress у таблиці enrollments є обчислюваними значеннями, проте вони підтримуються тригерами для оптимізації продуктивності запитів і завжди узгоджене з базовими таблицями payments, payouts та lessons_progress.

Окрім неявних індексів, створених для первинних ключів та обмежень UNIQUE, визначено 5 явних індексів для оптимізації типових запитів. Кожен індекс відповідає стовпцю зовнішнього ключа, за яким часто виконуються запити фільтрації через інтерфейс користувача:

- `idx_courses_author_id` – `courses(author_id)`: вибірка курсів конкретного викладача;
- `idx_enrollments_student_id` – `enrollments(student_id)`: вибірка записів конкретного студента;
- `idx_enrollments_course_id` – `enrollments(course_id)`: аналітика курсу (підрахунок студентів);
- `idx_payments_course_id` – `payments(course_id)`: аналітика курсу (обчислення доходу);
- `idx_payouts_teacher_id` – `payouts(teacher_id)`: перегляд виплат конкретного викладача;

2.4. Реалізація процедур бізнес-логіки (тригери, процедури, представлення)

На рівні бази даних реалізовано три тригери та дві допоміжні функції мовою PL/pgSQL, що забезпечують автоматичне підтримання цілісності обчислюваних значень. ДЖЕРЕЛО

Перерахунок прогресу навчання. Тригер `trg_lesson_progress_recompute` спрацьовує після вставки або видалення запису в таблиці `lesson_progress` (подія `AFTER INSERT OR DELETE`) та викликає функцію `recompute_enrollment_progress`. Алгоритм: визначається загальна кількість уроків курсу та кількість завершених уроків для конкретного запису, після чого прогрес обчислюється як відсоток завершених уроків від загальної кількості (ціле число від 0 до 100).

Перерахунок балансу при оплаті. Тригер `trg_payments_balance` спрацьовує після вставки нового запису або зміни стовпця `status` у таблиці `payments` (подія `AFTER INSERT OR UPDATE OF status`) та викликає функцію `trg_payments_recompute_balance`. Функція знаходить автора відповідного курсу та делегує обчислення спільній допоміжній функції `recompute_teacher_balance`.

Перерахунок балансу при виплаті. Тригер `trg_payouts_balance` спрацьовує після вставки або зміни стовпця `status` у таблиці `payouts` (подія `AFTER INSERT OR`

UPDATE OF status) та викликає функцію `trg_payouts_recompute_balance`, яка делегує обчислення тій самій допоміжній функції `recompute_teacher_balance`.

Допоміжна функція `recompute_teacher_balance`. Обидва фінансових тригери делегують обчислення спільній функції, що приймає ідентифікатор викладача та перераховує його баланс як різницю сум завершених оплат за курси та невідхиленних виплат викладачу. Функція повністю перераховує баланс із нуля при кожному виклику, що гарантує узгодженість навіть після ручних корекцій в базі даних

3. РОЗРОБКА ПРОГРАМНОГО КОДУ

3.1. Обґрунтування обраної архітектури (тип, мова програмування, система баз даних, протоколи обміну)

Систему побудовано за трирівневою клієнт-серверною архітектурою:

- Рівень представлення (клієнт) – односторінковий вебзастосунок (SPA – Single Page Application), що виконується у браузері користувача;
- Рівень бізнес-логіки (сервер) – REST API, що обробляє запити клієнта, реалізує правила авторизації та взаємодіє з базою даних;
- Рівень даних – реляційна СУБД, що зберігає усі дані системи та виконує тригери і збережені функції.

Обрані технології:

- Мова програмування сервера: C# (.NET 9). Обрано за типізацію, вбудовану підтримку ORM (Object-Relational Mapping) та зручну модель контролерів для побудови REST API;
- Серверний фреймворк: ASP.NET Core 9 Web API. Забезпечує маршрутизацію HTTP-запитів, автентифікацію на основі файлів cookie, авторизацію за ролями та серіалізацію відповідей у формат JSON;
- СУБД: PostgreSQL. Обрано через наявний досвід у роботі з цією СУБД, підтримку PL/pgSQL для тригерів та збережених функцій, відповідність стандарту SQL, а також наявність зрілого ORM-провайдера для .NET (Npgsql);
- ORM: Entity Framework Core 9 із бібліотекою EFCore.NamingConventions для автоматичного перетворення назв таблиць та стовпців у формат snake_case;
- Мова програмування клієнта: TypeScript. Забезпечує типізацію на рівні клієнтського коду та зменшує кількість помилок під час розробки;

- Клієнтський фреймворк: React 18 із бібліотекою React Router v7 для маршрутизації, TanStack Query для управління серверним станом та кешування, Tailwind CSS v4 для стилізації та shadcn/ui для базових компонентів інтерфейсу;
- Протокол обміну: HTTP (REST). Клієнт надсилає запити до API серверу, передаючи дані у форматі JSON. Автентифікація здійснюється через HTTP cookie, що автоматично включається у кожен запит.

3.2. Структурна модель інформаційної системи

Система складається з двох незалежних компонентів, що взаємодіють через HTTP: ДЖЕРЕЛО

Серверний застосунок – проект ASP.NET Core Web API. Обрано плоску структуру (без поділу на Application/Domain/Infrastructure шари). Основні каталоги:

- Controllers/ – контролери REST API, згруповані за функціональністю;
- Models/ – класи сутностей, що відображаються на таблиці бази даних;
- Data/ – контекст бази даних (AppDbContext) із конфігурацією зв'язків та обмежень;
- Migrations/ – міграції Entity Framework Core, включаючи SQL-код тригерів;
- Dtos/ – класи об'єктів передачі даних (DTO — Data Transfer Object) для запитів та відповідей API.

Клієнтський застосунок – React SPA. Основні каталоги:

- src/app/pages/ – сторінки, згруповані за ролями (студент, викладач, адміністратор);
- src/app/components/ – спільні компоненти (Layout, Pagination, DateRangePicker);
- src/lib/ – утилітарні модулі (api.ts – HTTP-клієнт, auth.tsx – контекст автентифікації).

Програма-завантажувач – окремий консольний проект .NET, що посилається на основний API-проект для використання спільних моделей та контексту бази даних.

3.3. Призначення модулів та компонентів системи

Серверні контролери. Контролери є точками входу для HTTP-запитів. Кожен контролер захищений атрибутом авторизації, що обмежує доступ відповідною роллю:

- **AuthController** – реєстрація нових користувачів (студентів та викладачів), вхід у систему та вихід. При реєстрації пароль хешується за допомогою **PasswordHasher**, при вході – перевіряється хеш та створюється автентифікаційний **cookie**.
- **MeController** – отримання інформації про поточного користувача. Додатково перевіряє статус облікового запису та примусово завершує сесію для призупинених користувачів.
- **CoursesController** – публічний каталог курсів. Підтримує фільтрацію за категоріями, ціною, рівнем складності та рейтингом, пошук за назвою, сортування та пагінацію. Також надає детальну інформацію про курс, включаючи список уроків та відгуки.
- **EnrollmentsController** – запис студента на курс (із створенням оплати в одній транзакції), перегляд власних записів, перегляд уроків, позначення уроків як завершених та скасування завершення.
- **ReviewsController** – створення, редагування та видалення відгуків студентами.
- **RecommendationsController** – отримання персоналізованих рекомендацій для поточного студента. Викликає збережену функцію **PostgreSQL get_recommendations**.

- CategoriesController – перегляд списку усіх категорій (використовується для фільтрів каталогу та редактора курсів).
- AuthorsController – перегляд публічного профілю викладача та його опублікованих курсів.
- TeacherCoursesController – CRUD-операції над курсами поточного викладача: створення чернетки, редагування, публікація, зняття з публікації, видалення (лише для курсів без записів).
- TeacherLessonsController – CRUD-операції над уроками: створення, редагування, видалення, зміна порядку (з використанням двопрхідного алгоритму для уникнення порушення обмеження UNIQUE).
- TeacherAnalyticsController – аналітика курсу (кількість студентів, дохід, середній рейтинг, відсоток завершення, нещодавні студенти) та часовий ряд записів.
- TeacherReviewsController – перегляд відгуків на курси поточного викладача з фільтрацією за курсом.
- TeacherEarningsController – перегляд балансу та історії виплат, створення запиту на виплату.
- TeacherProfileController – перегляд та редагування біографії.
- AdminUsersController – перегляд користувачів із фільтрацією за роллю, статусом та пошуком. Призупинення та відновлення облікових записів (із заборною призупинення власного облікового запису та інших адміністраторів).
- AdminFinanceController – фінансова панель із агрегованими показниками за обраний період, перегляд оплат із фільтрацією, оформлення повернення (в одній транзакції із видаленням запису на курс та каскадним видаленням прогресу й відгуку).
- AdminPayoutsController – перегляд виплат, зміна статусу (схвалення, відхилення, позначення виплаченим) із валідацією автомата станів.

Клієнтські утилітарні модулі:

- `api.ts` – централізований HTTP-клієнт. Усі запити до сервера проходять через єдину функцію-обгортку, що додає заголовок `Content-Type` та `cookie` автентифікації (`credentials: 'include'`). При отриманні відповіді 401 або 403 генерується подія для оновлення стану автентифікації.
- `auth.tsx` – контекст автентифікації (`AuthProvider + useAuth`). Зберігає поточного користувача, надає методи `login/register/logout`. Періодично (кожні 60 секунд) перевіряє стан сесії на сервері, що забезпечує синхронізацію між вкладками браузера та автоматичний вихід при призупиненні облікового запису.
- `RequireAuth` – компонент-обгортка для захисту маршрутів. Перенаправляє неавтентифікованих користувачів на сторінку входу та обмежує доступ за ролями.

3.4. Опис реалізації функцій системи

Автентифікація та авторизація. ДЖЕРЕЛО

Автентифікація реалізована на основі файлів `cookie` (`cookie-based authentication`) без використання `ASP.NET Identity`. Рішення обумовлене обмеженнями курсової роботи на кількість таблиць та стовпців – `ASP.NET Identity` додає 7 таблиць із понад 13 стовпцями до схеми користувачів.

При реєстрації пароль хешується за допомогою стандартного класу `PasswordHasher<User>`. При вході система перевіряє хеш, статус облікового запису та створює автентифікаційний `cookie` (ім'я `lp.auth`, термін дії 7 днів, `HttpOnly`, `SameSite=Lax`). API повертає коди 401 та 403 замість стандартних перенаправлень `ASP.NET`, оскільки клієнтом є SPA.

Запис на курс та обробка оплати.

Запис студента на курс відбувається в одній транзакції бази даних: створюється запис у таблиці `enrollments` та відповідний запис у таблиці `payments` зі

статусом “Completed”. Тригер `trg_payments_balance` автоматично оновлює баланс викладача. Повторний запис на той самий курс блокується обмеженням `UNIQUE` на рівні бази даних.

Відстеження прогресу.

При позначенні уроку як завершеного створюється запис у таблиці `lesson_progress`. Тригер `trg_lesson_progress_recompute` автоматично перераховує поле `progress` у відповідному записі `enrollments`. При скасуванні завершення запис видаляється, тригер перераховує прогрес у зворотному напрямку.

Повернення коштів.

Адміністратор може оформити повернення коштів за оплату. Операція виконується в одній транзакції: статус оплати змінюється на “Refunded”, а відповідний запис на курс видаляється. Каскадне видалення автоматично видаляє пов'язані записи прогресу уроків та відгук. Тригер `trg_payments_balance` перераховує баланс викладача.

Пошук та фільтрація.

Каталог курсів підтримує комбіновану фільтрацію за кількома критеріями одночасно: категорії (множинний вибір), діапазон ціни, рівень складності, мінімальний рейтинг та текстовий пошук за назвою. Результати підтримують сортування (за датою, ціною, рейтингом, популярністю) та пагінацію.

3.5. Опис реалізації елементів інтерфейсу

Інтерфейс побудовано як односторінковий застосунок (SPA) з бічною навігаційною панеллю та верхнім заголовком. Бічна панель адаптується залежно від ролі поточного користувача, відображаючи відповідний набір пунктів меню.

ДЖЕРЕЛО

Сторінки студента.

Панель навчання (My Learning) – відображає список курсів, на які записаний студент, з індикатором прогресу для кожного (рис. 3.2). Нижче – блок рекомендованих курсів із можливістю перегляду пояснень алгоритму.

Каталог курсів (Catalog) – сітка карток курсів із панеллю фільтрів зверху (рис. 3.3). Фільтри включають: випадальний список категорій із пошуком, діапазон ціни, рівень складності, мінімальний рейтинг та критерій сортування. Параметри фільтрів зберігаються в URL-адресі, що дозволяє ділитися посиланнями та зберігати стан при навігації. Для курсів, на які студент вже записаний, картка відображає відсоток прогресу.

Деталі курсу (Course Details) – повна інформація про курс: назва, опис, ціна, рівень, категорії, автор (з посиланням на профіль), список уроків, відгуки інших студентів та середня оцінка (рис. 3.4). Для незаписаних студентів відображається кнопка «Записатись», для записаних — кнопка «Продовжити навчання» та секція відгуку.

Перегляд уроку (Lesson Player) – текстовий вміст уроку з бічною панеллю навігації по курсу, що відображає статус кожного уроку – завершений чи ні (рис. 3.5). Внизу – кнопки «Позначити як завершений» / «Скасувати завершення» та навігація «Попередній / Наступний урок».

Сторінки викладача.

Панель курсів (My Courses) – сітка карток курсів поточного викладача з індикаторами статусу (чернетка/опубліковано), кількістю студентів та середньою оцінкою (рис. 3.6). Кнопка створення нового курсу.

Редактор курсу (Course Editor) – форма редагування метаданих курсу (назва, опис, ціна, рівень, категорії) та управління уроками: створення, редагування, видалення, зміна порядку кнопками вгору/вниз (рис. 3.7). Кнопка публікації доступна лише для курсів із щонайменше одним уроком. Секція видалення курсу відображається лише для курсів без записів.

Аналітика курсу (Course Analytics) – чотири KPI-картки (записані студенти, дохід, середня оцінка, відсоток завершення), лінійний графік динаміки записів за останні 30 днів та таблиця нещодавніх студентів (рис. 3.8).

Відгуки студентів (Student Reviews) – список відгуків із можливістю фільтрації за конкретним курсом (рис. 3.9).

Заробіток та виплати (Earnings & Payouts) – поточний баланс, форма запиту на виплату (з валідацією суми) та таблиця історії виплат зі статусами (рис. 3.10).

Сторінки адміністратора.

Фінансова панель (Financial Dashboard) – вибір діапазону дат (з попередніми налаштуваннями 7/30/90/365 днів), KPI-картки (валовий дохід, повернення, виплати викладачам, очікувані виплати), лінійний графік динаміки доходів та виплат, панель сповіщень (рис. 3.11).

Управління оплатами (Student Payments) – таблиця оплат із фільтрацією за статусом та пошуком за іменем студента або назвою курсу (рис. 3.12). Кнопка «Повернути кошти» для завершених оплат із діалогом підтвердження.

Управління виплатами (Teacher Payouts) – таблиця запитів на виплату із фільтрацією за статусом та пошуком (рис. 3.13). Кнопки зміни статусу відповідно до автомата станів (Pending → Approve/Reject, Approved → Mark Paid).

Управління користувачами (User Management) – таблиця користувачів із фільтрацією за роллю, статусом та пошуком (рис. 3.14). Кнопка призупинення/відновлення (неактивна для поточного адміністратора та інших адміністраторів).

4. ОПРАЦЮВАННЯ ДАНИХ

4.1. Характеристика первинного джерела даних (структура, обсяг, оцінка об'єктивності даних)

Для початкового завантаження даних використано відкритий набір даних “Udemy Courses” з платформи Kaggle. Набір містить інформацію про реальні курси, опубліковані на платформі Udemy. ДЖЕРЕЛО

Формат: CSV-файл (courses.csv), кодування UTF-8. Обсяг: 3 683 записи (після фільтрації — близько 3 600 курсів).

CSV-файл містить 12 стовпців. Для завантаження використовуються 6 з них:

- course_title – назва курсу (→ courses.title);
- price – ціна курсу (→ courses.price);
- num_subscribers – кількість підписників (для зваження ймовірності запису);
- num_lectures – кількість лекцій (→ кількість уроків, обмежена до 8);
- level – рівень складності (→ courses.level, з маппінгом значень);
- published_timestamp – дата публікації у форматі ISO 8601 (→ courses.created_at);
- subject — предмет/тема (→ categories.name)

4.2. Алгоритм трансформації даних при завантаженні

Завантаження первинних даних реалізовано окремим консольним застосунком (LearningPlatform.DataLoader), що посилається на основний API-проект для використання спільних моделей та контексту бази даних.

Алгоритм обробки CSV-файлу:

- Зчитування та фільтрація. Файл зчитується порядково. Записи з назвами, що містять символи поза діапазоном ASCII (коди 32–126), відкидаються. Назви обрізаються до 200 символів відповідно до обмеження бази даних;

- Вилучення категорій. Зі стовпця `subject` вилучаються унікальні назви категорій. Для кожної створюється запис у таблиці `categories`;
- Маппінг рівнів складності. Значення рівнів з CSV перетворюються на допустимі значення системи: «Beginner Level» → «Beginner», «Intermediate Level» → «Intermediate», «Expert Level» → «Advanced», усі інші → «Beginner»;
- Маппінг кількості уроків. Значення `num_lectures` обмежується зверху до 8 (максимальна кількість уроків на курс у системі). Мінімум – 1 урок;
- Прив'язка до авторів. 90% курсів призначаються викладачу, що спеціалізується на категорії курсу. 10% – випадковому викладачу, що імітує викладання суміжних дисциплін;
- Парсинг дати. Значення `published_timestamp` перетворюється з ISO 8601 на тип `DateTime` із явним зазначенням часового поясу UTC (вимога драйвера `Npgsql`).

4.3. Алгоритм генерації тестових даних

Для заповнення таблиць, що не мають прямого відповідника в CSV-файлі, використовується генерація за правилами. Генератор використовує фіксоване початкове значення (`seed = 42`) для детермінованого відтворення результатів.

Пропорції генерації. Кількість згенерованих записів обчислюється пропорційно до кількості курсів:

- викладачі: $\max(50, \text{кількість_курсів} / 6)$ — в середньому 6 курсів на викладача;
- студенти: $\max(100, \text{кількість_курсів} / 2)$;
- записи на курси: $\min(\text{кількість_курсів} \times 5, \text{кількість_студентів} \times 30)$ — в середньому 5 записів на курс.

Генерація користувачів. Імена генеруються з набору (120+ імен та 90+ прізвищ). Електронні адреси формуються за шаблоном `firstname.lastname{seq}@local.example`. Усі згенеровані користувачі мають однаковий тестовий пароль.

Генерація записів на курси. Кожному студенту призначається 1-2 улюблені категорії. При генерації записів 70% записів спрямовуються на курси улюблених категорій, 30% – на випадкові курси. Дата запису генерується випадково у межах від дати публікації курсу до зазначеної кінцевої дати.

Генерація оплат. Для кожного запису на курс створюється відповідна оплата зі статусом «Completed» та сумою, що дорівнює ціні курсу. Тригер `trg_payments_balance` автоматично оновлює баланси викладачів.

Генерація прогресу. Для кожного запису випадково визначається частка завершених уроків (від 0 до 100%). Для кожного завершеного уроку створюється запис у таблиці `lesson_progress` із випадковою датою завершення. Тригер `trg_lesson_progress_recompute` автоматично перераховує прогрес.

Генерація відгуків. 80% записів на курси отримують відгук. Розподіл оцінок зміщений у бік високих значень: понад 55% відгуків мають оцінку 5, менше 10% – оцінки 1-2. Текст коментаря обирається випадково з набору 12 шаблонних відгуків.

Генерація виплат. Для кожного викладача генерується 0-3 запити на виплату. Сума – від 10 до 60% балансу, обмежена зверху 200 доларів. Статус обирається випадково (Pending, Approved, Paid, Rejected).

Режими роботи. Програма-завантажувач підтримує кілька режимів запуску:

- без параметрів – повне завантаження з CSV-файлу (~3 600 курсів, ~50 000 записів);
- `--limit N` – завантаження лише перших N курсів із CSV;
- `--reset` – автоматичне очищення даних перед завантаженням (без підтвердження).

4.4. Опис додаткових методів обробки даних

Автоматичне обчислення похідних значень. Тригери бази даних забезпечують автоматичне обчислення двох похідних значень:

- прогрес навчання (enrollments.progress) – перераховується при кожному додаванні або видаленні запису в таблиці lesson_progress;
- баланс викладача (teacher_profiles.balance) – перераховується при кожній зміні статусу оплати або виплати.

Використання тригерів замість обчислення на рівні прикладного коду гарантує узгодженість значень незалежно від способу модифікації базових таблиць.

Основним додатковим методом обробки даних є алгоритм підбору рекомендованих курсів, реалізований як збережена функція PostgreSQL. ДЖЕРЕЛО

Результати роботи алгоритму обчислюються при кожному запиті студента, а не зберігаються в базі даних. Це забезпечує актуальність рекомендацій без потреби в окремому процесі оновлення, хоча для великих обсягів даних може бути доцільним кешування результатів.

Для реалізації додаткової функції підбору рекомендованих курсів обрано метод видобування асоціативних правил (association rule mining), також відомий як аналіз ринкового кошика (market basket analysis). Цей метод аналізу даних дозволяє виявляти закономірності в поведінці користувачів: які курси часто обираються разом.

Асоціативне правило має форму $\{A\} \rightarrow \{B\}$, що має умовне значення: «студенти, які записались на курс А, також схильні записуватись на курс В». Кожне правило оцінюється за трьома стандартними метриками.

- Підтримка (support) – частка студентів, записаних одночасно на обидва курси, серед усіх студентів:

$$support(A \rightarrow B) = \frac{N_A + N_B}{N_{All}}, \quad (4.1)$$

де N_A – кількість записаних студентів на курс А, N_B – кількість записаних студентів на курс В, N_{All} – кількість усіх студентів.

Вища підтримка означає, що закономірність є більш поширеною і, відповідно, надійнішою.

- Впевненість (confidence) – умовна ймовірність того, що студент, записаний на курс А, також записаний на курс В:

$$confidence(A \rightarrow B) = \frac{N_{AB}}{N_A}, \quad (4.2)$$

де N_{AB} – кількість записаних студентів на курси А та В одночасно.

Впевненість відображає передбачувану силу правила.

- Підйом (lift) – відношення впевненості до безумовної ймовірності запису на курс В:

$$lift(A \rightarrow B) = \frac{confidence(A \rightarrow B)}{P(B)} \quad (4.3)$$

$$P(B) = \frac{N_B}{N_{All}}, \quad (4.4)$$

Підйом показує, наскільки асоціація є сильнішою за випадковий збіг:

- $lift > 1$ – зв'язок сильніший за випадковий, реальна закономірність;
- $lift = 1$ – зв'язку немає, курси обираються незалежно;
- $lift < 1$ – від'ємна асоціація, курси обираються рідше, ніж очікувалось, «антирекомендація».

Формула оцінки. Для кожного курсу-кандидата обчислюється композитна оцінка, що поєднує п'ять параметрів:

$$score = \Sigma (confidence \times lift \times completion_weight) \times rating_multiplier \times category_multiplier \quad (4.5)$$

Параметр «Асоціативні правила» ($confidence \times lift$). Для кожного курсу, на який записаний поточний студент, алгоритм знаходить шаблони спільних записів з іншими студентами. Множення впевненості та підйому забезпечує пріоритет

правил, які є одночасно передбачувальними (висока впевненість) та нетривіальними (високий підйом).

Параметр «Вага завершення» (*completion_weight*). Не всі однокурсники є рівноцінними сигналами. Студент, який завершив спільний курс, є сильнішим індикатором, ніж той, хто записався, але не розпочав навчання:

$$completion_weight = 0.5 + 0.5 \times \frac{peer_progress}{100} \quad (4.6)$$

Мінімальна вага 0.5 означає, що сам факт запису є значущим сигналом – студент свідомо обрав та оплатив курс. Завершення посилює цей сигнал, але не є обов'язковою умовою.

Параметр «Якість рейтингу» (*rating_multiplier*). Курси з вищим середнім рейтингом отримують перевагу:

$$rating_multiplier = \frac{COALESCE(avg_rating, 3.0)}{5.0}, \quad (4.7)$$

де *avg_rating* – середній рейтинг курсу.

Курси без відгуків отримують нейтральне значення 3.0 з 5.0, щоб уникнути неоптимального штрафування нових курсів.

Параметр «Афінність категорій» (*category_multiplier*). Якщо рекомендований курс має спільні категорії з курсами, на які вже записаний студент, він отримує невелику надбавку:

$$category_multiplier = 1 + 0.2 \times N_{shared_categories}, \quad (4.8)$$

де $N_{shared_categories}$ – кількість спільних категорій у курсу та у студента.

Коефіцієнт 0.2 обрано навмисно низьким, адже кількість спільних категорій не є основним фактором підбору рекомендацій.

Параметр «Обробка холодного старту». Для студентів із нульовою кількістю записів асоціативні правила не можуть бути застосовані. У такому випадку алгоритм повертає найпопулярніші курси за кількістю записів. Як тільки студент записується на перший курс, рекомендації стають персоналізованими.

Алгоритм реалізовано як збережену функцію PostgreSQL `get_recommendations(target_student_id, max_results)`, що повертає набір записів

(course_id, score, confidence, lift, reason). Функція використовує 9 послідовних загальних табличних виразів (СТЕ — Common Table Expression): ДЖЕРЕЛО

- my_courses – курси поточного студента;
- my_categories – категорії цих курсів;
- course_enrollment_counts – кількість записів на кожен курс (знаменники для confidence та lift);
- peer_enrollments – тристороннє самооб'єднання таблиці enrollments: курси студента → однокурсники → інші курси однокурсників;
- rule_metrics – confidence, lift та зважена кількість однокурсників для кожного правила;
- best_rule – найсильніше правило для кожного рекомендованого курсу (для тексту пояснення);
- avg_ratings – середній рейтинг кожного курсу;
- cat_overlap – кількість спільних категорій;
- scored – фінальна композитна оцінка.

Кожна рекомендація супроводжується текстовим поясненням, наприклад: «47% студентів, які проходили "React for Beginners", також обрали цей курс (1.8x вище середнього)».

5. ФУНКЦІОНАЛЬНІ МОЖЛИВОСТІ СИСТЕМИ

5.1. Інтерфейси управління даними

Інтерфейс системи адаптується залежно від ролі поточного користувача. Бічна навігаційна панель відображає набір пунктів меню, доступних для відповідної ролі, а заголовок панелі змінюється: «Student Portal», «Teacher Portal» або «Admin Portal».

Управління курсами (роль: викладач).

Викладач має повний набір операцій для управління навчальним контентом. Створення курсу розпочинається з форми, де вказуються назва, опис, ціна, рівень складності та категорії (множинний вибір із пошуком). Новий курс зберігається у стані чернетки.

Редактор курсу поєднує редагування метаданих курсу та управління уроками на одній сторінці (див. рис. 5.1 – 5.2). Для уроків доступні операції створення (вбудована форма), редагування, видалення та зміни порядку (кнопками вгору/вниз). Публікація курсу вимагає наявності щонайменше одного уроку. Видалення курсу можливе лише за відсутності записаних студентів.

← Back to My Courses

Edit Course Published

Course details

Title
Technical Analysis: Understanding Price Action

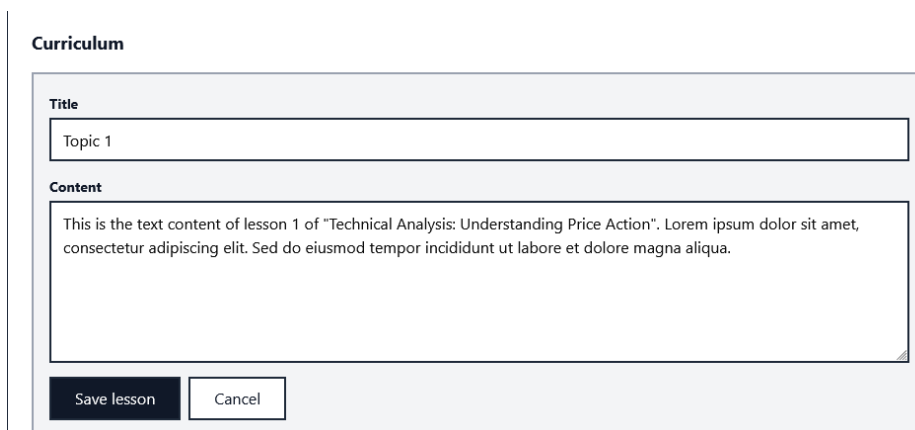
Description
Technical Analysis: Understanding Price Action.
Description for the course. Lorem Ipsum sit amet.

Price (\$) 45 **Level** Beginner

Categories
Business Finance

Save changes

Рис. 5.1. Форма редагування курсу з управлінням уроками

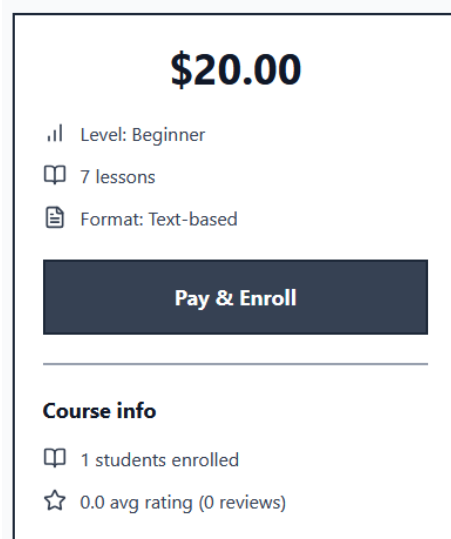


The screenshot shows a form titled "Curriculum" with two main sections: "Title" and "Content". The "Title" section has a text input field containing "Topic 1". The "Content" section has a larger text area containing placeholder text: "This is the text content of lesson 1 of 'Technical Analysis: Understanding Price Action'. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua." At the bottom of the form, there are two buttons: "Save lesson" (dark blue) and "Cancel" (light gray).

Рис. 5.2. Форма редагування уроків

Управління записами та прогресом (роль: студент)

Запис на курс здійснюється натисканням кнопки «Записатись» на сторінці деталей курсу(див. рис. 5.3). Після запису студент може переглядати уроки та позначати їх як завершені. Прогрес відображається у вигляді відсоткового індикатора на панелі навчання та на картках курсів у каталозі.

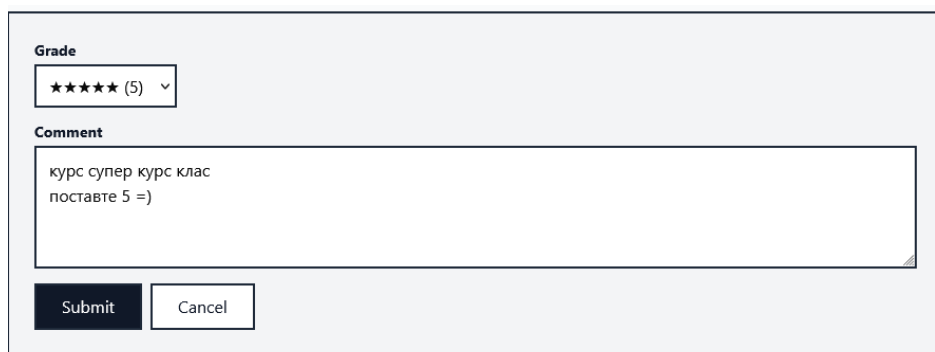


The screenshot shows a course card with the following information: A price tag of "\$20.00" in large bold text. Below it, three icons with text: a person icon for "Level: Beginner", a book icon for "7 lessons", and a document icon for "Format: Text-based". A large dark blue button labeled "Pay & Enroll" is centered below these details. A horizontal line separates this from the "Course info" section, which includes a book icon for "1 students enrolled" and a star icon for "0.0 avg rating (0 reviews)".

Рис. 5.3. Кнопка запису на курс

Управління відгуками (роль: студент)

Студент може залишити відгук на курс після запису. Форма відгуку містить вибір оцінки (від 1 до 5 зірок) та текстове поле для коментаря(див. рис. 5.4). Існуючий відгук можна редагувати або видалити. Обмеження: один студент – один відгук на курс.



Grade

★★★★★ (5) ▾

Comment

курс супер курс клас
поставте 5 =)

Submit Cancel

Рис. 5.4. Форма створення відгуку на курс

Управління користувачами (роль: адміністратор)

Адміністратор переглядає таблицю усіх користувачів із можливістю фільтрації за роллю (студент, викладач, адміністратор), статусом (активний, призупинений) та пошуком за іменем або електронною поштою (див. рис. 5.5). Для кожного користувача доступна операція призупинення або відновлення облікового запису. Призупинення власного облікового запису та облікових записів інших адміністраторів заблоковано на рівні інтерфейсу та серверної логіки.

User Management					
Role:	All ▾	Status:	All ▾	Search by name or email...	
User ID	Name	Email	Role	Status	Actions
U-0001	Admin User	admin@local	Admin	Active	Suspend
U-0002	Brooklyn Anderson	brooklyn.anderson0@local.example	Teacher	Active	Suspend
U-0003	Ava Turner	ava.turner0@local.example	Teacher	Active	Suspend

Рис. 5.5. Сторінка управління користувачами

Управління платежами (роль: адміністратор)

Таблиця оплат підтримує фільтрацію за статусом (завершена, повернена) та пошук за іменем студента або назвою курсу (див. рис. 5.6). Для завершених оплат доступна операція повернення коштів із діалогом підтвердження, що попереджає про видалення пов'язаного запису на курс, прогресу уроків та відгуку.

Student Payments & Refunds					
Status:	All	Search by student or course...			
Date	Student	Course	Amount	Status	Action
2026-04-25	Chloe Baker chloe.baker0@local.example	Cryptocurrency (BTC & ETH) Investment & Trading Course 2017	\$20.00	Completed	Issue Refund
2026-04-24	Aiden Cox aiden.cox0@local.example	Gibson's Learn & Master Guitar	\$50.00	Completed	Issue Refund

Рис. 5.6. Сторінка управління платежами

Управління виплатами

Викладач переглядає свій баланс та історію виплат. Форма запиту на виплату валідуює суму: мінімум \$1, максимум – поточний баланс.

Адміністратор переглядає усі запити на виплату з фільтрацією за статусом та пошуком за іменем або адресою викладача (рис. 5.4). Доступні дії відповідають автоматизованим станам: для запитів зі статусом “Pending” – кнопки “Approve” та “Reject”, для “Approved” – кнопка “Mark Paid”.

Teacher Payout Requests				
Status:	All	Search by teacher name or email...		
Date	Teacher	Amount	Status	Action
2026-04-25	Brooklyn Anderson brooklyn.anderson0@local.example	\$1234.00	Pending	Approve Reject

Рис. 5.7. Сторінка управління виплатами

5.2. Панель показників бізнес-процесу

Фінансова панель адміністратора

Головна сторінка адміністратора відображає фінансові показники за обраний часовий діапазон (див. рис. 5.8). Вибір діапазону здійснюється через два поля дати або кнопки попередніх налаштувань (7, 30, 90, 365 днів).

Показники подано у вигляді чотирьох KPI-карток:

- Валовий дохід – сума усіх завершених оплат за період;
- Повернення – сума повернутих коштів за період;
- Виплачено викладачам – сума виплат зі статусом «Paid» за період;
- Очікувані виплати – кількість та сума виплат зі статусом «Pending».

Панель сповіщень внизу інформує про наявність очікуючих запитів на виплату з посиланням на сторінку управління виплатами.

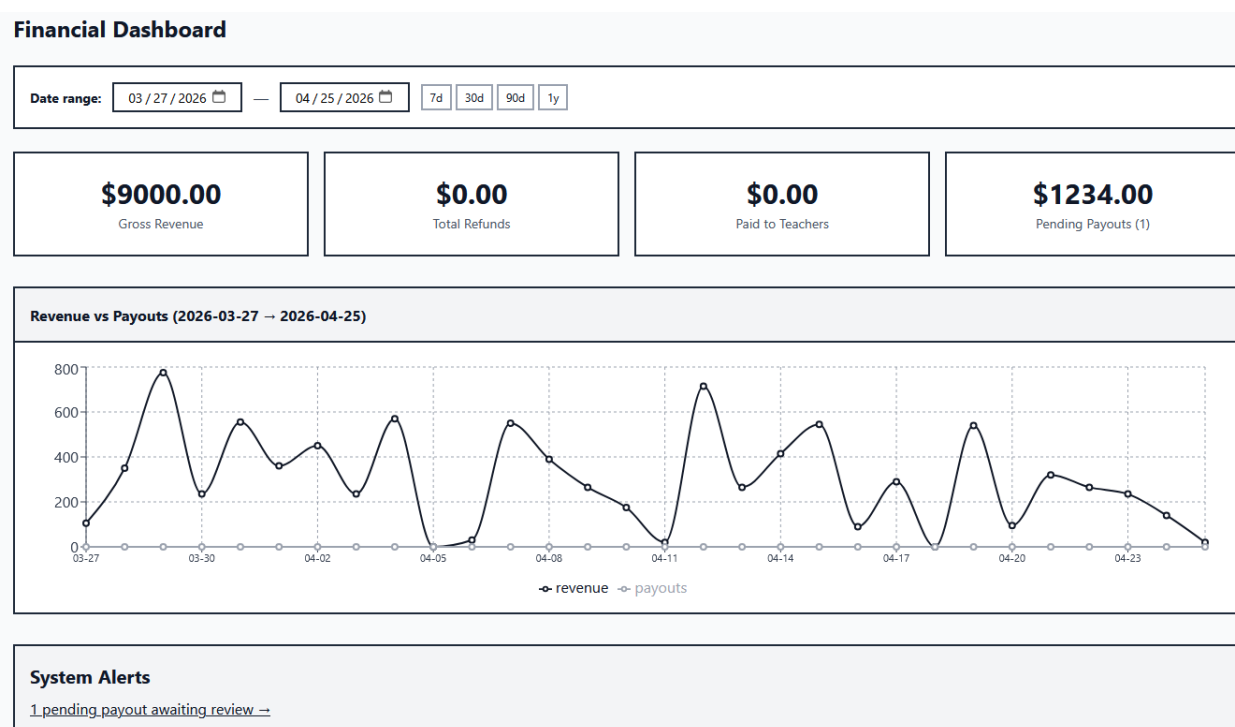


Рис. 5.8. Фінансова панель адміністратора

Аналітична панель курсу (викладач)

Для кожного курсу викладач може переглянути чотири КРІ-картки (див. рис. 5.9):

- Записані студенти – загальна кількість записів;
- Дохід – сума завершених оплат;
- Середня оцінка – середнє значення оцінок з відгуків;
- Відсоток завершення – середній прогрес усіх записаних студентів.

Нижче – таблиця нещодавніх студентів (останніх 10) із датою запису та поточним прогресом.

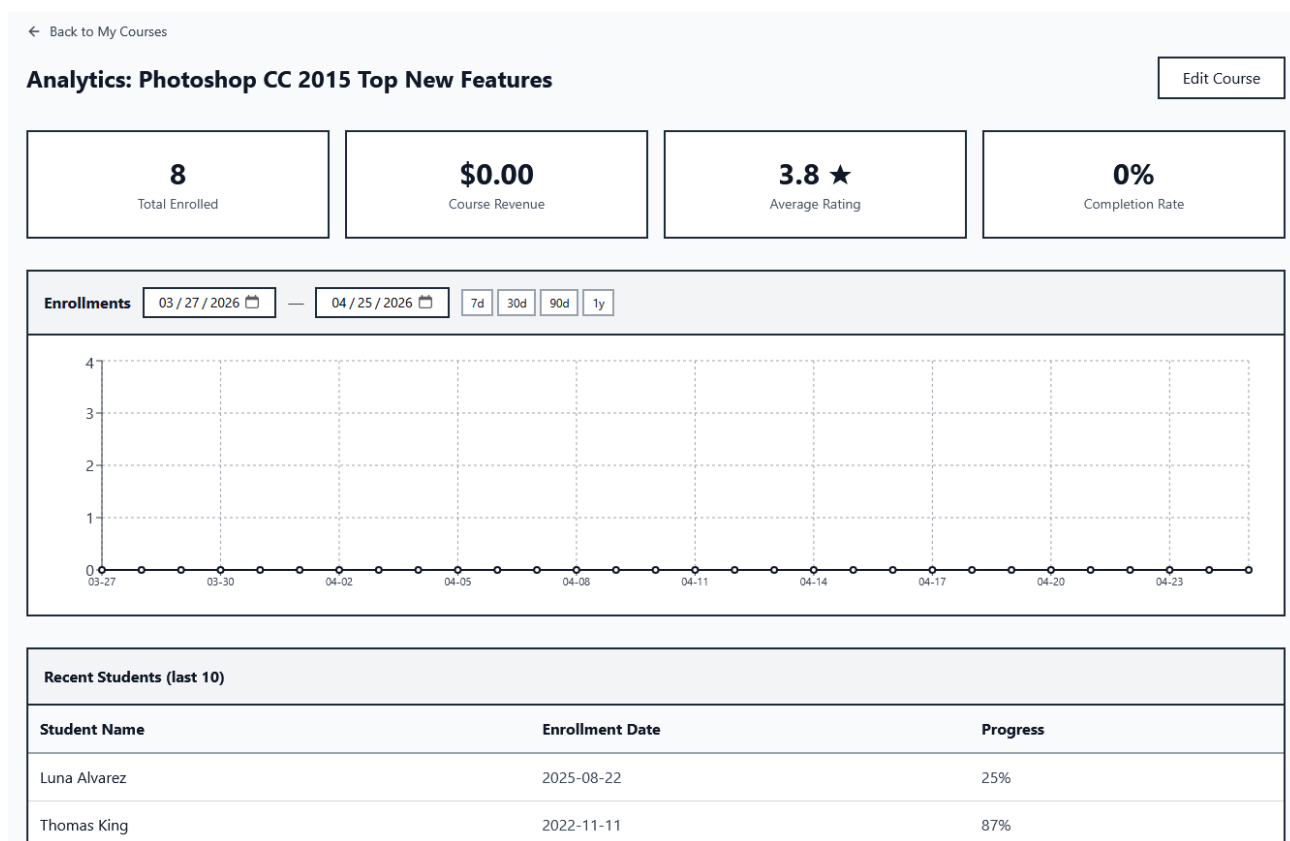


Рис. 5.9. Аналітична панель курсу для вчителя

5.3. Засоби аналітичного представлення даних

Графік динаміки доходів та виплат (адміністратор).

Фінансова панель включає лінійний графік із двома лініями: щоденний дохід та щоденні виплати за обраний період (див. рис. 5.8). Графік побудовано на основі часового ряду з нульовим заповненням (дні без транзакцій відображаються як нульові значення). Для побудови використано бібліотеку Recharts.

Графік динаміки записів (викладач).

Аналітична панель курсу включає лінійний графік щоденних записів за останні 30 днів (див. рис. 5.9). Графік дозволяє викладачу оцінити тренд популярності курсу.

Усі табличні подання (оплати, виплати, користувачі, відгуки) підтримують:

- фільтрацію за статусом (випадаючий список);
- текстовий пошук із затримкою введення (debounce);
- пагінацію зі сторінками та навігаційними кнопками.

Параметри фільтрації та пагінації передаються як параметри URL-запиту до серверного API.

5.4. Опис додаткових реалізованих функцій

Додатковою функцією, реалізованою відповідно до індивідуального завдання, є алгоритм підбору рекомендованих курсів для студентів. Алгоритм базується на методі видобування асоціативних правил та детально описаний раніше у підрозділі 4.4.

В інтерфейсі студента рекомендовані курси відображаються на панелі навчання під списком поточних курсів (див. рис. 5.10). Кожна рекомендація супроводжується текстовим поясненням, що вказує на основне правило: «47%

студентів, які проходили "React for Beginners", також обрали цей курс (1.8× вище середнього)».

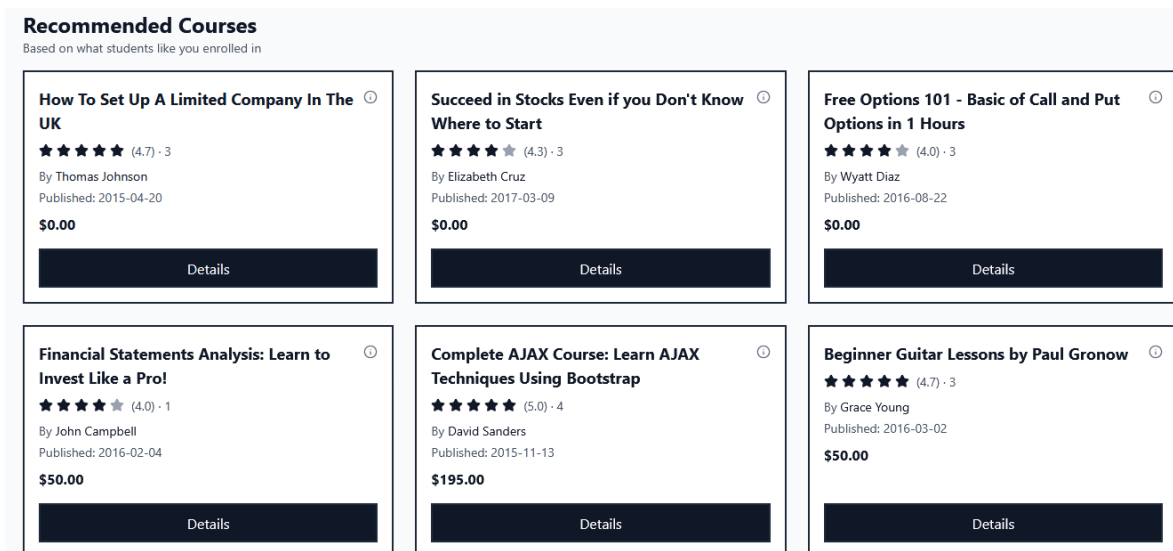


Рис. 5.10. Рекомендовані курси

Для детальнішого вивчення метрик рекомендацій передбачено перемикач «Показати пояснення», що розгортає додаткову інформацію: числове значення впевненості (confidence), підйому (lift) та загальної оцінки (score) (див. рис. 5.11). Для студентів без жодного запису (ситуація «холодного старту») відображаються найпопулярніші курси платформи із відповідним поясненням.

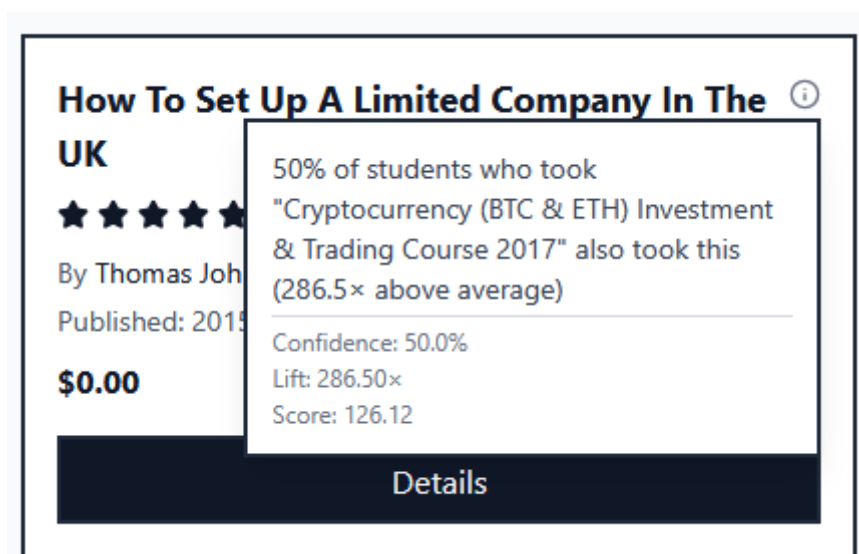


Рис. 5.11. Опис рекомендації курсу

ВИСНОВКИ

У рамках курсової роботи спроектував та реалізував інформаційну систему для управління навчальною онлайн-платформою. Система забезпечує повний цикл взаємодії між трьома ролями користувачів – студентом, викладачем та адміністратором – та охоплює три бізнес-процеси: продаж навчальних курсів, проходження навчальних матеріалів із автоматичним відстеженням прогресу та виплату винагороди викладачам.

Одним із найважливіших здобутих уроків стало розуміння різниці між реалізацією бізнес-правил у прикладному коді та на рівні бази даних. Обчислення прогресу навчання та балансу викладача було реалізовано тригерами, а не логікою контролерів. Це рішення виправдало себе: узгодженість похідних значень гарантується незалежно від того, чи дані змінюються через API, програмою-завантажувачем чи безпосередньо в СУБД.

Кілька проектних рішень виявились вдалими. Використання `enrollment_id` як первинного ключа таблиці `reviews` структурно унеможлиблює два класи помилок: повторний відгук та відгук без запису на курс. Зовнішній ключ `courses.author_id`, що посилається на `teacher_profiles` замість `users`, аналогічно гарантує на рівні схеми, що автором курсу може бути лише викладач. Зберігання ролей та статусів як текстових стовпців із обмеженнями `CHECK` замість типів `ENUM` виявилось зручнішим для еволюції схеми — додавання нового значення потребує лише зміни обмеження, без перебудови типу.

Водночас є деякі рішення, що потребують доопрацювання. Найсуттєвішим недоліком є відсутність функції видалення відгуків адміністратором та викладачем – наразі видалити відгук може лише його автор, що унеможлиблює модерацію контенту. Спрощена фінансова модель без комісії платформи не відповідає реальним бізнес-процесам: увесь дохід зараховується викладачу. Контент уроків обмежений текстовим форматом, що звужує можливості платформи порівняно з реальними аналогами. Система також не передбачає механізму сповіщень про події поза межами активної сесії браузера.

Якби проект розпочинався заново, доцільно було б з самого початку передбачити кращий механізм модерації контенту, оскільки його відсутність виявилась на етапі демонстрації. Також варто було б реалізувати комісію платформи одразу при проектуванні фінансових тригерів – додавання стовпця `platform_fee` та зміна формули обчислення балансу є технічно простою задачею, проте потребує повторного тестування усіх фінансових сценаріїв.

Реалізація алгоритму рекомендацій на основі асоціативних правил продемонструвала можливості SQL для аналітичної обробки даних. Збережена функція з дев'ятьма CTE-виразами виконує складний аналіз шаблонів поведінки користувачів повністю на стороні СУБД, без перенесення даних на рівень прикладного коду. Проте для великих обсягів даних такий підхід може потребувати оптимізації через матеріалізовані представлення або попередній розрахунок метрик.

Загалом виконання курсової роботи дозволило набути практичного досвіду проектування реляційної бази даних для реальної предметної області, реалізації серверної бізнес-логіки на кількох рівнях (СУБД та прикладний код), побудови клієнт-серверної архітектури з трирівневим розподілом відповідальності та завантаження та генерації тестових даних із зовнішнього джерела.

СПИСОК ЛІТЕРАТУРИ

Сучасний ринок пасажирських перевезень активно розвивається завдяки впровадженню цифрових технологій. Сервіси виклику таксі дозволяють пасажирам швидко замовляти поїздки, відстежувати місцезнаходження водія в реальному часі, автоматично розраховувати вартість на основі відстані та тарифу, а також здійснювати оплату різними способами. Автоматизація цих процесів суттєво підвищує якість обслуговування та ефективність управління транспортним бізнесом.

ДОДАТКИ

Додаток А. Скрипт створення бази даних

```

[25.04.2026 22:01] Бодя: -- DROP SCHEMA
public;

CREATE SCHEMA public AUTHORIZATION
pg_database_owner;

-- DROP SEQUENCE
public.categories_id_seq;

CREATE SEQUENCE public.categories_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;
-- DROP SEQUENCE public.courses_id_seq;

CREATE SEQUENCE public.courses_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;
-- DROP SEQUENCE public.enrollments_id_seq;

CREATE SEQUENCE public.enrollments_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;
-- DROP SEQUENCE public.lessons_id_seq;

CREATE SEQUENCE public.lessons_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;
-- DROP SEQUENCE public.payments_id_seq;

CREATE SEQUENCE public.payments_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;
-- DROP SEQUENCE public.payouts_id_seq;

CREATE SEQUENCE public.payouts_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;
-- DROP SEQUENCE public.users_id_seq;

CREATE SEQUENCE public.users_id_seq
INCREMENT BY 1
MINVALUE 1
MAXVALUE 2147483647
START 1
CACHE 1
NO CYCLE;

NO
CYCLE;--
public."__EFMigrationsHistory" definition
-- Drop table
-- DROP TABLE
public."__EFMigrationsHistory";
CREATE TABLE
public."__EFMigrationsHistory" (
migration_id varchar(150) NOT NULL,
product_version varchar(32) NOT NULL,
CONSTRAINT pk__ef_migrations_history
PRIMARY KEY (migration_id)
);
-- public.categories definition
-- Drop table
-- DROP TABLE public.categories;
CREATE TABLE public.categories (
id int4 GENERATED BY DEFAULT AS
IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
"name" varchar(60) NOT NULL,
CONSTRAINT pk_categories PRIMARY KEY
(id)
);
CREATE UNIQUE INDEX ix_categories_name ON
public.categories USING btree (name);
-- public.users definition
-- Drop table
-- DROP TABLE public.users;
CREATE TABLE public.users (
id int4 GENERATED BY DEFAULT AS
IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
first_name varchar(100) NOT NULL,
last_name varchar(100) NOT NULL,
email varchar(255) NOT NULL,
password_hash text NOT NULL,
"role" varchar(20) NOT NULL,
status varchar(20) DEFAULT
'Active'::character varying NOT NULL,
CONSTRAINT ck_users_role CHECK
(((role)::text = ANY ((ARRAY['Admin'::character
varying, 'Teacher'::character varying,
'Student'::character varying])::text[]))),
CONSTRAINT ck_users_status CHECK
(((status)::text = ANY
((ARRAY['Active'::character varying,
'Suspended'::character varying])::text[]))),
CONSTRAINT pk_users PRIMARY KEY (id)
);
CREATE UNIQUE INDEX ix_users_email ON
public.users USING btree (email);
-- public.teacher_profiles definition
-- Drop table
-- DROP TABLE public.teacher_profiles;

```

```

CREATE TABLE public.teacher_profiles (
    user_id int4 NOT NULL,
    bio varchar(2000) NOT NULL,
    balance numeric(10, 2) DEFAULT 0.0 NOT
NULL,
    CONSTRAINT pk_teacher_profiles PRIMARY
KEY (user_id),
    CONSTRAINT
fk_teacher_profiles_users_user_id FOREIGN KEY
(user_id) REFERENCES public.users(id) ON DELETE
CASCADE
);

-- public.courses definition

-- Drop table

-- DROP TABLE public.courses;
[25.04.2026 22:01] Бодя: CREATE TABLE
public.courses (
    id int4 GENERATED BY DEFAULT AS
IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
    author_id int4 NOT NULL,
    title varchar(200) NOT NULL,
    description text NOT NULL,
    price numeric(10, 2) NOT NULL,
    "level" varchar(20) NOT NULL,
    status varchar(20) NOT NULL,
    created_at timestampz DEFAULT now()
NOT NULL,
    CONSTRAINT ck_courses_level CHECK
(((level)::text = ANY
((ARRAY['Beginner'::character_varying,
'Intermediate'::character_varying,
'Advanced'::character_varying])::text[]))),
    CONSTRAINT ck_courses_price CHECK
((price >= (0)::numeric)),
    CONSTRAINT ck_courses_status CHECK
(((status)::text = ANY
((ARRAY['Draft'::character_varying,
'Published'::character_varying])::text[]))),
    CONSTRAINT pk_courses PRIMARY KEY (id),
    CONSTRAINT
fk_courses_teacher_profiles_author_id FOREIGN
KEY (author_id) REFERENCES
public.teacher_profiles(user_id) ON DELETE
RESTRICT
);
CREATE INDEX idx_courses_author_id ON
public.courses USING btree (author_id);
CREATE INDEX ix_courses_author_id ON
public.courses USING btree (author_id);

-- public.enrollments definition

-- Drop table

-- DROP TABLE public.enrollments;

CREATE TABLE public.enrollments (
    id int4 GENERATED BY DEFAULT AS
IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
    student_id int4 NOT NULL,
    course_id int4 NOT NULL,
    enrolled_at timestampz DEFAULT now()
NOT NULL,
    progress int4 DEFAULT 0 NOT NULL,
    CONSTRAINT ck_enrollments_progress
CHECK (((progress >= 0) AND (progress <= 100))),
    CONSTRAINT pk_enrollments PRIMARY KEY
(id),

```

```

CONSTRAINT
fk_enrollments_courses_course_id FOREIGN KEY
(course_id) REFERENCES public.courses(id) ON
DELETE RESTRICT,
    CONSTRAINT
fk_enrollments_users_student_id FOREIGN KEY
(student_id) REFERENCES public.users(id) ON
DELETE RESTRICT
);
CREATE INDEX idx_enrollments_course_id ON
public.enrollments USING btree (course_id);
CREATE INDEX idx_enrollments_student_id
ON public.enrollments USING btree (student_id);
CREATE INDEX ix_enrollments_course_id ON
public.enrollments USING btree (course_id);
CREATE UNIQUE INDEX
ix_enrollments_student_id_course_id
ON public.enrollments USING btree (student_id,
course_id);

-- public.lessons definition

-- Drop table

-- DROP TABLE public.lessons;

CREATE TABLE public.lessons (
    id int4 GENERATED BY DEFAULT AS
IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
    course_id int4 NOT NULL,
    order_number int4 NOT NULL,
    title varchar(200) NOT NULL,
    "content" text NOT NULL,
    CONSTRAINT pk_lessons PRIMARY KEY (id),
    CONSTRAINT
fk_lessons_courses_course_id FOREIGN KEY
(course_id) REFERENCES public.courses(id) ON
DELETE CASCADE
);
CREATE UNIQUE INDEX
ix_lessons_course_id_order_number
ON public.lessons USING btree (course_id,
order_number);

-- public.payments definition

-- Drop table

-- DROP TABLE public.payments;

CREATE TABLE public.payments (
    id int4 GENERATED BY DEFAULT AS
IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
    student_id int4 NOT NULL,
    course_id int4 NOT NULL,
    amount numeric(10, 2) NOT NULL,
    status varchar(20) NOT NULL,
    created_at timestampz DEFAULT now()
NOT NULL,
    CONSTRAINT ck_payments_amount CHECK
((amount >= (0)::numeric)),
    CONSTRAINT ck_payments_status CHECK
(((status)::text = ANY
((ARRAY['Completed'::character_varying,
'Refunded'::character_varying])::text[]))),
    CONSTRAINT pk_payments PRIMARY KEY
(id),
    CONSTRAINT
fk_payments_courses_course_id FOREIGN KEY
(course_id) REFERENCES public.courses(id) ON
DELETE RESTRICT,
    CONSTRAINT
fk_payments_users_student_id FOREIGN KEY

```

```

(student_id) REFERENCES public.users(id) ON
DELETE RESTRICT
);
CREATE INDEX idx_payments_course_id ON
public.payments USING btree (course_id);
CREATE INDEX ix_payments_course_id ON
public.payments USING btree (course_id);
CREATE INDEX ix_payments_student_id ON
public.payments USING btree (student_id);

-- Table Triggers
[25.04.2026 22:01] Водя: CREATE TRIGGER
trg_payments_balance AFTER
INSERT
OR
UPDATE
OF status ON
public.payments FOR EACH ROW EXECUTE
FUNCTION trg_payments_recompute_balance();

-- public.payouts definition
-- Drop table
-- DROP TABLE public.payouts;

CREATE TABLE public.payouts (
  id int4 GENERATED BY DEFAULT AS
  IDENTITY( INCREMENT BY 1 MINVALUE 1 MAXVALUE
  2147483647 START 1 CACHE 1 NO CYCLE) NOT NULL,
  teacher_id int4 NOT NULL,
  amount numeric(10, 2) NOT NULL,
  status varchar(20) NOT NULL,
  requested_at timestamptz DEFAULT now()
  NOT NULL,
  CONSTRAINT ck_payouts_amount CHECK
  ((amount >= (0)::numeric)),
  CONSTRAINT ck_payouts_status CHECK
  (((status)::text = ANY
  ((ARRAY['Pending'::character_varying,
  'Approved'::character_varying,
  'Rejected'::character_varying, 'Paid'::character
  varying])::text[]))),
  CONSTRAINT pk_payouts PRIMARY KEY (id),
  CONSTRAINT
  fk_payouts_teacher_profiles_teacher_id FOREIGN
  KEY (teacher_id) REFERENCES
  public.teacher_profiles(user_id) ON DELETE
  RESTRICT
);
CREATE INDEX idx_payouts_teacher_id ON
public.payouts USING btree (teacher_id);
CREATE INDEX ix_payouts_teacher_id ON
public.payouts USING btree (teacher_id);

-- Table Triggers

CREATE TRIGGER trg_payouts_balance AFTER
INSERT
OR
UPDATE
OF status ON
public.payouts FOR EACH ROW EXECUTE
FUNCTION trg_payouts_recompute_balance();

-- public.reviews definition
-- Drop table
-- DROP TABLE public.reviews;

CREATE TABLE public.reviews (
  enrollment_id int4 NOT NULL,
  grade int4 NOT NULL,
  "comment" varchar(2000) NOT NULL,
  created_at timestamptz DEFAULT now()
  NOT NULL,
  CONSTRAINT ck_reviews_grade CHECK
  (((grade >= 1) AND (grade <= 5))),
  CONSTRAINT pk_reviews PRIMARY KEY
  (enrollment_id),
  CONSTRAINT
  fk_reviews_enrollments_enrollment_id FOREIGN KEY
  (enrollment_id) REFERENCES
  public.enrollments(id) ON DELETE CASCADE
);

-- public.course_categories definition
-- Drop table
-- DROP TABLE public.course_categories;

CREATE TABLE public.course_categories (
  categories_id int4 NOT NULL,
  courses_id int4 NOT NULL,
  CONSTRAINT pk_course_categories PRIMARY
  KEY (categories_id, courses_id),
  CONSTRAINT
  fk_course_categories_categories_categories_id
  FOREIGN KEY (categories_id) REFERENCES
  public.categories(id) ON DELETE CASCADE,
  CONSTRAINT
  fk_course_categories_courses_courses_id FOREIGN
  KEY (courses_id) REFERENCES public.courses(id)
  ON DELETE CASCADE
);
CREATE INDEX ix_course_categories_courses_id ON
public.course_categories USING btree
(courses_id);

-- public.lesson_progress definition
-- Drop table
-- DROP TABLE public.lesson_progress;

CREATE TABLE public.lesson_progress (
  enrollment_id int4 NOT NULL,
  lesson_id int4 NOT NULL,
  completed_at timestamptz DEFAULT now()
  NOT NULL,
  CONSTRAINT pk_lesson_progress PRIMARY
  KEY (enrollment_id, lesson_id),
  CONSTRAINT
  fk_lesson_progress_enrollments_enrollment_id
  FOREIGN KEY (enrollment_id) REFERENCES
  public.enrollments(id) ON DELETE CASCADE,
  CONSTRAINT
  fk_lesson_progress_lessons_lesson_id FOREIGN KEY
  (lesson_id) REFERENCES public.lessons(id) ON
  DELETE RESTRICT
);
CREATE INDEX ix_lesson_progress_lesson_id ON
public.lesson_progress USING btree
(lesson_id);

-- Table Triggers

CREATE TRIGGER
trg_lesson_progress_recompute AFTER
INSERT
OR
DELETE
ON
public.lesson_progress FOR EACH ROW
EXECUTE FUNCTION
recompute_enrollment_progress();

```

```

-- DROP FUNCTION
public.get_recommendations(int4, int4);

CREATE OR REPLACE FUNCTION
public.get_recommendations(target_student_id
integer, max_results integer DEFAULT 6)
RETURNS TABLE(course_id integer, score
numeric, confidence numeric, lift numeric, reason
text)
LANGUAGE plpgsql
AS $function$
DECLARE
    student_course_count INT;
    total_students INT;
BEGIN
    SELECT COUNT(*) INTO
student_course_count
FROM enrollments WHERE student_id =
target_student_id;

    SELECT COUNT(DISTINCT student_id)
INTO total_students
FROM enrollments;
    [25.04.2026 22:01] Боя: -- cold start
when no enrollments: show most popular courses
    IF student_course_count = 0 THEN
        RETURN QUERY
        SELECT
            c.id AS course_id,
            COUNT(e.id)::NUMERIC AS
score,
            0::NUMERIC AS confidence,
            0::NUMERIC AS lift,
            'Popular on the
platform'::TEXT AS reason
FROM courses c
LEFT JOIN enrollments e ON
e.course_id = c.id
WHERE c.status = 'Published'
GROUP BY c.id
ORDER BY score DESC
LIMIT max_results;
        RETURN;
    END IF;

    --
    RETURN QUERY
    WITH my_courses AS (
        SELECT e.course_id
        FROM enrollments e
        WHERE e.student_id =
target_student_id
    ),
    my_categories AS (
        SELECT DISTINCT cc.categories_id
AS cat_id
FROM my_courses mc
JOIN course_categories cc ON
cc.courses_id = mc.course_id
    ),

    -- enrollment counts for each
student's enrolled course (for confidence and
lift calculations)
    course_enrollment_counts AS (
        SELECT enr.course_id,
COUNT(DISTINCT enr.student_id) AS cnt
FROM enrollments enr
GROUP BY enr.course_id
    ),

    -- other students' enrollments with
completion weight
    peer_enrollments AS (
        SELECT
            e_mine.course_id AS
shared_course_id,
            e_other.student_id AS
peer_id,
            -- for each of mine
courses, find other students who also are
enrolled in the same course
            e_other.course_id AS
rec_course_id,
            -- for each peer find other
courses they're enrolled in
            e_peer.progress AS
peer_shared_progress
FROM enrollments e_mine
JOIN enrollments e_peer
ON e_peer.course_id =
e_mine.course_id
AND e_peer.student_id !=
target_student_id
JOIN enrollments e_other
ON e_other.student_id =
e_peer.student_id
WHERE e_mine.student_id =
target_student_id
AND e_other.course_id NOT IN
(SELECT mc.course_id FROM my_courses mc)
    ),

    -- per-rule metrics
    rule_metrics AS (
        SELECT
            pe.rec_course_id,
            pe.shared_course_id,
            COUNT(DISTINCT pe.peer_id) AS
co_count,
            -- how many students took
both courses
            -- confidence: co_count /
enrolled_in_shared
            COUNT(DISTINCT
pe.peer_id)::NUMERIC
/ NULLIF(cec_shared.cnt,
0) AS rule_confidence,
            -- lift: confidence /
(enrolled_in_recommended / probability of
recommended)
            (COUNT(DISTINCT
pe.peer_id)::NUMERIC
/ NULLIF(cec_shared.cnt,
0))
            --confidence
            /
NULLIF(cec_rec.cnt::NUMERIC
/ NULLIF(total_students, 0), 0)
            --P(recommended)
            AS rule_lift,
            -- Completion-weighted peer
count
            SUM(0.5 + 0.5 *
pe.peer_shared_progress / 100.0) AS
weighted_peers
FROM peer_enrollments pe
JOIN course_enrollment_counts
cec_shared
ON cec_shared.course_id =
pe.shared_course_id
JOIN course_enrollment_counts
cec_rec
ON cec_rec.course_id =
pe.rec_course_id
    )

```

```

GROUP BY pe.rec_course_id,
pe.shared_course_id,
cec_rec.cnt
),
-- best rule per recommended course
(highest confidence * lift)
best_rule AS (
SELECT DISTINCT ON
(rm.rec_course_id)
rm.rec_course_id,
rm.shared_course_id,
rm.co_count,
rm.rule_confidence,
rm.rule_lift
FROM rule_metrics rm
ORDER BY rm.rec_course_id,
(rm.rule_confidence * rm.rule_lift) DESC
),
-- rating quality
avg_ratings AS (
SELECT
e.course_id AS rec_course_id,
COALESCE(AVG(r.grade), 3.0)
AS avg_rating
FROM reviews r
JOIN enrollments e ON e.id =
r.enrollment_id
GROUP BY e.course_id
),
[25.04.2026 22:01] Бодя: -- category
overlap
cat_overlap AS (
SELECT
cc.courses_id AS
rec_course_id,
COUNT(mc.cat_id) AS
overlap_count
FROM course_categories cc
JOIN my_categories mc ON
mc.cat_id = cc.categories_id
GROUP BY cc.courses_id
),
-- final score
scored AS (
SELECT
rm.rec_course_id,
SUM(rm.rule_confidence *
COALESCE(rm.rule_lift, 1)
* rm.weighted_peers)
* (COALESCE(ar.avg_rating, 3.0) / 5.0)
* (1.0 + 0.2 *
COALESCE(co.overlap_count, 0))
AS final_score,
br.shared_course_id,
br.co_count,
br.rule_confidence AS
best_confidence,
br.rule_lift AS best_lift
FROM rule_metrics rm
JOIN best_rule br ON
br.rec_course_id = rm.rec_course_id
LEFT JOIN avg_ratings ar ON
ar.rec_course_id = rm.rec_course_id
LEFT JOIN cat_overlap co ON
co.rec_course_id = rm.rec_course_id
GROUP BY rm.rec_course_id,
br.shared_course_id, br.co_count,
br.rule_confidence,
br.rule_lift,
ar.avg_rating,
co.overlap_count
)

```

```

SELECT
s.rec_course_id AS course_id,
ROUND(s.final_score, 2) AS score,
ROUND(s.best_confidence * 100, 1)
AS confidence,
ROUND(COALESCE(s.best_lift, 0),
2) AS lift,
FORMAT('%s%% of students who took
"%s" also took this (%sx above average)',
ROUND(s.best_confidence *
(SELECT c2.title FROM courses
c2 WHERE c2.id = s.shared_course_id),
1), 1)
)::TEXT AS reason
FROM scored s
JOIN courses c ON c.id =
s.rec_course_id AND c.status = 'Published'
ORDER BY s.final_score DESC
LIMIT max_results;
END;
$function$
;

-- DROP FUNCTION
public.recompute_enrollment_progress();

CREATE OR REPLACE FUNCTION
public.recompute_enrollment_progress()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
target_enrollment_id INT;
total_lessons INT;
done_lessons INT;
BEGIN
target_enrollment_id :=
COALESCE(NEW.enrollment_id, OLD.enrollment_id);

SELECT COUNT(*) INTO total_lessons
FROM lessons l
JOIN enrollments e ON e.course_id =
l.course_id
WHERE e.id = target_enrollment_id;

SELECT COUNT(*) INTO done_lessons
FROM lesson_progress
WHERE enrollment_id =
target_enrollment_id;

UPDATE enrollments
SET progress = CASE
WHEN total_lessons = 0 THEN 0
ELSE (done_lessons * 100) /
total_lessons
END
WHERE id = target_enrollment_id;

RETURN NULL;
END;
$function$
;

-- DROP FUNCTION
public.recompute_teacher_balance(int4);

CREATE OR REPLACE FUNCTION
public.recompute_teacher_balance(target_teacher
_id integer)
RETURNS void
LANGUAGE plpgsql
AS $function$
BEGIN
UPDATE teacher_profiles tp
SET balance = COALESCE((

```

```

        SELECT SUM(p.amount)
        FROM payments p
        JOIN courses c ON c.id =
p.course_id
        WHERE c.author_id = tp.user_id
        AND p.status = 'Completed'
    ), 0) - COALESCE((
        SELECT SUM(po.amount)
        FROM payouts po
        WHERE po.teacher_id = tp.user_id
        AND po.status IN ('Pending',
'Approved', 'Paid')
    ), 0)
    WHERE tp.user_id = target_teacher_id;
END;
$function$
;

--          DROP          FUNCTION
public.trg_payments_recompute_balance();

CREATE OR REPLACE FUNCTION
public.trg_payments_recompute_balance()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
    affected_teacher INT;
BEGIN
    SELECT          author_id          INTO
affected_teacher

```

```

        FROM courses
        WHERE id = COALESCE(NEW.course_id,
OLD.course_id);

    IF affected_teacher IS NOT NULL THEN
        PERFORM
recompute_teacher_balance(affected_teacher);
    END IF;

    RETURN NULL;
END;
$function$
;

--          DROP          FUNCTION
public.trg_payouts_recompute_balance();

CREATE OR REPLACE FUNCTION
public.trg_payouts_recompute_balance()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
BEGIN
    PERFORM
recompute_teacher_balance(COALESCE(NEW.teacher_
id, OLD.teacher_id));
    RETURN NULL;
END;
$function$
;

```

Додаток Б. Скрипт завантаження первинних даних

```

using System.Globalization;
using System.Text.RegularExpressions;
using CsvHelper;
using CsvHelper.Configuration;
using
CsvHelper.Configuration.Attributes;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.DataLoader;

internal static partial class FullLoader
{
    private const int CoursesPerTeacher =
6; // target avg
    private const int StudentsPerCourse =
5; // target avg enrollments per course
    private const int MaxLessonsPerCourse
= 8;
    private const double
ReviewProbability = 0.8; // 80% of enrollments
have a review

    private static readonly string
CsvPath = Path.Combine(
    AppContext.BaseDirectory,
    "data", "courses.csv");

    // ?
    // Matches strings that contain ONLY
basic Latin letters, digits, common punctuation,
and whitespace.
    // Rows whose title fails this check
are skipped (non-Latin/emoji titles render as
tofu).
    [GeneratedRegex(@"^[a-zA-Z0-9\.\-\_\,\;\:\&'\"]+$")]
    private static partial Regex
AsciiOnlyRegex();

    public static async Task
RunAsync(AppDbContext db, int? limit = null,
    DateTime? enrollFromOverride =
null, DateTime? enrollToOverride = null)
    {
        if (!File.Exists(CsvPath))
        {
            Console.Error.WriteLine($"ERROR: dataset not
found at {CsvPath}");
            Environment.Exit(1);
        }

        var hasher = new
PasswordHasher<User>();
        var rng = new Random(42);
        var enrollTo = enrollToOverride
?? DateTime.UtcNow;

        Console.WriteLine("Reading
CSV...");
        var allRows = ReadCsv(CsvPath);
        Console.WriteLine($"
{allRows.Count} rows in file");

        // filter out the rows with
invalid (non ascii) titles
        var rows = allRows.Where(r =>

!string.IsNullOrEmpty(r.CourseTitle) &&
AsciiOnlyRegex().IsMatch(r.CourseTitle)).ToList
();

        var skipped = allRows.Count -
rows.Count;

        if (skipped > 0)
            Console.WriteLine($"
skipped {skipped} rows with non ascii titles");

        if (limit is not null && limit <
rows.Count)
        {
            Console.WriteLine($" taking
first {limit} rows (--limit)");
            rows = rows
.Take(limit.Value)];

        // categories
        Console.WriteLine("Inserting
categories...");
        var subjectNames = rows
.Select(r => r.Subject)
.Where(s =>
!string.IsNullOrEmpty(s))
.Select(s => s!.Trim())
.Distinct()
.ToList();

        var categories =
subjectNames.Select(n => new Category { Name = n
}).ToList();

        db.Categories.AddRange(categories);
        await db.SaveChangesAsync();

        var categoryByName =
categories.ToDictionary(c => c.Name, c => c);
        Console.WriteLine($"
{categories.Count} categories");

        // teachers

        // teacher count is based on
course count
        // each teacher has
~CoursesPerTeacher courses.

        var teacherCount = Math.Max(50,
rows.Count / CoursesPerTeacher);
        Console.WriteLine($"Generating
{teacherCount} teachers...");

        var teachers = new List<User>();
        var teacherCategories = new
List<int>(); // parallel: each teacher's primary
category id
        var usedEmails = new
HashSet<string>(StringComparer.OrdinalIgnoreCas
e);
        for (var i = 0; i < teacherCount;
i++)
        {
            var (first, last) =
NamePool.Random(rng);
            string email;
            var seq = 0;
            do { email =
NamePool.EmailFor(first, last, seq++); } while
(!usedEmails.Add(email));
            teachers.Add(new User
            {
                FirstName = first,
                LastName = last,
                Email = email,
                Role = Roles.Teacher,
                PasswordHash =
hasher.HashPassword(null!, "password123"),
            });
        }
    }
}

```

```

// Each teacher specializes
in one category

teacherCategories.Add(categories[i %
categories.Count].Id);
}

db.Users.AddRange(teachers);
await db.SaveChangesAsync();

var teacherProfiles = new
List<TeacherProfile>();
foreach (var t in teachers)
    teacherProfiles.Add(new
TeacherProfile { UserId = t.Id, Bio = "" });

db.TeacherProfiles.AddRange(teacherProfiles);
await db.SaveChangesAsync();

Console.WriteLine($"
{teachers.Count} teachers (~{rows.Count /
teacherCount} courses each)");

// category -> teachers lookup
index for course assignment
var teachersByCategory = new
Dictionary<int, List<int>>(); // categoryId:
[teacherIds]
for (var i = 0; i <
teachers.Count; i++)
{
    var catId =
teacherCategories[i];
    if
(!teachersByCategory.TryGetValue(catId, out var
list))
    {
        list = new List<int>();
    }
    teachersByCategory[catId] = list;
    list.Add(i);
}

// courses

Console.WriteLine("inserting
courses...");

var courses = new List<Course>();
for (var i = 0; i < rows.Count;
i++)
{
    var r = rows[i];

    var category =
categoryByName.TryGetValue(r.Subject?.Trim() ??
"", out var c) ? c : null;

    // assign the course to a
teacher who specializes in this category
    // 10% of courses go to a
random teacher

    int teacherIdx;
    if (category is not null &&
rng.NextDouble() > 0.1
    &&
teachersByCategory.TryGetValue(category.Id, out
var categoryTeachers) && categoryTeachers.Count
> 0)
    {
        teacherIdx =
categoryTeachers[rng.Next(categoryTeachers.Coun
t)];

```

```

}
else
{
    teacherIdx =
rng.Next(teachers.Count);
}

var course = new Course
{
    AuthorId =
teachers[teacherIdx].Id,
    Title =
Truncate(r.CourseTitle!.Trim(), 200),
    Description =
"${r.CourseTitle.Trim()}. \n\nDescription for the
course. Lorem Ipsum sit amet.",
    Price = r.IsPaid ?
Math.Max(0m, r.Price) : 0m,
    Level =
MapLevel(r.Level),
    Status =
CourseStatus.Published,
    CreatedAt =
ParseTimestampUtc(r.PublishedTimestamp),
    Categories = category is
null ? [] : [category],
};
courses.Add(course);
}

const int chunkSize = 500;
for (var i = 0; i < courses.Count;
i += chunkSize)
{
    db.Courses.AddRange(courses.GetRange(i,
Math.Min(chunkSize, courses.Count - i)));
    await db.SaveChangesAsync();
    Console.WriteLine($"r inserting
courses... {Math.Min(i + chunkSize,
courses.Count)} / {courses.Count}");
}
Console.WriteLine();
Console.WriteLine($"
{courses.Count} courses");

// lessons

Console.WriteLine("Inserting
lessons...");
var totalLessons = 0;
for (var i = 0; i < courses.Count;
i += chunkSize)
{
    var batch = new
List<Lesson>();
    var end = Math.Min(i +
chunkSize, courses.Count);
    for (var j = i; j < end; j++)
    {
        var course = courses[j];
        var rawCount =
rows[j].NumLectures;
        var count =
Math.Clamp(rawCount, 1, MaxLessonsPerCourse);
        for (var k = 1; k <=
count; k++)
        {
            batch.Add(new Lesson
            {
                CourseId =
course.Id,
                OrderNumber = k,
                Title = $"Topic
{k}",

```

```

        Content = $"This
is the text content of lesson {k} of
\"{course.Title}\". " +
        "Lorem
ipsum dolor sit amet, consectetur adipiscing
elit. Sed do " +
        "eiusmod tempor incididunt ut labore et dolore
magna aliqua.",
        });
        totalLessons++;
    }
    db.Lessons.AddRange(batch);
    await db.SaveChangesAsync();
    Console.WriteLine($"r inserting
lessons... {totalLessons} so far");
}
Console.WriteLine();
Console.WriteLine($"
{totalLessons} lessons");

// students

var studentCount = Math.Max(100,
rows.Count / 2);
Console.WriteLine($"Generating
{studentCount} students...");
var students = new List<User>();

// each student has 1-2 preferred
categories
// 70% of their enrollments are
biased towards courses in those categories
// 30% random

var studentPreferences = new
List<HashSet<int>>(); // categoryId sets,
parallel with students
for (var i = 0; i < studentCount;
i++)
{
    var (first, last) =
NamePool.Random(rng);
    string email;
    var seq = 0;
    do { email =
NamePool.EmailFor(first, last, seq++); } while
(!usedEmails.Add(email));
    students.Add(new User
    {
        FirstName = first,
        LastName = last,
        Email = email,
        Role = Roles.Student,
        PasswordHash =
hasher.HashPassword(null!, "password123"),
    });

    var prefCount = rng.Next(1,
3); // 1 or 2 favourite categories
    var prefs = new
HashSet<int>();
    while (prefs.Count <
prefCount && prefs.Count < categories.Count)
        prefs.Add(categories[rng.Next(categories.Count)
].Id);

    studentPreferences.Add(prefs);
}

db.Users.AddRange(students);
await db.SaveChangesAsync();

```

```

        Console.WriteLine($"
+
{students.Count} students (each with 1-2
preferred categories)");

var targetEnrollmentsCount =
Math.Min(courses.Count * StudentsPerCourse,
students.Count * 30);

// category -> course index for
enrollment with bias
var coursesByCategory = new
Dictionary<int, List<int>>(); // categoryId:
[courseIndexes]
for (var i = 0; i < courses.Count;
i++)
{
    foreach (var cat in
courses[i].Categories)
    {
        if
(!coursesByCategory.TryGetValue(cat.Id, out var
list))
        {
            list = new
List<int>();

            coursesByCategory[cat.Id] = list;
        }
        list.Add(i);
    }
}

// enrollments

Console.WriteLine("Generating
enrollments...");
var taken = new HashSet<(int sid,
int cid)>();
var enrollmentList = new
List<(Enrollment e, Payment p)>();
var attempts = 0;
var maxAttempts =
targetEnrollmentsCount * 4;

while (taken.Count <
targetEnrollmentsCount && attempts <
maxAttempts)
{
    attempts++;
    var studentIdx =
rng.Next(students.Count);
    var student =
students[studentIdx];
    var prefs =
studentPreferences[studentIdx];

    int courseIdx;
    if (rng.NextDouble() < 0.7 &&
prefs.Count > 0 && coursesByCategory.Count > 0)
    {
        // 70% chance to pick a
course from a preferred category
        var prefCatId =
prefs.ElementAt(rng.Next(prefs.Count));
        // take random course
with the category
        if
(coursesByCategory.TryGetValue(prefCatId, out
var catCourses) && catCourses.Count > 0)
        {
            courseIdx =
catCourses[rng.Next(catCourses.Count)];
        }
        else
        {
            courseIdx =
rng.Next(courses.Count); // fallback
        }
    }
    else

```

```

        {
            // 30% chance to pick any
            random course
            courseId =
            rng.Next(courses.Count);
        }
        var course =
        courses[courseIdx];
        if (!taken.Add((student.Id,
        course.Id))) continue;

        var earliest =
        enrollFromOverride ?? course.CreatedAt;
        var latest = enrollTo;
        if (earliest > latest)
        earliest = latest;
        var enrolledAt =
        RandomDateBetween(rng, earliest, latest);

        var enrollment = new
        Enrollment
        {
            StudentId = student.Id,
            CourseId = course.Id,
            EnrolledAt = enrolledAt,
        };
        var payment = new Payment
        {
            StudentId = student.Id,
            CourseId = course.Id,
            Amount = course.Price,
            Status =
            PaymentStatus.Completed,
            CreatedAt = enrolledAt,
        };

        enrollmentList.Add((enrollment, payment));

        if (enrollmentList.Count >=
        chunkSize)
        {
            db.Enrollments.AddRange(enrollmentList.Select(x
            => x.e));

            db.Payments.AddRange(enrollmentList.Select(x =>
            x.p));

            await
            db.SaveChangesAsync();

            Console.WriteLine($"r
            inserting enrollments...
            {taken.Count}/{targetEnrollmentsCount}");
            enrollmentList.Clear();
        }

        db.Enrollments.AddRange(enrollmentList.Select(x
        => x.e));

        db.Payments.AddRange(enrollmentList.Select(x =>
        x.p));

        await db.SaveChangesAsync();

        Console.WriteLine();
        Console.WriteLine($"
        {taken.Count} enrollments (and matching
        payments)");

        // lesson progress

        Console.WriteLine("Generating
        lesson progress...");

```

```

        // courseId -> lessonIds map
        var lessonsByCourse = await
        db.Lessons
            .GroupBy(l => l.CourseId)
            .ToDictionaryAsync(g
            =>
            g.Key, g => g.Select(l => l.Id).ToList());

        var allEnrollments = await
        db.Enrollments.ToListAsync();
        var progressBatch = new
        List<LessonProgress>();
        var totalProgress = 0;

        foreach (var enrollment in
        allEnrollments)
        {
            if
            (!lessonsByCourse.TryGetValue(enrollment.Course
            Id, out var lessonIds)) continue;

            var completionRatio =
            rng.NextDouble();
            // completionRatio =
            Math.Min(1.0, completionRatio);

            var lessonsToComplete =
            (int)(lessonIds.Count * completionRatio);

            for (var i = 0; i <
            lessonsToComplete; i++)
            {
                var completedAt =
                RandomDateBetween(rng,
                enrollment.EnrolledAt,
                enrollTo);

                progressBatch.Add(new
                LessonProgress
                {
                    EnrollmentId =
                    enrollment.Id,
                    LessonId =
                    lessonIds[i],
                    CompletedAt =
                    completedAt,
                });
                totalProgress++;
            }

            if (progressBatch.Count >=
            chunkSize)
            {
                db.LessonProgress.AddRange(progressBatch);
                await
                db.SaveChangesAsync();

                Console.WriteLine($"r
                inserting lesson progress... {totalProgress} so
                far");

                progressBatch.Clear();
            }

            if (progressBatch.Count > 0)
            {
                db.LessonProgress.AddRange(progressBatch);
                await db.SaveChangesAsync();

                Console.WriteLine();
                Console.WriteLine($"
                {totalProgress} lesson progress records");
            }

            // reviews

```

```

        Console.WriteLine("Generating
reviews...");
        var reviewComments = new[]
        {
            "Great course! Learned a lot
and would recommend to anyone interested in this
topic.",
            "Good content but could use
more practical examples.",
            "Excellent! The instructor
explains everything clearly.",
            "Decent course. Some parts
felt a bit rushed.",
            "Very well structured. I
particularly enjoyed the hands-on exercises.",
            "Not bad, but I expected more
depth on certain topics.",
            "Outstanding quality. One of
the best courses I've taken.",
            "Solid course for beginners.
More advanced students might find it too basic.",
            "The material is good but the
pacing could be better.",
            "Highly recommend! Worth
every penny.",
            "Informative and engaging
throughout.",
            "A bit too theoretical for my
taste, but still useful.",
        };

        var reviewBatch = new
List<Review>();
        foreach (var enrollment in
allEnrollments)
        {
            if (rng.NextDouble() >
ReviewProbability) continue;

            //biased
            var grade = rng.NextDouble()

            switch
            {
                < 0.05 => 1,
                < 0.10 => 2,
                < 0.25 => 3,
                < 0.55 => 4,
                - => 5,
            };

            var reviewedAt =
RandomDateBetween(rng, enrollment.EnrolledAt,
enrollTo);

            reviewBatch.Add(new Review
            {
                EnrollmentId =
enrollment.Id,
                Grade = grade,
                Comment =
reviewComments[rng.Next(reviewComments.Length)]
,
                CreatedAt = reviewedAt,
            });

            if (reviewBatch.Count >=
chunkSize)
            {
                db.Reviews.AddRange(reviewBatch);
                await
db.SaveChangesAsync();

                Console.WriteLine($"r
inserting reviews... {reviewBatch.Count} so
far");

                reviewBatch.Clear();
            }
        }
    }

```

```

db.Reviews.AddRange(reviewBatch);
await db.SaveChangesAsync();

        var totalReviews = await
db.Reviews.CountAsync();
        Console.WriteLine();
        Console.WriteLine($"
+
{totalReviews} reviews
(~{(int)(ReviewProbability * 100)}% of
{allEnrollments.Count} enrollments");

        // teacher payouts

        Console.WriteLine("Generating
teacher payouts...");
        var payoutStatuses = new[] {
PayoutStatus.Pending, PayoutStatus.Approved,
PayoutStatus.Paid, PayoutStatus.Rejected };
        var balances = await
db.TeacherProfiles.ToDictionaryAsync(tp =>
tp.UserId, tp => tp.Balance);
        var payoutBatch = new
List<Payout>();
        foreach (var teacher in teachers)
        {
            var earnings =
balances.GetValueOrDefault(teacher.Id, 0m);
            if (earnings <= 0) continue;
            var remaining = earnings;
            var payoutCount = rng.Next(0,
4); // each teacher requests 0-3 payouts
            for (var i = 0; i <
payoutCount && remaining > 10m; i++)
            {
                var maxAmount =
Math.Min(remaining * 0.6m, 200m);
                var amount =
Math.Round(Math.Max(10m,
(decimal)rng.NextDouble() * maxAmount), 2);
                var status =
payoutStatuses[rng.Next(payoutStatuses.Length)]
;
                var requestedAt =
RandomDateBetween(rng, enrollTo.AddMonths(-6),
enrollTo);

                payoutBatch.Add(new
Payout
                {
                    TeacherId =
teacher.Id,
                    Amount = amount,
                    Status = status,
                    RequestedAt =
requestedAt,
                });
                if (status !=
PayoutStatus.Rejected)
                    remaining -= amount;
            }
        }

        db.Payouts.AddRange(payoutBatch);
        await db.SaveChangesAsync();
        Console.WriteLine($"
+
{payoutBatch.Count} teacher payouts");

        Console.WriteLine();
        Console.WriteLine("Done.");
        Console.WriteLine($"
Total
records inserted (approx): " +

        $"{categories.Count + teachers.Count +
students.Count + courses.Count + totalLessons +

```

```

taken.Count + totalProgress + totalReviews +
payoutBatch.Count}");
    }

    // helpers functions

    private static List<UdemyRow>
ReadCsv(string path)
    {
        using var reader = new
StreamReader(path);
        var config = new
CsvConfiguration(CultureInfo.InvariantCulture)
        {
            HasHeaderRecord = true,
            MissingFieldFound = null,
            BadDataFound = null,
        };
        using var csv = new
CsvReader(reader, config);
        return
csv.GetRecords<UdemyRow>().ToList();
    }

    private static string
MapLevel(string? raw) => raw?.Trim() switch
    {
        "Beginner Level" =>
CourseLevel.Beginner,
        "Intermediate Level" =>
CourseLevel.Intermediate,
        "Expert Level" =>
CourseLevel.Advanced,
        "All Levels" =>
CourseLevel.Beginner,
        _ => CourseLevel.Beginner,
    };

    private static DateTime
ParseTimestampUtc(string? raw)
    {
        if
(string.IsNullOrEmpty(raw)) return
DateTime.UtcNow;
        if (DateTime.TryParse(raw,
CultureInfo.InvariantCulture,
DateTimeStyles.AssumeUniversal |
DateTimeStyles.AdjustToUniversal, out var dt))
            return
DateTime.SpecifyKind(dt, DateTimeKind.Utc);
        return DateTime.UtcNow;
    }

```

```

    }

    private static DateTime
RandomDateBetween(Random rng, DateTime from,
DateTime to)
    {
        var range = (to -
from).TotalSeconds;
        if (range <= 0) return from;
        var offset = rng.NextDouble() *
range;
        return
DateTime.SpecifyKind(from.AddSeconds(offset),
DateTimeKind.Utc);
    }

    private static string Truncate(string
s, int max) =>
s.Length <= max ? s :
s.Substring(0, max);

    // CSV row mapping
    private sealed class UdemyRow
    {
        [Name("course_id")] public int
CourseId { get; set; }
        [Name("course_title")] public
string? CourseTitle { get; set; }
        [Name("url")] public string? Url
{ get; set; }
        [Name("is_paid")] public bool
IsPaid { get; set; }
        [Name("price")] public decimal
Price { get; set; }
        [Name("num_subscribers")] public
int NumSubscribers { get; set; }
        [Name("num_reviews")] public int
NumReviews { get; set; }
        [Name("num_lectures")] public int
NumLectures { get; set; }
        [Name("level")] public string?
Level { get; set; }
        [Name("content_duration")]
public decimal ContentDuration { get; set; }
        [Name("published_timestamp")]
public string? PublishedTimestamp { get; set; }
        [Name("subject")] public string?
Subject { get; set; }
    }

```

Додаток В. Програмний код реалізації бізнес-логіки

```
//File: Controllers/CoursesController.cs
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/courses")]
[Authorize]
public class CoursesController(AppDbContext db)
: ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<PagedResult<CourseListItemDto>>> List([FromQuery] CourseQuery filters)
    {
        var query = db.Courses
            .Where(c => c.Status == CourseStatus.Published)
            .AsQueryable();

        if (!string.IsNullOrEmpty(filters.Q))
        {
            var term = $"{filters.Q.Trim()}%";
            query = query.Where(c => EF.Functions.ILike(c.Title, term));
        }

        if (filters.CategoryIds is { Length: > 0 })
        {
            query = query.Where(c => c.Categories.Any(cat => filters.CategoryIds.Contains(cat.Id)));
            if (filters.MinPrice is not null) query = query.Where(c => c.Price >= filters.MinPrice);
            if (filters.MaxPrice is not null) query = query.Where(c => c.Price <= filters.MaxPrice);
            if (filters.CreatedAfter is not null) query = query.Where(c => c.CreatedAt >= filters.CreatedAfter);
            if (filters.CreatedBefore is not null) query = query.Where(c => c.CreatedAt <= filters.CreatedBefore);
        }

        // project with rating so we can filter/sort using rating
        var projected = query.Select(c => new
        {
            Course = c,
            AverageRating = db.Reviews
                .Where(r
                    .Enrollment!.CourseId == c.Id)
                .Select(r
                    .Grade)
                .Average() ?? 0d,
            ReviewCount = db.Reviews.Count(r => r.Enrollment!.CourseId == c.Id),
            EnrollmentCount = db.Enrollments.Count(e => e.CourseId == c.Id),
        });

        if (filters.MinRating is not null) projected = projected.Where(p => p.AverageRating >= filters.MinRating);
        if (filters.MaxRating is not null) projected = projected.Where(p => p.AverageRating <= filters.MaxRating);

        projected = filters.Sort switch
        {

```

```
"oldest" => projected.OrderBy(p => p.Course.CreatedAt),
"price_asc" => projected.OrderBy(p => p.Course.Price),
"price_desc" => projected.OrderByDescending(p => p.Course.Price),
"rating_desc" => projected.OrderByDescending(p => p.AverageRating),
_ => projected.OrderByDescending(p => p.Course.CreatedAt), // newest (default)
    };

    var page = Math.Max(1, filters.Page);
    var pageSize = Math.Clamp(filters.PageSize, 1, 100);
    var totalCount = await projected.CountAsync();

    var rows = await projected
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(p => new CourseListItemDto(
            p.Course.Id,
            p.Course.Title,
            p.Course.Price,
            p.Course.Level,
            p.Course.CreatedAt,
            new AuthorDto(p.Course.Author!.UserId, p.Course.Author.User!.FirstName, p.Course.Author.User.LastName),
            p.Course.Categories.Select(cat => new CategoryDto(cat.Id, cat.Name)).ToList(),
            p.AverageRating,
            p.ReviewCount,
            p.EnrollmentCount))
        .ToListAsync();

    return new PagedResult<CourseListItemDto>(rows, totalCount, page, pageSize);

    [HttpGet("{id:int}")]
    public async Task<ActionResult<CourseDetailDto>> Get(int id)
    {
        var course = await db.Courses
            .Include(c
                .Author!)
            .ThenInclude(a => a.User)
            .Include(c => c.Categories)
            .Include(c => c.Lessons.OrderBy(l => l.OrderNumber))
            .SingleOrDefaultAsync(c => c.Id == id && c.Status == CourseStatus.Published);

        if (course is null) return NotFound();

        var reviews = await db.Reviews
            .Where(r => r.Enrollment!.CourseId == id)
            .OrderByDescending(r => r.CreatedAt)
            .Select(r => new ReviewDto(
                r.Grade,
                r.Comment,
                r.CreatedAt,
                r.Enrollment!.Student!.FirstName,
                r.Enrollment!.Student!.LastName))
            .ToListAsync();

        var avg = reviews.Count == 0 ? 0d : reviews.Average(r => (double)r.Grade);
    }
}
```

```

        var enrollmentCount = await
db.Enrollments.CountAsync(e => e.CourseId ==
id);

return new CourseDetailDto(
    course.Id,
    course.Title,
    course.Description,
    course.Price,
    course.Level,
    course.Status,
    course.CreatedAt,
    new AuthorDto(course.Author!.UserId,
course.Author.User!.FirstName,
course.Author.User.LastName),
    course.Categories.Select(c => new
CategoryDto(c.Id, c.Name)).ToList(),
    course.Lessons.Select(l => new
LessonSummaryDto(l.Id, l.OrderNumber,
l.Title)).ToList(),
    reviews,
    avg,
    enrollmentCount);
}

//File:
Controllers/TeacherAnalyticsController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Authorize(Roles = Roles.Teacher)]
public class
TeacherAnalyticsController(AppDbContext db) :
ControllerBase
{
    private int CurrentUserId =>
int.Parse(User.FindFirstValue(ClaimTypes.NameId
entifier));

[HttpGet("api/teacher/courses/{id:int}/analytic
s")]
public async
Task<ActionResult<CourseAnalyticsDto>>
Analytics(int id)
{
    var teacherId = CurrentUserId;
    var course = await db.Courses
.SingleOrDefaultAsync(c => c.Id ==
id && c.AuthorId == teacherId);
    if (course is null) return NotFound();

    var enrollments = await db.Enrollments
.Where(e => e.CourseId == id)
.Select(e => new { e.Progress,
e.EnrolledAt, e.Student!.FirstName,
e.Student.LastName })
.ToListAsync();

    var revenue = await db.Payments
.Where(p => p.CourseId == id &&
p.Status == PaymentStatus.Completed)
.SumAsync(p => (decimal?)p.Amount)
?? 0m;

    var avgRating = await db.Reviews

```

```

.Where(r => r.Enrollment!.CourseId
== id)
.Select(r => (double?)r.Grade)
.AverageAsync() ?? 0d;

    var completionRate = enrollments.Count
== 0
    ? 0d
    : (double)enrollments.Count(e =>
e.Progress == 100) / enrollments.Count * 100d;

    var recent = enrollments
.OrderByDescending(e
=>
e.EnrolledAt)
.Take(10)
.Select(e
=>
new
AnalyticsRecentStudentDto(e.FirstName,
e.LastName, e.EnrolledAt, e.Progress))
.ToList();

    return new CourseAnalyticsDto(
        course.Id,
        course.Title,
        enrollments.Count,
        revenue,
        avgRating,
        completionRate,
        recent);
}

[HttpGet("api/teacher/courses/{id:int}/enrollme
nts-timeline")]
public async
Task<ActionResult<List<TimelinePointDto>>>
Timeline(
    int id, [FromQuery] DateTime? from,
[FromQuery] DateTime? to)
{
    var teacherId = CurrentUserId;
    var owns = await db.Courses.AnyAsync(c =>
c.Id == id && c.AuthorId == teacherId);
    if (!owns) return NotFound();

    var toDate = DateTime.SpecifyKind((to ??
DateTime.UtcNow).Date, DateTimeKind.Utc);
    var fromDate =
DateTime.SpecifyKind((from ?? toDate.AddDays(-
29)).Date, DateTimeKind.Utc);
    if (fromDate > toDate) return
BadRequest(new { error = "Start date cannot be
after end date." });
    var toExclusive = toDate.AddDays(1);
    var totalDays = (toExclusive -
fromDate).Days;

    var rows = await db.Enrollments
.Where(e => e.CourseId == id &&
e.EnrolledAt >= fromDate && e.EnrolledAt <
toExclusive)
.GroupBy(e => e.EnrolledAt.Date)
.Select(g => new { Date = g.Key,
Count = g.Count() })
.ToListAsync();

    var byDate = rows.ToDictionary(r =>
r.Date, r => r.Count);
    var result = new
List<TimelinePointDto>(totalDays);
    for (var i = 0; i < totalDays; i++)
    {
        var d = fromDate.AddDays(i);
        result.Add(new TimelinePointDto(d,
byDate.GetValueOrDefault(d, 0)));
    }
    return result;
}

```

```

[HttpGet("api/teacher/reviews")]
public async
Task<ActionResult<PagedResult<TeacherReviewDto>
>> Reviews(
    [FromQuery] int? courseId, [FromQuery]
int page = 1, [FromQuery] int pageSize = 20)
{
    var teacherId = CurrentUserId;
    var query = db.Reviews
        .Where(r =>
r.Enrollment!.Course!.AuthorId == teacherId);
    if (courseId is not null) query =
query.Where(r => r.Enrollment!.CourseId ==
courseId);

    page = Math.Max(1, page);
    pageSize = Math.Clamp(pageSize, 1, 100);
    var totalCount = await
query.CountAsync();

    var items = await query
        .OrderByDescending(r => r.CreatedAt)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(r => new TeacherReviewDto(
            r.Enrollment!.CourseId,
            r.Enrollment.Course!.Title,

r.Enrollment.Student!.FirstName,
            r.Enrollment.Student.LastName,
            r.Grade,
            r.Comment,
            r.CreatedAt))
        .ToListAsync();

    return new
PagedResult<TeacherReviewDto>(items, totalCount,
page, pageSize);
}

```

```

//File: Controllers/CategoriesController.cs
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

```

```

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/categories")]
[Authorize]
public class CategoriesController(AppDbContext
db) : ControllerBase
{
    [HttpGet]
    public async
Task<ActionResult<List<CategoryDto>>> Get() =>
        await db.Categories
            .OrderBy(c => c.Name)
            .Select(c => new CategoryDto(c.Id,
c.Name))
            .ToListAsync();
}

```

```

//File: Controllers/TeacherCoursesController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

```

```

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Authorize(Roles = Roles.Teacher)]
public class
TeacherCoursesController(AppDbContext db) :
ControllerBase
{
    private int CurrentUserId =>
int.Parse(User.FindFirstValue(ClaimTypes.NameId
entifier)!);

```

```

[HttpGet("api/teacher/courses")]
public async
Task<ActionResult<List<TeacherCourseListItemDto
>>> List()
{
    var teacherId = CurrentUserId;
    var rows = await db.Courses
        .Where(c => c.AuthorId == teacherId)
        .OrderByDescending(c => c.CreatedAt)
        .Select(c => new
TeacherCourseListItemDto(
            c.Id,
            c.Title,
            c.Status,
            c.Level,
            c.Price,
            db.Enrollments.Count(e
=>
e.CourseId == c.Id),
            db.Reviews
                .Where(r
=>
r.Enrollment!.CourseId == c.Id)
                .Select(r
=>
(double?)r.Grade).Average() ?? 0d,
            c.Lessons.Count,
            c.CreatedAt))
        .ToListAsync();
    return rows;
}

```

```

[HttpGet("api/teacher/courses/{id:int}")]
public async
Task<ActionResult<TeacherCourseDetailDto>>
Get(int id)
{
    var teacherId = CurrentUserId;
    var course = await db.Courses
        .Include(c => c.Categories)
        .Include(c => c.Lessons.OrderBy(l =>
l.OrderNumber))
        .SingleOrDefaultAsync(c => c.Id ==
id && c.AuthorId == teacherId);
    if (course is null) return NotFound();

    return new TeacherCourseDetailDto(
        course.Id,
        course.Title,
        course.Description,
        course.Price,
        course.Level,
        course.Status,
        course.CreatedAt,
        course.Categories.Select(c => new
CategoryDto(c.Id, c.Name)).ToList(),
        course.Lessons.Select(l => new
LessonEditDto(l.Id, l.OrderNumber, l.Title,
l.Content)).ToList());
}

```

```

[HttpPost("api/teacher/courses")]
public async
Task<ActionResult<TeacherCourseDetailDto>>
Create(CreateCourseRequest req)
{

```

```

        if
        (!CourseLevel.All.Contains(req.Level))
            return BadRequest(new { error =
"Invalid level." });
        if (req.Price < 0)
            return BadRequest(new { error =
"Price cannot be negative." });
        if
        (string.IsNullOrEmpty(req.Title))
            return BadRequest(new { error =
"Title is required." });

        var teacherId = CurrentUserId;
        var categories = await db.Categories
            .Where(c
                req.CategoryIds.Contains(c.Id))
            .ToListAsync();

        var course = new Course
        {
            AuthorId = teacherId,
            Title = req.Title.Trim(),
            Description = req.Description ??
string.Empty,
            Price = req.Price,
            Level = req.Level,
            Status = CourseStatus.Draft,
            Categories = categories,
        };
        db.Courses.Add(course);
        await db.SaveChangesAsync();

        return await Get(course.Id);
    }

    [HttpPut("api/teacher/courses/{id:int}")]
    public async Task<ActionResult<TeacherCourseDetailDto>>
        Update(int id, UpdateCourseRequest req)
    {
        if
        (!CourseLevel.All.Contains(req.Level))
            return BadRequest(new { error =
"Invalid level." });
        if (req.Price < 0)
            return BadRequest(new { error =
"Price cannot be negative." });

        var teacherId = CurrentUserId;
        var course = await db.Courses
            .Include(c => c.Categories)
            .SingleOrDefaultAsync(c => c.Id ==
id && c.AuthorId == teacherId);
        if (course is null) return NotFound();

        course.Title = req.Title.Trim();
        course.Description = req.Description ??
string.Empty;
        course.Price = req.Price;
        course.Level = req.Level;

        var newCats = await
        db.Categories.Where(c
            req.CategoryIds.Contains(c.Id)).ToListAsync();
        course.Categories.Clear();
        foreach (var c in newCats)
            course.Categories.Add(c);

        await db.SaveChangesAsync();
        return await Get(id);
    }

    [HttpDelete("api/teacher/courses/{id:int}")]
    public async Task<IActionResult> Delete(int
id)
    {

```

```

        var teacherId = CurrentUserId;
        var course = await
        db.Courses.SingleOrDefaultAsync(c => c.Id == id
&& c.AuthorId == teacherId);
        if (course is null) return NotFound();

        var hasEnrollments = await
        db.Enrollments.AnyAsync(e => e.CourseId == id);
        if (hasEnrollments)
            return BadRequest(new { error =
"Cannot delete a course that has enrollments."
});

        db.Courses.Remove(course);
        await db.SaveChangesAsync();
        return NoContent();
    }

    [HttpPost("api/teacher/courses/{id:int}/publish
")]
    public async Task<IActionResult> Publish(int
id)
    {
        var teacherId = CurrentUserId;
        var course = await db.Courses
            .Include(c => c.Lessons)
            .SingleOrDefaultAsync(c => c.Id ==
id && c.AuthorId == teacherId);
        if (course is null) return NotFound();

        if (course.Lessons.Count == 0)
            return BadRequest(new { error =
"Cannot publish a course with no lessons." });

        course.Status = CourseStatus.Published;
        await db.SaveChangesAsync();
        return NoContent();
    }

    // lessons

    [HttpPost("api/teacher/courses/{courseId:int}/l
essons")]
    public async
    Task<ActionResult<LessonEditDto>> AddLesson(int
courseId, CreateLessonRequest req)
    {
        var teacherId = CurrentUserId;
        var course = await db.Courses
            .Include(c => c.Lessons)
            .SingleOrDefaultAsync(c => c.Id ==
courseId && c.AuthorId == teacherId);
        if (course is null) return NotFound();

        if
        (string.IsNullOrEmpty(req.Title))
            return BadRequest(new { error =
"Title is required." });

        var nextOrder = (course.Lessons.Count ==
0 ? 0 : course.Lessons.Max(l => l.OrderNumber))
+ 1;
        var lesson = new Lesson
        {
            CourseId = courseId,
            OrderNumber = nextOrder,
            Title = req.Title.Trim(),
            Content = req.Content ??
string.Empty,
        };
        db.Lessons.Add(lesson);
        await db.SaveChangesAsync();
    }

```

```

        return new LessonEditDto(lesson.Id,
lesson.OrderNumber, lesson.Title,
lesson.Content);
    }

    [HttpPut("api/teacher/lessons/{id:int}")]
    public async Task<ActionResult<LessonEditDto>>
UpdateLesson(int id, UpdateLessonRequest req)
    {
        var teacherId = CurrentUserId;
        var lesson = await db.Lessons
            .Include(l => l.Course)
            .SingleOrDefaultAsync(l => l.Id ==
id);
        if (lesson is null ||
lesson.Course!.AuthorId != teacherId) return
NotFound();

        if
(string.IsNullOrEmpty(req.Title))
            return BadRequest(new { error =
"Title is required." });

        lesson.Title = req.Title.Trim();
        lesson.Content = req.Content ??
string.Empty;
        await db.SaveChangesAsync();

        return new LessonEditDto(lesson.Id,
lesson.OrderNumber, lesson.Title,
lesson.Content);
    }

    [HttpDelete("api/teacher/lessons/{id:int}")]
    public async Task<ActionResult>
DeleteLesson(int id)
    {
        var teacherId = CurrentUserId;
        var lesson = await db.Lessons
            .Include(l => l.Course)
            .SingleOrDefaultAsync(l => l.Id ==
id);
        if (lesson is null ||
lesson.Course!.AuthorId != teacherId) return
NotFound();

        var hasProgress = await
db.LessonProgress.AnyAsync(lp => lp.LessonId ==
id);
        if (hasProgress)
            return BadRequest(new { error =
"Cannot delete a lesson that students have
already started." });

        if (lesson.Course.Status ==
CourseStatus.Published)
        {
            var lessonCount = await
db.Lessons.CountAsync(l => l.CourseId ==
lesson.CourseId);
            if (lessonCount <= 1)
                return BadRequest(new { error =
"A published course must have at least one
lesson. Add another lesson first." });
        }

        var courseId = lesson.CourseId;
        var deletedOrder = lesson.OrderNumber;
        db.Lessons.Remove(lesson);
        await db.SaveChangesAsync();

        var laterLessons = await db.Lessons
            .Where(l => l.CourseId == courseId &&
l.OrderNumber > deletedOrder)
            .OrderBy(l => l.OrderNumber)

```

```

                .ToListAsync();
                foreach (var l in laterLessons)
                    l.OrderNumber -= 1;
                await db.SaveChangesAsync();

                return NoContent();
            }

            [HttpPut("api/teacher/courses/{courseId:int}/le
ssons/reorder")]
            public async Task<ActionResult> Reorder(int
courseId, ReorderLessonsRequest req)
            {
                var teacherId = CurrentUserId;
                var course = await db.Courses
                    .Include(c => c.Lessons)
                    .SingleOrDefaultAsync(c => c.Id ==
courseId && c.AuthorId == teacherId);
                if (course is null) return NotFound();

                var existingIds = =
course.Lessons.Select(l => l.Id).ToHashSet();
                if (req.LessonIds.Length !=
course.Lessons.Count || req.LessonIds.Any(id =>
!existingIds.Contains(id)))
                    return BadRequest(new { error =
"Reorder list must contain exactly the course's
lesson ids." });

                await using var tx = await
db.Database.BeginTransactionAsync();
                var lessonsById = =
course.Lessons.ToDictionary(l => l.Id);

                // Hack
                // shift everything after the deleted
                lesson down by 1.
                // change everything to negative slots
                var temp = -1;
                foreach (var l in course.Lessons)
                    l.OrderNumber = temp--;
                await db.SaveChangesAsync();

                // assign final order
                for (var i = 0; i < req.LessonIds.Length;
i++)
                    lessonsById[req.LessonIds[i]].OrderNumber = i +
1;

                await db.SaveChangesAsync();

                await tx.CommitAsync();
                return NoContent();
            }
        }

        //File: Controllers/TeacherEarningsController.cs
        using System.Security.Claims;
        using LearningPlatform.Api.Data;
        using LearningPlatform.Api.Dtos;
        using LearningPlatform.Api.Models;
        using Microsoft.AspNetCore.Authorization;
        using Microsoft.AspNetCore.Mvc;
        using Microsoft.EntityFrameworkCore;

        namespace LearningPlatform.Api.Controllers;

        [ApiController]
        [Authorize(Roles = Roles.Teacher)]
        public class
TeacherEarningsController(AppDbContext db) :
ControllerBase
    {

```

```

        private int CurrentUserId =>
int.Parse(User.FindFirstValue(ClaimTypes.NameId
entifier));

        [HttpGet("api/teacher/earnings")]
        public async Task<ActionResult<EarningsDto>>
Get()
        {
            var teacherId = CurrentUserId;
            var profile = await
db.TeacherProfiles.FindAsync(teacherId);
            if (profile is null) return NotFound();

            var history = await db.Payouts
.Where(p => p.TeacherId ==
teacherId)
.OrderByDescending(p
=>
p.RequestedAt)
.Select(p => new
PayoutHistoryItemDto(p.Id, p.Amount, p.Status,
p.RequestedAt))
.ToListAsync();

            return new EarningsDto(profile.Balance,
history);
        }

        [HttpPost("api/teacher/payouts")]
        public async
Task<ActionResult<PayoutHistoryItemDto>>
RequestPayout(CreatePayoutRequest req)
        {
            if (req.Amount <= 0)
                return BadRequest(new { error =
"Amount must be greater than zero." });

            var teacherId = CurrentUserId;
            var profile = await
db.TeacherProfiles.FindAsync(teacherId);
            if (profile is null) return NotFound();

            if (profile.Balance < req.Amount)
                return BadRequest(new { error =
"Insufficient balance." });

            var payout = new Payout
            {
                TeacherId = teacherId,
                Amount = req.Amount,
                Status = PayoutStatus.Pending,
            };
            db.Payouts.Add(payout);
            await db.SaveChangesAsync();
            // Trigger trg_payouts_balance
recomputes profile.balance automatically.

            return new
PayoutHistoryItemDto(payout.Id, payout.Amount,
payout.Status, payout.RequestedAt);
        }
    }

//File:
Controllers/MyTeacherProfileController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/me/teacher-profile")]
[Authorize(Roles = Roles.Teacher)]

```

```

        public class
MyTeacherProfileController(AppDbContext db) :
ControllerBase
        {
            private int CurrentUserId =>
int.Parse(User.FindFirstValue(ClaimTypes.NameId
entifier));

            [HttpGet]
            public async
Task<ActionResult<MyTeacherProfileDto>> Get()
            {
                var profile = await
db.TeacherProfiles.FindAsync(CurrentUserId);
                if (profile is null) return NotFound();
                return new
MyTeacherProfileDto(profile.Bio,
profile.Balance);
            }

            [HttpPut]
            public async
Task<ActionResult<MyTeacherProfileDto>>
Update(UpdateMyTeacherProfileRequest req)
            {
                var profile = await
db.TeacherProfiles.FindAsync(CurrentUserId);
                if (profile is null) return NotFound();

                profile.Bio = req.Bio ?? string.Empty;
                await db.SaveChangesAsync();
                return new
MyTeacherProfileDto(profile.Bio,
profile.Balance);
            }
        }

//File: Controllers/AdminUsersController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/admin/users")]
[Authorize(Roles = Roles.Admin)]
public class AdminUsersController(AppDbContext
db) : ControllerBase
    {
        private int CurrentUserId =>
int.Parse(User.FindFirstValue(ClaimTypes.NameId
entifier));

        [HttpGet]
        public async
Task<ActionResult<PagedResult<AdminUserDto>>>
List(
            [FromQuery] string? role,
            [FromQuery] string? status,
            [FromQuery] string? q,
            [FromQuery] int page = 1,
            [FromQuery] int pageSize = 20)
        {
            var query = db.Users.AsQueryable();
            if (!string.IsNullOrEmpty(role))
                query = query.Where(u => u.Role == role);
            if (!string.IsNullOrEmpty(status))
                query = query.Where(u => u.Status == status);
            if (!string.IsNullOrEmpty(q))
            {
                var term = $"{q.Trim()}%";

```

```

        query = query.Where(u =>
            EF.Functions.ILike(u.Email,
term) ||
            EF.Functions.ILike(u.FirstName +
" " + u.LastName, term));
    }

    page = Math.Max(1, page);
    pageSize = Math.Clamp(pageSize, 1, 100);
    var totalCount = await
query.CountAsync();

    var items = await query
        .OrderBy(u => u.Id)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(u => new AdminUserDto(u.Id,
u.FirstName, u.LastName, u.Email, u.Role,
u.Status))
        .ToListAsync();

    return new
PagedResult<AdminUserDto>(items, totalCount,
page, pageSize);
}

[HttpPut("{id:int}/status")]
public async Task<ActionResult>
UpdateStatus(int id, UpdateUserStatusRequest
req)
{
    if
(!UserStatus.All.Contains(req.Status))
        return BadRequest(new { error =
"Invalid status." });

    if (id == CurrentUserId)
        return BadRequest(new { error = "You
cannot change your own status." });

    var user = await db.Users.FindAsync(id);
    if (user is null) return NotFound();

    if (user.Role == Roles.Admin)
        return BadRequest(new { error =
"Admin accounts cannot be suspended." });

    user.Status = req.Status;
    await db.SaveChangesAsync();
    return NoContent();
}
}

//File: Controllers/AuthController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authentication;
using
Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/auth")]
public class AuthController(AppDbContext db,
IPasswordHasher<User> hasher) : ControllerBase
{
    [HttpPost("register")]
    public async
Task<ActionResult<UserResponse>>
Register(RegisterRequest req)

```

```

    {
        if
(!Roles.SelfRegisterable.Contains(req.Role))
            return BadRequest(new { error = "Role
must be Student or Teacher." });

        var email =
req.Email.Trim().ToLowerInvariant();
        if (await db.Users.AnyAsync(u => u.Email
== email))
            return BadRequest(new { error =
"Email is already registered." });

        if (req.Password.Length < 6)
            return BadRequest(new { error =
"Password must be at least 6 characters." });

        var user = new User
        {
            Email = email,
            FirstName = req.FirstName.Trim(),
            LastName = req.LastName.Trim(),
            Role = req.Role,
        };
        user.PasswordHash =
hasher.HashPassword(user, req.Password);

        db.Users.Add(user);

        if (req.Role == Roles.Teacher)
            db.TeacherProfiles.Add(new
TeacherProfile { User = user });

        await db.SaveChangesAsync();
        await SignIn(user);
        return ToResponse(user);
    }

    [HttpPost("login")]
    public async
Task<ActionResult<UserResponse>>
Login(LoginRequest req)
    {
        var email =
req.Email.Trim().ToLowerInvariant();
        var user = await
db.Users.SingleOrDefaultAsync(u => u.Email ==
email);
        if (user is null)
            return Unauthorized(new { error =
"Invalid credentials." });

        if (user.Status == UserStatus.Suspended)
            return Unauthorized(new { error =
"This account has been suspended." });

        var result =
hasher.VerifyHashedPassword(user,
user.PasswordHash, req.Password);
        if (result ==
PasswordVerificationResult.Failed)
            return Unauthorized(new { error =
"Invalid credentials." });

        if (result ==
PasswordVerificationResult.SuccessRehashNeeded)
        {
            user.PasswordHash =
hasher.HashPassword(user, req.Password);
            await db.SaveChangesAsync();
        }

        await SignIn(user);
        return ToResponse(user);
    }

    [HttpPost("logout")]

```

```

    public async Task<IActionResult> Logout()
    {
        await
HttpContext.SignOutAsync(CookieAuthenticationDe
faults.AuthenticationScheme);
        return NoContent();
    }

    private async Task SignIn(User user)
    {
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier,
user.Id.ToString()),
            new(ClaimTypes.Email, user.Email),
            new(ClaimTypes.Role, user.Role),
        };
        var identity = new
ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationSche
me);
        await HttpContext.SignInAsync(
CookieAuthenticationDefaults.AuthenticationSche
me,
            new ClaimsPrincipal(identity),
            new AuthenticationProperties {
IsPersistent = true });
    }

    private static UserResponse ToResponse(User
u) =>
        new(u.Id, u.Email, u.FirstName,
u.LastName, u.Role);
    }

//File: Controllers/AuthorsController.cs
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/authors")]
[Authorize] // any logged-in user can view an
author profile
public class AuthorsController(AppDbContext db)
: ControllerBase
{
    [HttpGet("{id:int}")]
    public async
Task<ActionResult<AuthorPublicDto>> Get(int id)
    {
        var profile = await db.TeacherProfiles
.Include(p => p.User)
.SingleOrDefaultAsync(p => p.UserId
== id);
        if (profile is null || profile.User is
null) return NotFound();

        var courses = await db.Courses
.Where(c => c.AuthorId == id &&
c.Status == CourseStatus.Published)
.OrderByDescending(c => c.CreatedAt)
.Select(c => new
AuthorPublicCourseDto(
            c.Id,
            c.Title,
            c.Price,
            c.Level,
            c.CreatedAt,
            db.Reviews.Where(r
=> r.Enrollment!.CourseId == c.Id).Select(r
=> (double?)r.Grade).Average() ?? 0d,
            db.Reviews.Count(r
=> r.Enrollment!.CourseId == c.Id)))
.ToListAsync();

        return new AuthorPublicDto(
            profile.UserId,
            profile.User.FirstName,
            profile.User.LastName,
            profile.Bio,
            courses);
    }
}

//File: Controllers/MeController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authentication;
using
Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
// using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Route("api/me")]
[Authorize]
public class MeController(AppDbContext db) :
ControllerBase
{
    [HttpGet]
    public async
Task<ActionResult<UserResponse>> Get()
    {
        var idClaim =
User.FindFirstValue(ClaimTypes.NameIdentifier);
        if (!int.TryParse(idClaim, out var id))
return Unauthorized();

        var user = await db.Users.FindAsync(id);
        if (user is null) return Unauthorized();

        //kill cookie
        if (user.Status == UserStatus.Suspended)
        {
            await
HttpContext.SignOutAsync(CookieAuthenticationDe
faults.AuthenticationScheme);
            return Unauthorized(new { error =
"This account has been suspended." });
        }

        return new UserResponse(user.Id,
user.Email, user.FirstName, user.LastName,
user.Role);
    }
}

//File: Controllers/AdminPayoutsController.cs
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]

```

```
[Route("api/admin/payouts")]
[Authorize(Roles = Roles.Admin)]
public class AdminPayoutsController(AppDbContext db) : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<PagedResult<AdminPayoutDto>>>
    List(
        [FromQuery] string? status, [FromQuery]
        string? q,
        [FromQuery] int page = 1, [FromQuery] int
        pageSize = 20)
    {
        var query = db.Payouts.AsQueryable();
        if (!string.IsNullOrEmpty(status))
            query = query.Where(p => p.Status ==
        status);
        if (!string.IsNullOrEmpty(q))
        {
            var term = $"{q.Trim()}%";
            query = query.Where(p =>

        EF.Functions.ILike(p.Teacher!.User!.FirstName +
        " " + p.Teacher.User.LastName, term) ||

        EF.Functions.ILike(p.Teacher!.User!.Email,
        term));
        }

        page = Math.Max(1, page);
        pageSize = Math.Clamp(pageSize, 1, 100);
        var totalCount = await
        query.CountAsync();

        var items = await query
            .OrderByDescending(p
            p.RequestedAt)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Select(p => new AdminPayoutDto(
                p.Id,
                p.RequestedAt,
                p.TeacherId,
                p.Teacher!.User!.FirstName + " "
        + p.Teacher.User.LastName,
                p.Teacher.User.Email,
                p.Amount,
                p.Status))
            .ToListAsync();

        return new
        PagedResult<AdminPayoutDto>(items, totalCount,
        page, pageSize);
    }

    [HttpPost("{id:int}/approve")]
    public Task<IActionResult> Approve(int id)
    => Transition(id, PayoutStatus.Pending,
        PayoutStatus.Approved);

    [HttpPost("{id:int}/reject")]
    public Task<IActionResult> Reject(int id) =>
        Transition(id, PayoutStatus.Pending,
        PayoutStatus.Rejected);

    [HttpPost("{id:int}/mark-paid")]
    public Task<IActionResult> MarkPaid(int id)
    => Transition(id, PayoutStatus.Approved,
        PayoutStatus.Paid);

    private async Task<IActionResult>
    Transition(int id, string fromStatus, string
    toStatus)
    {
        var payout = await
        db.Payouts.FindAsync(id);
```

```
        if (payout is null) return NotFound();
        if (payout.Status != fromStatus)
            return BadRequest(new { error =
        $"Payout is not in '{fromStatus}' state." });

        payout.Status = toStatus;
        await db.SaveChangesAsync();
        // trg_payouts_balance triggers
        automatically
        return NoContent();
    }
}

//File: Controllers/AdminFinanceController.cs
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Authorize(Roles = Roles.Admin)]
public class AdminFinanceController(AppDbContext
    db) : ControllerBase
{
    [HttpGet("api/admin/finance-dashboard")]
    public async Task<ActionResult<AdminFinanceDashboardDto>>
    Dashboard(
        [FromQuery] DateTime? from,
        [FromQuery] DateTime? to)
    {
        // Defaults: last 30 days, inclusive on
        both ends.
        // Postgres `timestamp with time zone`
        requires DateTimeKind.Utc, so
        // strip whatever kind the model binder
        produced and re-stamp it as UTC.
        var toDate = DateTime.SpecifyKind((to ??
        DateTime.UtcNow).Date, DateTimeKind.Utc);
        var fromDate =
        DateTime.SpecifyKind((from ?? toDate.AddMonths(-
        1)).Date, DateTimeKind.Utc);

        if (fromDate > toDate) return
        BadRequest(new { error = "Start date cannot be
        after end date." });

        // inclusive <= comparison
        var toExclusive = toDate.AddDays(1);

        var totalDays = (toExclusive -
        fromDate).Days;

        var grossRevenue = await db.Payments
            .Where(p => p.Status ==
        PaymentStatus.Completed
            && p.CreatedAt >= fromDate
            && p.CreatedAt < toExclusive)
            .SumAsync(p => (decimal?)p.Amount)
            ?? 0m;

        var totalRefunds = await db.Payments
            .Where(p => p.Status ==
        PaymentStatus.Refunded
            && p.CreatedAt >= fromDate
            && p.CreatedAt < toExclusive)
            .SumAsync(p => (decimal?)p.Amount)
            ?? 0m;

        var paidToTeachers = await db.Payouts
            .Where(p => p.Status ==
        PayoutStatus.Paid
```

```

        && p.RequestedAt >=
fromDate && p.RequestedAt < toExclusive)
        .SumAsync(p => (decimal?)p.Amount)
?? 0m;

var pendingPayouts = await db.Payouts
    .Where(p => p.Status ==
PayoutStatus.Pending)
    .ToListAsync();

var revenueRows = await db.Payments
    .Where(p => p.Status ==
PaymentStatus.Completed
    && p.CreatedAt >= fromDate
    && p.CreatedAt < toExclusive)
    .GroupBy(p => p.CreatedAt.Date)
    .Select(g => new { Date = g.Key,
Amount = g.Sum(p => p.Amount) })
    .ToListAsync();

var payoutRows = await db.Payouts
    .Where(p => p.Status ==
PayoutStatus.Paid
    && p.RequestedAt >=
fromDate && p.RequestedAt < toExclusive)
    .GroupBy(p => p.RequestedAt.Date)
    .Select(g => new { Date = g.Key,
Amount = g.Sum(p => p.Amount) })
    .ToListAsync();

var revByDate =
revenueRows.ToDictionary(r => r.Date, r =>
r.Amount);
var payByDate =
payoutRows.ToDictionary(r => r.Date, r =>
r.Amount);

var revenueTimeline = new
List<TimelinePointMoneyDto>(totalDays);
var payoutsTimeline = new
List<TimelinePointMoneyDto>(totalDays);
for (var i = 0; i < totalDays; i++)
{
    var d = fromDate.AddDays(i);
    revenueTimeline.Add(new
TimelinePointMoneyDto(d,
revByDate.GetValueOrDefault(d, 0m)));
    payoutsTimeline.Add(new
TimelinePointMoneyDto(d,
payByDate.GetValueOrDefault(d, 0m)));
}

return new AdminFinanceDashboardDto(
    grossRevenue,
    totalRefunds,
    paidToTeachers,
    pendingPayouts.Sum(p => p.Amount),
    pendingPayouts.Count,
    revenueTimeline,
    payoutsTimeline);
}

[HttpGet("api/admin/payments")]
public async Task<ActionResult<PagedResult<AdminPaymentDto>>
> ListPayments(
    [FromQuery] string? status, [FromQuery]
string? q,
    [FromQuery] int page = 1, [FromQuery] int
pageSize = 20)
{
    var query = db.Payments.AsQueryable();
    if (!string.IsNullOrEmpty(status))
        query = query.Where(p => p.Status ==
status);
    if (!string.IsNullOrEmpty(q))
    {

```

```

        var term = $"{q.Trim()}%";
        query = query.Where(p =>

EF.Functions.ILike(p.Course!.Title, term) ||

EF.Functions.ILike(p.Student!.FirstName + " " +
p.Student.LastName, term));
    }

    page = Math.Max(1, page);
    pageSize = Math.Clamp(pageSize, 1, 100);
    var totalCount = await
query.CountAsync();

    var items = await query
        .OrderByDescending(p => p.CreatedAt)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(p => new AdminPaymentDto(
            p.Id,
            p.CreatedAt,
            p.StudentId,
            p.Student!.FirstName + " " +
p.Student.LastName,
            p.Student.Email,
            p.CourseId,
            p.Course!.Title,
            p.Amount,
            p.Status))
        .ToListAsync();

    return new
PagedResult<AdminPaymentDto>(items, totalCount,
page, pageSize);
}

[HttpPost("api/admin/payments/{id:int}/refund")]
public async Task<ActionResult> Refund(int
id)
{
    var payment = await
db.Payments.FindAsync(id);
    if (payment is null) return NotFound();
    if (payment.Status ==
PaymentStatus.Refunded)
        return BadRequest(new { error =
"Payment is already refunded." });

    //transactin
    await using var tx = await
db.Database.BeginTransactionAsync();

    // triggers trg_payments_balance
recomputes the teacher's balance.
    payment.Status = PaymentStatus.Refunded;

    // cascade-delete with the enrollment
    var enrollment = await db.Enrollments
        .SingleOrDefaultAsync(e =>
e.StudentId == payment.StudentId && e.CourseId ==
payment.CourseId);
    if (enrollment is not null)
        db.Enrollments.Remove(enrollment);

    await db.SaveChangesAsync();
    await tx.CommitAsync();

    return NoContent();
}

//File: Controllers/EnrollmentsController.cs
using System.Security.Claims;
using LearningPlatform.Api.Data;

```

```

using LearningPlatform.Api.Dtos;
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Controllers;

[ApiController]
[Authorize(Roles = Roles.Student)]
public class EnrollmentsController(AppDbContext db) : ControllerBase
{
    private int CurrentUserId =>
    int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier));

    [HttpGet("api/me/recommendations")]
    public async Task<ActionResult<List<RecommendedCourseDto>>>
    Recommendations()
    {
        var studentId = CurrentUserId;

        // call the sql function
        var rawResults = await db.Database

.SqlQuery<RawRecommendation>($"SELECT * FROM
get_recommendations({studentId}, 6)")
.ToListAsync();

        if (rawResults.Count == 0) return new
List<RecommendedCourseDto>();

        var courseIds = rawResults.Select(r =>
r.course_id).ToList();
        var reasonMap =
rawResults.ToDictionary(r => r.course_id, r =>
(r.score, r.reason));

        // load all data
        var courses = await db.Courses
        .Where(c =>
courseIds.Contains(c.Id))
        .Include(c =>
c.Author!).ThenInclude(a => a.User)
        .Include(c => c.Categories)
        .ToListAsync();

        var result = new
List<RecommendedCourseDto>();
        foreach (var rec in rawResults)
        {
            var c = courses.FirstOrDefault(c =>
c.Id == rec.course_id);
            if (c is null) continue;

            var avgRating = await db.Reviews
            .Where(r =>
r.Enrollment!.CourseId == c.Id)
            .Select(r => (double?)r.Grade)
            .AverageAsync() ?? 0d;

            var reviewCount = await
db.Reviews.CountAsync(r =>
r.Enrollment!.CourseId == c.Id);
            var enrollmentCount = await
db.Enrollments.CountAsync(e => e.CourseId ==
c.Id);

            result.Add(new RecommendedCourseDto(
                c.Id, c.Title, c.Price, c.Level,
                c.CreatedAt,
                new AuthorDto(c.Author!.UserId,
                c.Author.User!.FirstName,
                c.Author.User.LastName),
                c.Categories.Select(cat => new
CategoryDto(cat.Id, cat.Name)).ToList(),

```

```

                avgRating,
                reviewCount,
                enrollmentCount,
                (double)rec.score,
                (double)rec.confidence,
                (double)rec.lift,
                rec.reason));
        }

        return result;
    }

    // raw result structure from the sql function
    private class RawRecommendation
    {
        public int course_id { get; set; }
        public decimal score { get; set; }
        public decimal confidence { get; set; }
        public decimal lift { get; set; }
        public string reason { get; set; } = "";
    }

    [HttpGet("api/me/enrollments")]
    public async Task<ActionResult<List<EnrollmentListItemDto>>>
    List()
    {
        var studentId = CurrentUserId;
        var rows = await db.Enrollments
        .Where(e => e.StudentId ==
studentId)
        .OrderByDescending(e =>
e.EnrolledAt)
        .Select(e => new
        {
            e.Id,
            e.CourseId,
            CourseTitle = e.Course!.Title,
            e.Progress,
            e.EnrolledAt,
            NextLessonId = e.Course.Lessons
            .Where(l =>
!db.LessonProgress.Any(lp => lp.EnrollmentId ==
e.Id && lp.LessonId == l.Id))
            .OrderBy(l => l.OrderNumber)
            .Select(l => (int?)l.Id)
            .FirstOrDefault(),
        })
        .ToListAsync();

        return rows.Select(r => new
EnrollmentListItemDto(
            r.Id, r.CourseId, r.CourseTitle,
            r.Progress, r.NextLessonId,
            r.EnrolledAt)).ToList();
    }

    [HttpGet("api/me/enrollments/{id:int}")]
    public async Task<ActionResult<EnrollmentDetailDto>>
    Get(int id)
    {
        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
        .Include(e =>
e.Course!).ThenInclude(c => c.Lessons.OrderBy(l
=> l.OrderNumber))
        .SingleOrDefaultAsync(e => e.Id ==
id && e.StudentId == studentId);

        if (enrollment is null) return
NotFound();

        var completed = await db.LessonProgress
        .Where(lp => lp.EnrollmentId == id)
        .Select(lp => lp.LessonId)
        .ToListAsync();

        var myReview = await db.Reviews

```

```

        .Where(r => r.EnrollmentId == id)
        .Select(r => new
MyReviewDto(r.Grade, r.Comment, r.CreatedAt))
        .SingleOrDefaultAsync();

    return new EnrollmentDetailDto(
        enrollment.Id,
        enrollment.CourseId,
        enrollment.Course!.Title,
        enrollment.Progress,
        enrollment.EnrolledAt,
        enrollment.Course.Lessons.Select(l
=> new LessonSummaryDto(l.Id, l.OrderNumber,
l.Title)).ToList(),
        completed,
        myReview);
    }

[HttpGet("api/me/enrollments/{id:int}/lessons/{
lessonId:int}")]
    public async Task<ActionResult<LessonContentDto>>
GetLesson(int id, int lessonId)
    {
        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
            .SingleOrDefaultAsync(e => e.Id ==
id && e.StudentId == studentId);
        if (enrollment is null) return
NotFound();

        var lesson = await db.Lessons
            .SingleOrDefaultAsync(l => l.Id ==
lessonId && l.CourseId == enrollment.CourseId);
        if (lesson is null) return NotFound();

        return new LessonContentDto(lesson.Id,
lesson.OrderNumber, lesson.Title,
lesson.Content);
    }

[HttpPost("api/courses/{courseId:int}/enroll")]
    public async Task<ActionResult<EnrollResultDto>> Enroll(int
courseId)
    {
        var studentId = CurrentUserId;

        var course = await db.Courses
            .Include(c => c.Lessons.OrderBy(l =>
l.OrderNumber))
            .SingleOrDefaultAsync(c => c.Id ==
courseId && c.Status == CourseStatus.Published);
        if (course is null) return NotFound(new
{ error = "Course not found." });

        if (await db.Enrollments.AnyAsync(e =>
e.StudentId == studentId && e.CourseId ==
courseId))
            return BadRequest(new { error =
"Already enrolled." });

        await using var tx = await
db.Database.BeginTransactionAsync();

        db.Payments.Add(new Payment
        {
            StudentId = studentId,
            CourseId = courseId,
            Amount = course.Price,
            Status = PaymentStatus.Completed,
        });

        var enrollment = new Enrollment
        {

```

```

            StudentId = studentId,
            CourseId = courseId,
        };
        db.Enrollments.Add(enrollment);

        //trigger

        await db.SaveChangesAsync();
        await tx.CommitAsync();

        var firstLessonId =
course.Lessons.FirstOrDefault()?.Id;
        return new
EnrollResultDto(enrollment.Id, firstLessonId);
    }

[HttpDelete("api/enrollments/{id:int}/lessons/{
lessonId:int}/complete")]
    public async Task<ActionResult>
UncompleteLesson(int id, int lessonId)
    {
        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
            .SingleOrDefaultAsync(e => e.Id ==
id && e.StudentId == studentId);
        if (enrollment is null) return
NotFound();

        var row = await db.LessonProgress
            .SingleOrDefaultAsync(lp =>
lp.EnrollmentId == id && lp.LessonId ==
lessonId);
        if (row is null) return NoContent();

        db.LessonProgress.Remove(row);
        await db.SaveChangesAsync();

        // trigger updates enrollments.progress
        return NoContent();
    }

[HttpPost("api/enrollments/{id:int}/lessons/{le
ssonId:int}/complete")]
    public async Task<ActionResult>
CompleteLesson(int id, int lessonId)
    {
        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
            .SingleOrDefaultAsync(e => e.Id ==
id && e.StudentId == studentId);
        if (enrollment is null) return
NotFound();

        var lesson = await db.Lessons
            .SingleOrDefaultAsync(l => l.Id ==
lessonId && l.CourseId == enrollment.CourseId);
        if (lesson is null) return NotFound(new
{ error = "Lesson does not belong to this course."
});

        var already = await
db.LessonProgress.AnyAsync(lp => lp.EnrollmentId
== id && lp.LessonId == lessonId);
        if (already) return NoContent();

        db.LessonProgress.Add(new LessonProgress
{ EnrollmentId = id, LessonId = lessonId });
        await db.SaveChangesAsync();
        //
        // trigger updates enrollments.progress
        return NoContent();
    }

[HttpPut("api/courses/{courseId:int}/reviews")]

```

```

        public async Task<ActionResult<ReviewDto>>
UpdateReview(int courseId, CreateReviewRequest
req)
    {
        if (req.Grade is < 1 or > 5)
            return BadRequest(new { error =
"Grade must be between 1 and 5." });

        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
.Include(e => e.Student)
.SingleOrDefaultAsync(e
=>
e.StudentId == studentId && e.CourseId ==
courseId);
        if (enrollment is null) return
BadRequest(new { error = "You are not enrolled in
this course." });

        var review =
await
db.Reviews.SingleOrDefaultAsync(r
=>
r.EnrollmentId == enrollment.Id);
        if (review is null) return NotFound(new
{ error = "No review to update." });

        review.Grade = req.Grade;
        review.Comment = req.Comment ??
string.Empty;
        await db.SaveChangesAsync();

        return new ReviewDto(review.Grade,
review.Comment, review.CreatedAt,
enrollment.Student!.FirstName,
enrollment.Student.LastName);
    }

[HttpDelete("api/courses/{courseId:int}/reviews
")]
    public async Task<IActionResult>
DeleteReview(int courseId)
    {
        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
.SingleOrDefaultAsync(e
=>
e.StudentId == studentId && e.CourseId ==
courseId);
        if (enrollment is null) return
BadRequest(new { error = "You are not enrolled in
this course." });

        var review =
await
db.Reviews.SingleOrDefaultAsync(r
=>
r.EnrollmentId == enrollment.Id);
        if (review is null) return NoContent();

        db.Reviews.Remove(review);
        await db.SaveChangesAsync();
        return NoContent();
    }

[HttpPost("api/courses/{courseId:int}/reviews")]
    public async Task<ActionResult<ReviewDto>>
CreateReview(int courseId, CreateReviewRequest
req)
    {
        if (req.Grade is < 1 or > 5)
            return BadRequest(new { error =
"Grade must be between 1 and 5." });

        var studentId = CurrentUserId;
        var enrollment = await db.Enrollments
.Include(e => e.Student)
.SingleOrDefaultAsync(e
=>
e.StudentId == studentId && e.CourseId ==
courseId);

```

```

        if (enrollment is null) return
BadRequest(new { error = "You are not enrolled in
this course." });

        if (await db.Reviews.AnyAsync(r
=>
r.EnrollmentId == enrollment.Id))
            return BadRequest(new { error = "You
have already reviewed this course." });

        var review = new Review
        {
            EnrollmentId = enrollment.Id,
            Grade = req.Grade,
            Comment = req.Comment ??
string.Empty,
        };
        db.Reviews.Add(review);
        await db.SaveChangesAsync();

        return new ReviewDto(review.Grade,
review.Comment, review.CreatedAt,
enrollment.Student!.FirstName,
enrollment.Student.LastName);
    }
}

//File: Data/DbSeeder.cs
using LearningPlatform.Api.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Data;

//not used anymore
public static class DbSeeder
{
    public static async Task
SeedAsync(IServiceProvider services)
    {
        var db =
services.GetRequiredService<AppDbContext>();
        var hasher =
services.GetRequiredService<IPasswordHasher<Use
r>>();

        const string adminEmail = "admin@local";
        if (await db.Users.AnyAsync(u => u.Email
== adminEmail)) return;

        var admin = new User
        {
            Email = adminEmail,
            FirstName = "Admin",
            LastName = "User",
            Role = Roles.Admin,
        };
        admin.PasswordHash =
hasher.HashPassword(admin, "admin123");
        db.Users.Add(admin);
        await db.SaveChangesAsync();
    }
}

```

```

//File: Data/AppDbContext.cs
using LearningPlatform.Api.Models;
using Microsoft.EntityFrameworkCore;

namespace LearningPlatform.Api.Data;

public class
AppDbContext (DbContextOptions<AppDbContext>
options) : DbContext(options)
{
    public DbSet<User> Users => Set<User>();
}

```

```

        public DbSet<TeacherProfile> TeacherProfiles
=> Set<TeacherProfile>();
        public DbSet<Course> Courses =>
Set<Course>();
        public DbSet<Category> Categories =>
Set<Category>();
        public DbSet<Lesson> Lessons =>
Set<Lesson>();
        public DbSet<Enrollment> Enrollments =>
Set<Enrollment>();
        public DbSet<LessonProgress> LessonProgress
=> Set<LessonProgress>();
        public DbSet<Review> Reviews =>
Set<Review>();
        public DbSet<Payment> Payments =>
Set<Payment>();
        public DbSet<Payout> Payouts =>
Set<Payout>();

        protected override void
OnModelCreating(ModelBuilder b)
        {
            b.Entity<User>(e =>
            {
                e.HasIndex(u => u.Email).IsUnique();
                e.Property(u
=>
u.FirstName).IsRequired().HasMaxLength(100);
                e.Property(u
=>
u.LastName).IsRequired().HasMaxLength(100);
                e.Property(u
=>
u.Email).IsRequired().HasMaxLength(255);
                e.Property(u
=>
u.PasswordHash).IsRequired();
                e.Property(u
=>
u.Role).IsRequired().HasMaxLength(20);
                e.Property(u
=>
u.Status).IsRequired().HasMaxLength(20).HasDefa
ultValue(UserStatus.Active);
                e.ToTable(t =>
                {
                    t.HasCheckConstraint("ck_users_role", "role IN
('Admin', 'Teacher', 'Student')");
                    t.HasCheckConstraint("ck_users_status", "status
IN ('Active', 'Suspended')");
                });
            });

            b.Entity<TeacherProfile>(e =>
            {
                e.HasKey(p => p.UserId);
                e.Property(p
=>
p.Bio).HasMaxLength(2000);
                e.Property(p
=>
p.Balance).HasColumnType("numeric(10,2)").HasDe
faultValue(0m);
                e.HasOne(p => p.User)
                    .WithOne(u => u.TeacherProfile)
                    .HasForeignKey<TeacherProfile>(p => p.UserId)
                    .OnDelete(DeleteBehavior.Cascade);
            });

            b.Entity<Category>(e =>
            {
                e.HasIndex(c => c.Name).IsUnique();
                e.Property(c
=>
c.Name).IsRequired().HasMaxLength(60);
            });

            b.Entity<Course>(e =>
            {
                e.Property(c
=>
c.Title).IsRequired().HasMaxLength(200);
                e.Property(c
=>
c.Description).IsRequired();
                e.Property(c
=>
c.Price).HasColumnType("numeric(10,2)");
                e.Property(c
=>
c.Level).IsRequired().HasMaxLength(20);
                e.Property(c
=>
c.Status).IsRequired().HasMaxLength(20);
                e.Property(c
=>
c.CreatedAt).HasDefaultValueSql("now()");
                e.HasOne(c => c.Author)
                    .WithMany()
                    .HasForeignKey(c => c.AuthorId)
                    .OnDelete(DeleteBehavior.Restrict);
                e.HasMany(c => c.Categories)
                    .WithMany(cat => cat.Courses)
                    .UsingEntity(j
=>
j.ToTable("course_categories"));
                e.ToTable(t =>
                {
                    t.HasCheckConstraint("ck_courses_status",
"status IN ('Draft', 'Published')");
                    t.HasCheckConstraint("ck_courses_level", "level
IN ('Beginner', 'Intermediate', 'Advanced')");
                    t.HasCheckConstraint("ck_courses_price", "price
>= 0");
                });
            });

            b.Entity<Lesson>(e =>
            {
                e.Property(l
=>
l.Title).IsRequired().HasMaxLength(200);
                e.Property(l
=>
l.Content).IsRequired();
                e.HasOne(l => l.Course)
                    .WithMany(c => c.Lessons)
                    .HasForeignKey(l => l.CourseId)
                    .OnDelete(DeleteBehavior.Cascade);
                e.HasIndex(l => new { l.CourseId,
l.OrderNumber }).IsUnique();
            });

            b.Entity<Enrollment>(e =>
            {
                e.Property(en
=>
en.EnrolledAt).HasDefaultValueSql("now()");
                e.Property(en
=>
en.Progress).HasDefaultValue(0);
                e.HasOne(en => en.Student)
                    .WithMany()
                    .HasForeignKey(en
=>
en.StudentId)
                    .OnDelete(DeleteBehavior.Restrict);
                e.HasOne(en => en.Course)
                    .WithMany()
                    .HasForeignKey(en
=>
en.CourseId)
                    .OnDelete(DeleteBehavior.Restrict);
                e.HasIndex(en => new { en.StudentId,
en.CourseId }).IsUnique();
                e.ToTable(t
=>
t.HasCheckConstraint("ck_enrollments_progress",
"progress BETWEEN 0 AND 100"));
            });

            b.Entity<LessonProgress>(e =>
            {
                e.HasKey(lp
=>
new {
lp.EnrollmentId, lp.LessonId });

```

```

        e.Property(lp
lp.CompletedAt).HasDefaultValueSql("now()");
        e.HasOne(lp => lp.Enrollment)
            .WithMany(en
en.LessonProgress)
            .HasForeignKey(lp
lp.EnrollmentId)

.OnDelete(DeleteBehavior.Cascade);
        e.HasOne(lp => lp.Lesson)
            .WithMany()
            .HasForeignKey(lp
lp.LessonId)

.OnDelete(DeleteBehavior.Restrict);
    });

    b.Entity<Review>(e =>
    {
        e.HasKey(r => r.EnrollmentId);
        e.Property(r
r.Comment).HasMaxLength(2000);
        e.Property(r
r.CreatedAt).HasDefaultValueSql("now()");
        e.HasOne(r => r.Enrollment)
            .WithOne(en => en.Review)
            .HasForeignKey<Review>(r
r.EnrollmentId)

.OnDelete(DeleteBehavior.Cascade);
        e.ToTable(t
t.HasCheckConstraint("ck_reviews_grade", "grade
BETWEEN 1 AND 5"));
    });

    b.Entity<Payment>(e =>
    {
        e.Property(p
p.Amount).HasColumnType("numeric(10,2)");
        e.Property(p
p.Status).IsRequired().HasMaxLength(20);
        e.Property(p
p.CreatedAt).HasDefaultValueSql("now()");
        e.HasOne(p => p.Student)
            .WithMany()
            .HasForeignKey(p => p.StudentId)

.OnDelete(DeleteBehavior.Restrict);
        e.HasOne(p => p.Course)
            .WithMany()
            .HasForeignKey(p => p.CourseId)

.OnDelete(DeleteBehavior.Restrict);
        e.ToTable(t =>
        {
            t.HasCheckConstraint("ck_payments_status",
"status IN ('Completed','Refunded')");

            t.HasCheckConstraint("ck_payments_amount",
"amount >= 0");
        });

        b.Entity<Payout>(e =>
        {
            e.Property(p
p.Amount).HasColumnType("numeric(10,2)");
            e.Property(p
p.Status).IsRequired().HasMaxLength(20);
            e.Property(p
p.RequestedAt).HasDefaultValueSql("now()");
            e.HasOne(p => p.Teacher)
                .WithMany()
                .HasForeignKey(p => p.TeacherId)

.OnDelete(DeleteBehavior.Restrict);
        });

        e.ToTable(t =>
        {
            t.HasCheckConstraint("ck_payouts_status",
"status
IN
('Pending','Approved','Rejected','Paid')");

            t.HasCheckConstraint("ck_payouts_amount",
"amount >= 0");
        });
    }

//File: Dtos/EnrollmentDtos.cs
using System.ComponentModel.DataAnnotations;

namespace LearningPlatform.Api.Dtos;

public record EnrollmentListItemDto(
    int Id,
    int CourseId,
    string CourseTitle,
    int Progress,
    int? NextLessonId,
    DateTime EnrolledAt);

public record LessonProgressDto(int LessonId,
bool Completed);

public record LessonContentDto(int Id, int
OrderNumber, string Title, string Content);

public record MyReviewDto(int Grade, string
Comment, DateTime CreatedAt);

public record EnrollmentDetailDto(
    int Id,
    int CourseId,
    string CourseTitle,
    int Progress,
    DateTime EnrolledAt,
    List<LessonSummaryDto> Lessons,
    List<int> CompletedLessonIds,
    MyReviewDto? MyReview);

public record EnrollResultDto(int EnrollmentId,
int? FirstLessonId);

public record CreateReviewRequest([Range(1, 5)]
int Grade, [MaxLength(2000)] string Comment);

//File: Dtos/PaginationDtos.cs
namespace LearningPlatform.Api.Dtos;

public record PagedResult<T>(List<T> Items, int
TotalCount, int Page, int PageSize);

//File: Dtos/AuthorDtos.cs
namespace LearningPlatform.Api.Dtos;

public record AuthorPublicCourseDto(
    int Id,
    string Title,
    decimal Price,
    string Level,
    DateTime CreatedAt,
    double AverageRating,
    int ReviewCount);

public record AuthorPublicDto(
    int Id,
    string FirstName,
    string LastName,

```

```

        string Bio,
        List<AuthorPublicCourseDto> Courses);

//File: Dtos/CourseDtos.cs
namespace LearningPlatform.Api.Dtos;

public record CategoryDto(int Id, string Name);

public record AuthorDto(int Id, string FirstName,
string LastName);

public record CourseListItemDto(
    int Id,
    string Title,
    decimal Price,
    string Level,
    DateTime CreatedAt,
    AuthorDto Author,
    List<CategoryDto> Categories,
    double AverageRating,
    int ReviewCount,
    int EnrollmentCount);

public record LessonSummaryDto(int Id, int
    OrderNumber, string Title);

public record ReviewDto(
    int Grade,
    string Comment,
    DateTime CreatedAt,
    string StudentFirstName,
    string StudentLastName);

public record CourseDetailDto(
    int Id,
    string Title,
    string Description,
    decimal Price,
    string Level,
    string Status,
    DateTime CreatedAt,
    AuthorDto Author,
    List<CategoryDto> Categories,
    List<LessonSummaryDto> Lessons,
    List<ReviewDto> Reviews,
    double AverageRating,
    int EnrollmentCount);

public record RecommendedCourseDto(
    int Id,
    string Title,
    decimal Price,
    string Level,
    DateTime CreatedAt,
    AuthorDto Author,
    List<CategoryDto> Categories,
    double AverageRating,
    int ReviewCount,
    int EnrollmentCount,
    double Score,
    double Confidence,
    double Lift,
    string Reason);

public record CourseQuery(
    string? Q,
    int[]? CategoryIds,
    decimal? MinPrice,
    decimal? MaxPrice,
    double? MinRating,
    double? MaxRating,
    DateTime? CreatedAfter,
    DateTime? CreatedBefore,
    string? Sort,
    int Page = 1,
    int PageSize = 12);

//File: Dtos/TeacherDtos.cs
using System.ComponentModel.DataAnnotations;

namespace LearningPlatform.Api.Dtos;

// courses

public record TeacherCourseListItemDto(
    int Id,
    string Title,
    string Status,
    string Level,
    decimal Price,
    int EnrollmentCount,
    double AverageRating,
    int LessonCount,
    DateTime CreatedAt);

public record LessonEditDto(int Id, int
    OrderNumber, string Title, string Content);

public record TeacherCourseDetailDto(
    int Id,
    string Title,
    string Description,
    decimal Price,
    string Level,
    string Status,
    DateTime CreatedAt,
    List<CategoryDto> Categories,
    List<LessonEditDto> Lessons);

public record CreateCourseRequest(
    [Required, MinLength(1), MaxLength(200)]
    string Title,
    [Required, MinLength(1), MaxLength(4000)]
    string Description,
    [Range(0, 9999)] decimal Price,
    [Required] string Level,
    int[] CategoryIds);

public record UpdateCourseRequest(
    [Required, MinLength(1), MaxLength(200)]
    string Title,
    [Required, MinLength(1), MaxLength(4000)]
    string Description,
    [Range(0, 9999)] decimal Price,
    [Required] string Level,
    int[] CategoryIds);

public record CreateLessonRequest(
    [Required, MinLength(1), MaxLength(200)]
    string Title,
    [Required, MinLength(1), MaxLength(50000)]
    string Content);
public record UpdateLessonRequest(
    [Required, MinLength(1), MaxLength(200)]
    string Title,
    [Required, MinLength(1), MaxLength(50000)]
    string Content);
public record ReorderLessonsRequest([Required]
    int[] LessonIds);

// analytics

public record AnalyticsRecentStudentDto(string
    FirstName, string LastName, DateTime EnrolledAt,
    int Progress);

public record CourseAnalyticsDto(
    int CourseId,
    string CourseTitle,
    int EnrollmentCount,
    decimal Revenue,
    double AverageRating,

```

```

        double CompletionRate,
        List<AnalyticsRecentStudentDto>
RecentStudents);

public record TimelinePointDto(DateTime Date,
int Count);

// reviews

public record TeacherReviewDto(
    int CourseId,
    string CourseTitle,
    string StudentFirstName,
    string StudentLastName,
    int Grade,
    string Comment,
    DateTime CreatedAt);

// earnings payouts

public record PayoutHistoryItemDto(int Id,
decimal Amount, string Status, DateTime
RequestedAt);

public record EarningsDto(decimal Balance,
List<PayoutHistoryItemDto> History);

public record CreatePayoutRequest([Range(0.01,
999999)] decimal Amount);

// teacher_profile

public record MyTeacherProfileDto(string Bio,
decimal Balance);
public record UpdateMyTeacherProfileRequest([MaxLength(2000)]
string Bio);

//File: Dtos/AuthDtos.cs
using System.ComponentModel.DataAnnotations;

namespace LearningPlatform.Api.Dtos;

public record RegisterRequest(
    [Required, EmailAddress, MaxLength(255)]
string Email,
    [Required, MinLength(6), MaxLength(100)]
string Password,
    [Required, MinLength(1), MaxLength(100)]
string FirstName,
    [Required, MinLength(1), MaxLength(100)]
string LastName,
    [Required] string Role);

public record LoginRequest(
    [Required, EmailAddress] string Email,
    [Required] string Password);

public record UserResponse(int Id, string Email,
string FirstName, string LastName, string Role);

//File: Dtos/AdminDtos.cs
using System.ComponentModel.DataAnnotations;

namespace LearningPlatform.Api.Dtos;

public record AdminFinanceDashboardDto(
    decimal GrossRevenue,
    decimal TotalRefunds,
    decimal PaidToTeachers,
    decimal PendingPayoutsAmount,
    int PendingPayoutsCount,
    List<TimelinePointMoneyDto> RevenueTimeline,

```

```

        List<TimelinePointMoneyDto>
PayoutsTimeline);

public record TimelinePointMoneyDto(DateTime
Date, decimal Amount);

public record AdminPaymentDto(
    int Id,
    DateTime CreatedAt,
    int StudentId,
    string StudentName,
    string StudentEmail,
    int CourseId,
    string CourseTitle,
    decimal Amount,
    string Status);

public record AdminPayoutDto(
    int Id,
    DateTime RequestedAt,
    int TeacherId,
    string TeacherName,
    string TeacherEmail,
    decimal Amount,
    string Status);

public record AdminUserDto(
    int Id,
    string FirstName,
    string LastName,
    string Email,
    string Role,
    string Status);

public record UpdateUserStatusRequest([Required]
string Status);

//File: Models/Payment.cs
namespace LearningPlatform.Api.Models;

public static class PaymentStatus
{
    public const string Completed = "Completed";
    public const string Refunded = "Refunded";
    public static readonly string[] All =
[Completed, Refunded];
}

public static class PayoutStatus
{
    public const string Pending = "Pending";
    public const string Approved = "Approved";
    public const string Rejected = "Rejected";
    public const string Paid = "Paid";
    public static readonly string[] All =
[Pending, Approved, Rejected, Paid];
}

public class Payment
{
    public int Id { get; set; }
    public int StudentId { get; set; }
    public int CourseId { get; set; }
    public decimal Amount { get; set; }
    public string Status { get; set; } =
PaymentStatus.Completed;
    public DateTime CreatedAt { get; set; }

    public User? Student { get; set; }
    public Course? Course { get; set; }
}

public class Payout

```

```

{
    public int Id { get; set; }
    public int TeacherId { get; set; }
    public decimal Amount { get; set; }
    public string Status { get; set; } = PayoutStatus.Pending;
    public DateTime RequestedAt { get; set; }

    public TeacherProfile? Teacher { get; set; }
}

//File: Models/User.cs
namespace LearningPlatform.Api.Models;

public static class Roles
{
    public const string Admin = "Admin";
    public const string Teacher = "Teacher";
    public const string Student = "Student";

    public static readonly string[] All = [Admin, Teacher, Student];
    public static readonly string[] SelfRegisterable = [Teacher, Student];
}

public static class UserStatus
{
    public const string Active = "Active";
    public const string Suspended = "Suspended";

    public static readonly string[] All = [Active, Suspended];
}

public class User
{
    public int Id { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public string Role { get; set; } = Roles.Student;
    public string Status { get; set; } = UserStatus.Active;

    public TeacherProfile? TeacherProfile { get; set; }
}

public class TeacherProfile
{
    public int UserId { get; set; }
    public string Bio { get; set; } = string.Empty;
    public decimal Balance { get; set; }

    public User? User { get; set; }
}

//File: Models/Enrollment.cs
namespace LearningPlatform.Api.Models;

public class Enrollment
{
    public int Id { get; set; }
    public int StudentId { get; set; }
    public int CourseId { get; set; }
    public DateTime EnrolledAt { get; set; }
    public int Progress { get; set; }

```

```

    public User? Student { get; set; }
    public Course? Course { get; set; }
    public List<LessonProgress> LessonProgress { get; set; } = [];
    public Review? Review { get; set; }
}

public class LessonProgress
{
    public int EnrollmentId { get; set; }
    public int LessonId { get; set; }
    public DateTime CompletedAt { get; set; }

    public Enrollment? Enrollment { get; set; }
    public Lesson? Lesson { get; set; }
}

public class Review
{
    public int EnrollmentId { get; set; }
    public int Grade { get; set; }
    public string Comment { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }

    public Enrollment? Enrollment { get; set; }
}

//File: Models/Course.cs
namespace LearningPlatform.Api.Models;

public static class CourseStatus
{
    public const string Draft = "Draft";
    public const string Published = "Published";
    public static readonly string[] All = [Draft, Published];
}

public static class CourseLevel
{
    public const string Beginner = "Beginner";
    public const string Intermediate = "Intermediate";
    public const string Advanced = "Advanced";
    public static readonly string[] All = [Beginner, Intermediate, Advanced];
}

public class Course
{
    public int Id { get; set; }
    public int AuthorId { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public string Level { get; set; } = CourseLevel.Beginner;
    public string Status { get; set; } = CourseStatus.Draft;
    public DateTime CreatedAt { get; set; }

    public TeacherProfile? Author { get; set; }
    public List<Lesson> Lessons { get; set; } = [];
    public List<Category> Categories { get; set; } = [];
}

public class Category
{
    public int Id { get; set; }

```

```

        public string Name { get; set; } =
string.Empty;

        public List<Course> Courses { get; set; } =
[];
    }

public class Lesson
{
    public int Id { get; set; }
    public int CourseId { get; set; }
    public int OrderNumber { get; set; }
    public string Title { get; set; } =
string.Empty;
    public string Content { get; set; } =
string.Empty;

    public Course? Course { get; set; }
}

//File: Program.cs
using LearningPlatform.Api.Data;
using LearningPlatform.Api.Models;
using
Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

var builder =
WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();

builder.Services.AddDbContext<AppDbContext>(opt
ions =>
    options

.UseNpgsql(builder.Configuration.GetConnectionS
tring("Default"))
    .UseSnakeCaseNamingConvention());

builder.Services.AddSingleton<IPasswordHasher<U
ser>, PasswordHasher<User>>();

builder.Services

.AddAuthentication(CookieAuthenticationDefaults
.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.Cookie.Name = "lp.auth";
        options.Cookie.HttpOnly = true;

        options.Cookie.SameSite
SameSiteMode.Lax;
        options.ExpireTimeSpan
TimeSpan.FromDays(7);
        options.SlidingExpiration = true;
        options.Events.OnRedirectToLogin = ctx
=>
        {
            ctx.Response.StatusCode
StatusCodes.Status401Unauthorized;
            return Task.CompletedTask;
        };
        options.Events.OnRedirectToAccessDenied
= ctx =>
        {
            ctx.Response.StatusCode
StatusCodes.Status403Forbidden;
            return Task.CompletedTask;
        };
    });

builder.Services.AddAuthorization();

builder.Services.AddCors(options =>
    options.AddDefaultPolicy(p => p
        .WithOrigins("http://localhost:5173")
        .AllowAnyHeader()
        .AllowAnyMethod()
        .AllowCredentials()));

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

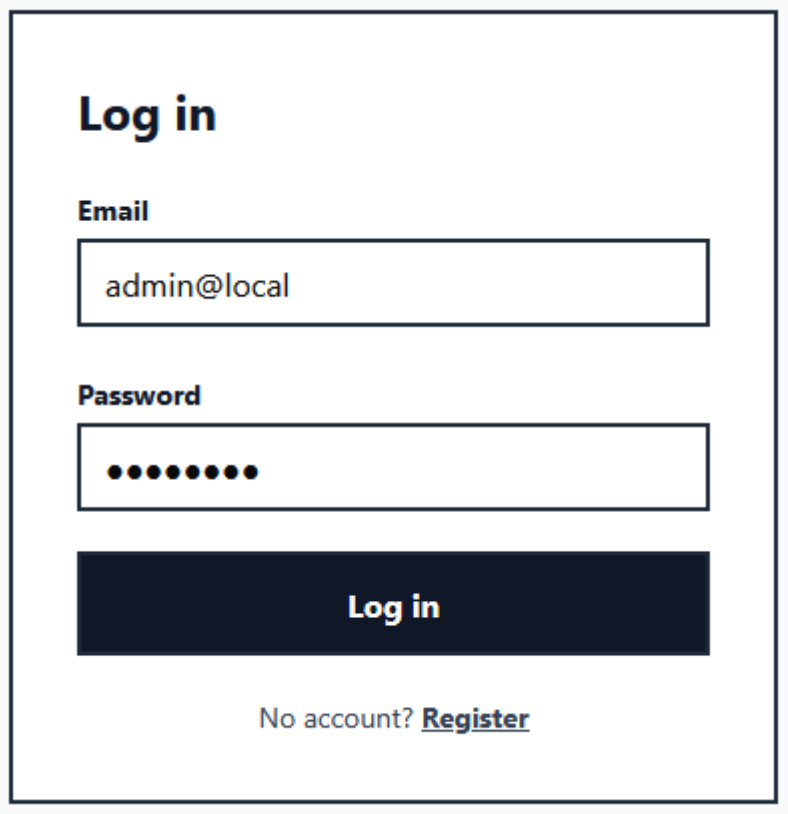
using (var scope = app.Services.CreateScope())
{
    var db
scope.ServiceProvider.GetRequiredService<AppDbC
ontext>();
    db.Database.Migrate();
    //
    //
    DbSeeder.SeedAsync(scope.ServiceProvider);
}

app.UseCors();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

Додаток Г. Зразки сторінок графічного інтерфейсу



Log in

Email

admin@local

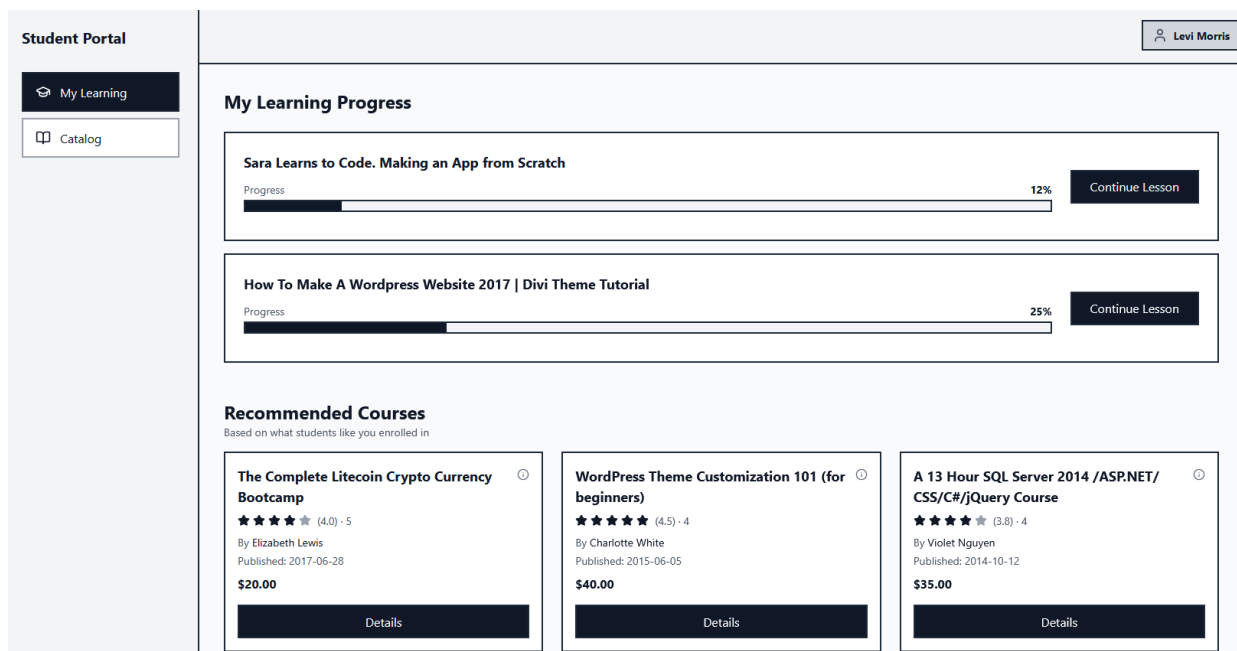
Password

●●●●●●●●

Log in

No account? [Register](#)

Рис. Г.1. Сторінка авторизації



Student Portal

My Learning Progress

Sara Learns to Code. Making an App from Scratch

Progress: 12% [Continue Lesson](#)

How To Make A Wordpress Website 2017 | Divi Theme Tutorial

Progress: 25% [Continue Lesson](#)

Recommended Courses
Based on what students like you enrolled in

The Complete Litecoin Crypto Currency Bootcamp ★★★★★ (4.0) · 5 By Elizabeth Lewis Published: 2017-06-28 \$20.00 Details	WordPress Theme Customization 101 (for beginners) ★★★★★ (4.5) · 4 By Charlotte White Published: 2015-06-05 \$40.00 Details	A 13 Hour SQL Server 2014 /ASP.NET/ CSS/C#/jQuery Course ★★★★★ (3.8) · 4 By Violet Nguyen Published: 2014-10-12 \$35.00 Details
---	--	---

Рис. Г.2. Головна сторінка (Студент)

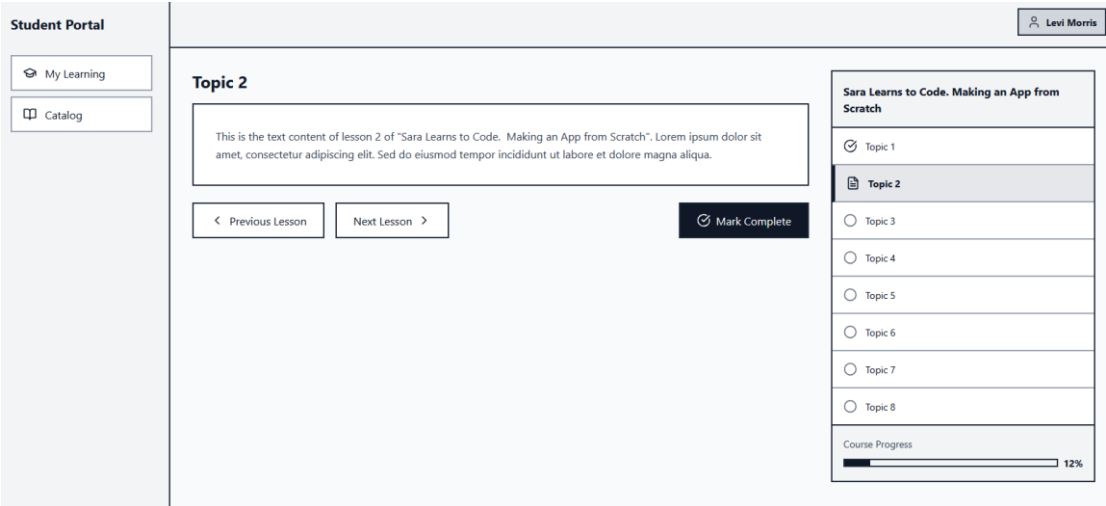


Рис. Г.3. Сторінка навчання (Студент)

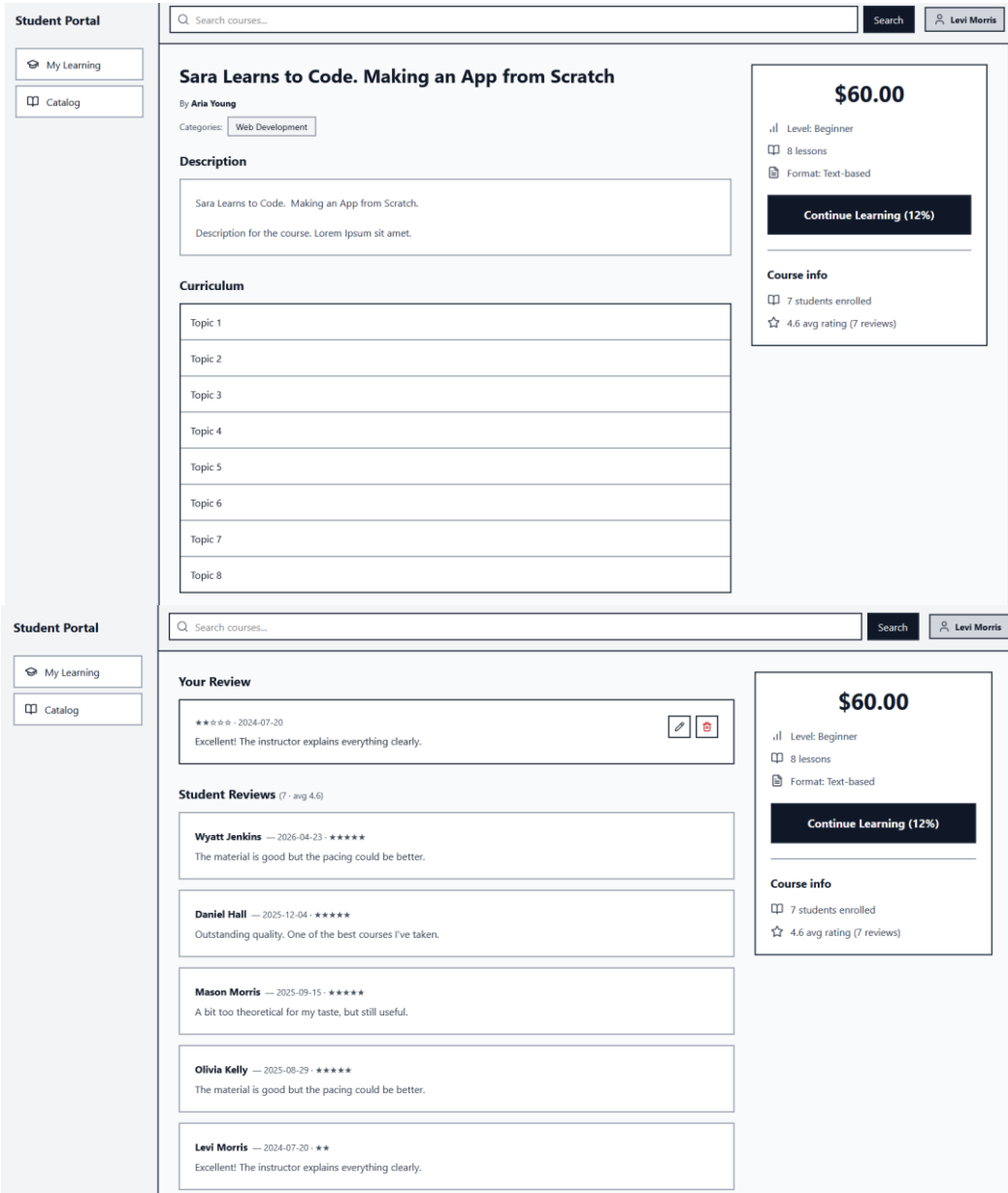


Рис. Г.4-Г.5. Сторінка курсу (Студент)

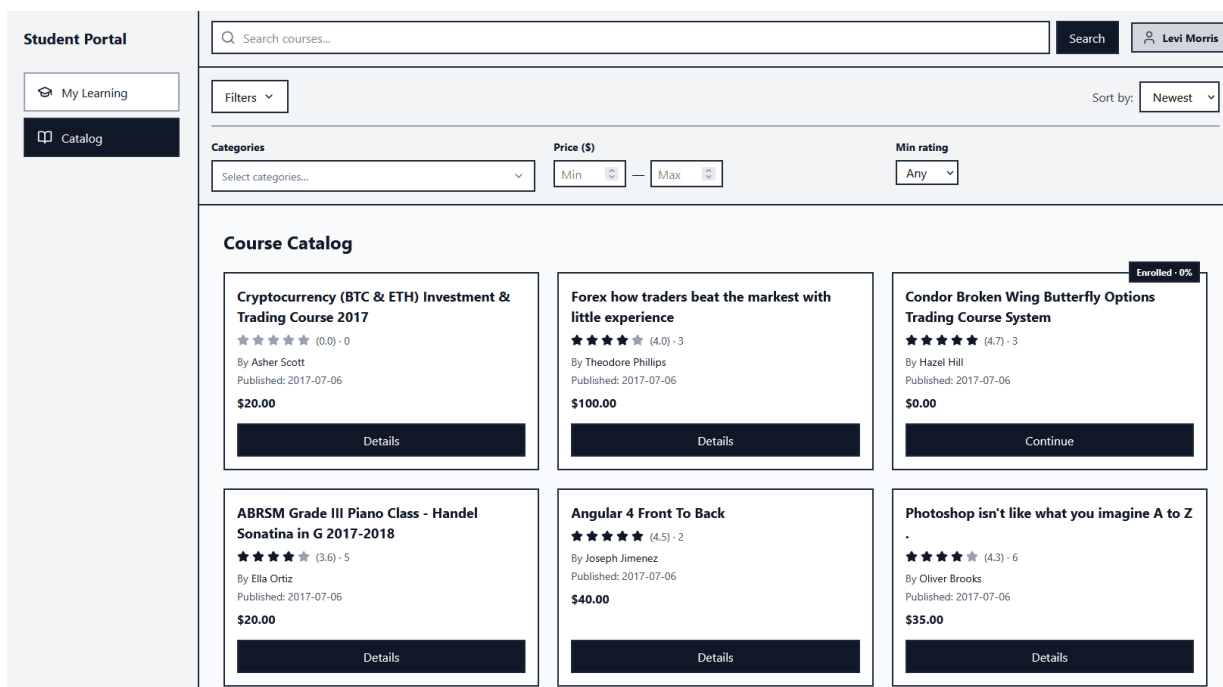


Рис. Г.6. Сторінка каталогу курсів (Студент)

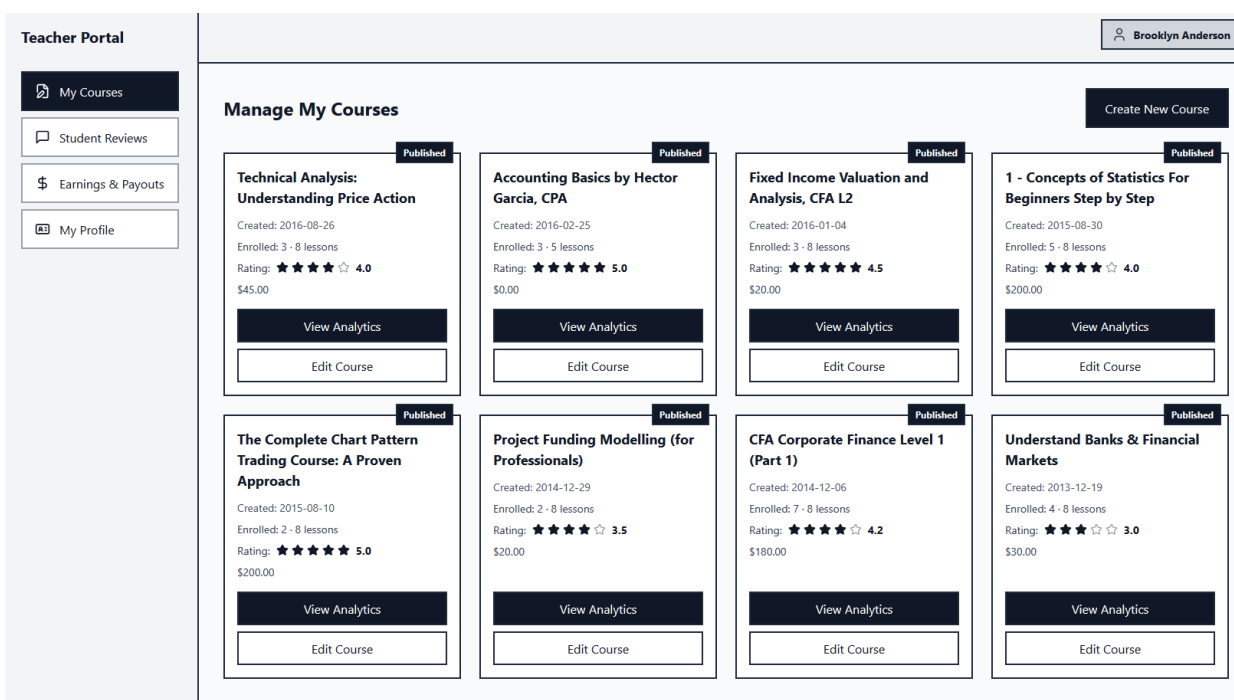


Рис. Г.7. Головна сторінка (Викладач)

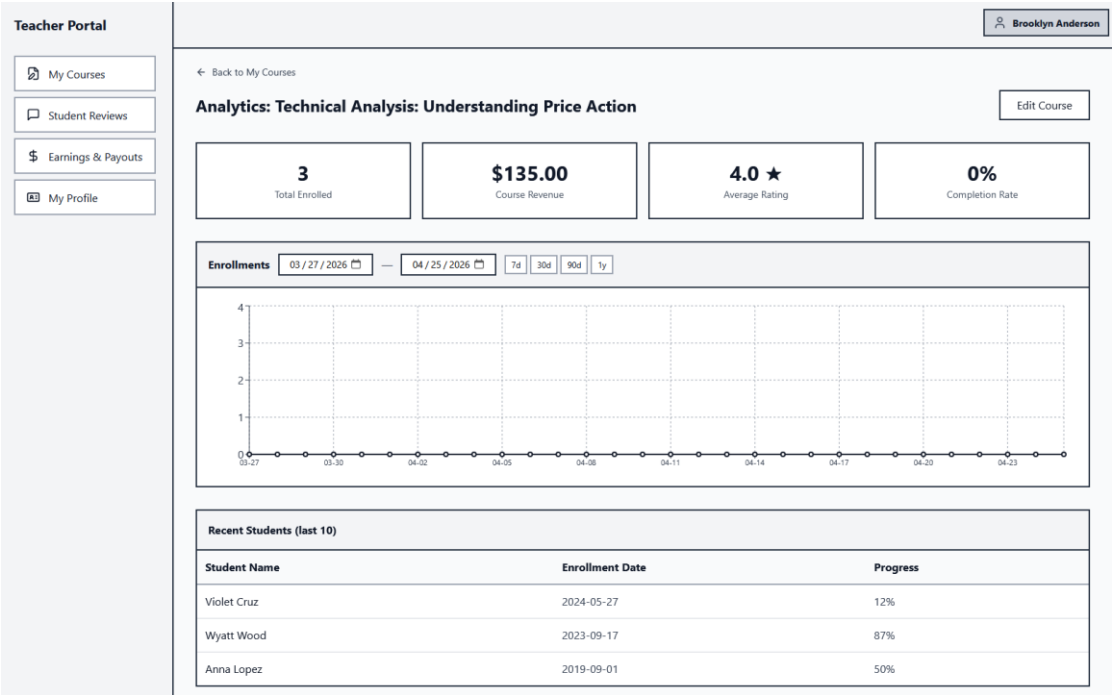


Рис. Г.8 Сторінка аналітики курсу (Викладач)

Teacher Portal

My Courses

Student Reviews

Earnings & Payouts

My Profile

Brooklyn Anderson

Back to My Courses

Edit CoursePublished

Course details

Title

Technical Analysis: Understanding Price Action

Description

Technical Analysis: Understanding Price Action.
Description for the course. Lorem Ipsum sit amet.

Price (\$)

45

Level

Beginner

Categories

Business Finance

Save changes

Teacher Portal

My Courses

Student Reviews

Earnings & Payouts

My Profile

Brooklyn Anderson

Lesson 2

Topic 2

Lesson 3

Topic 3

Lesson 4

Topic 4

Lesson 5

Topic 5

Lesson 6

Topic 6

Lesson 7

Topic 7

Lesson 8

Topic 8

Add lesson

Delete course

Deleting a course is permanent. You can only delete a course that has no enrollments.

Delete course

Рис. Г.9-10. Сторінки редагування курсу (Викладач)

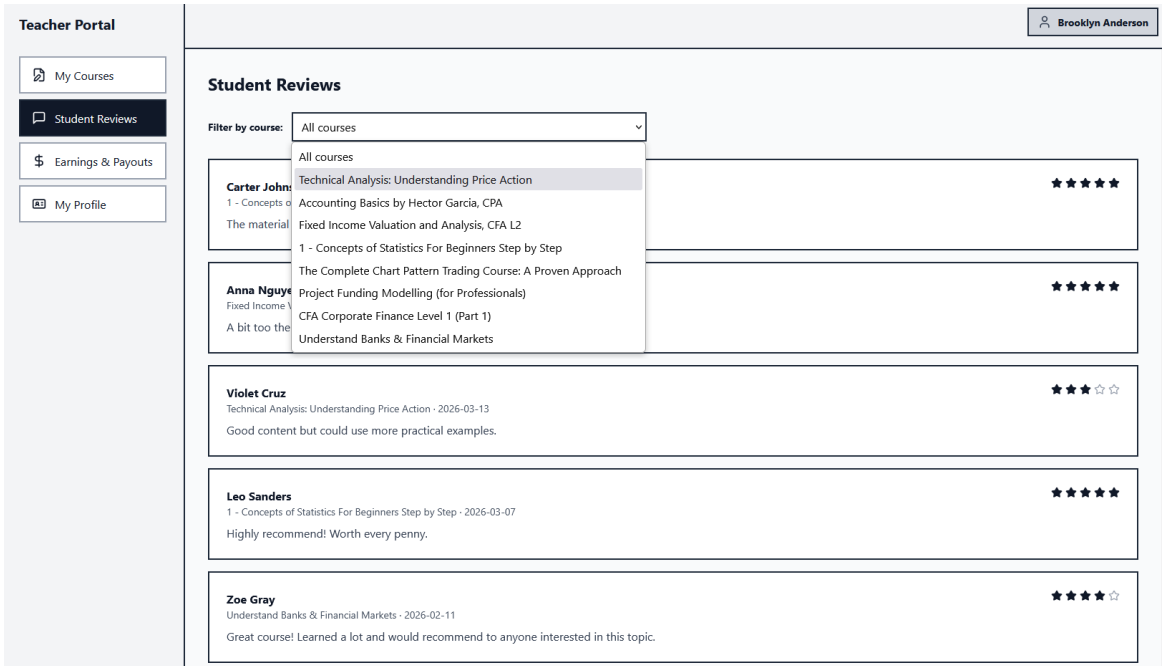


Рис. Г.11. Сторінка відгуків (Викладач)

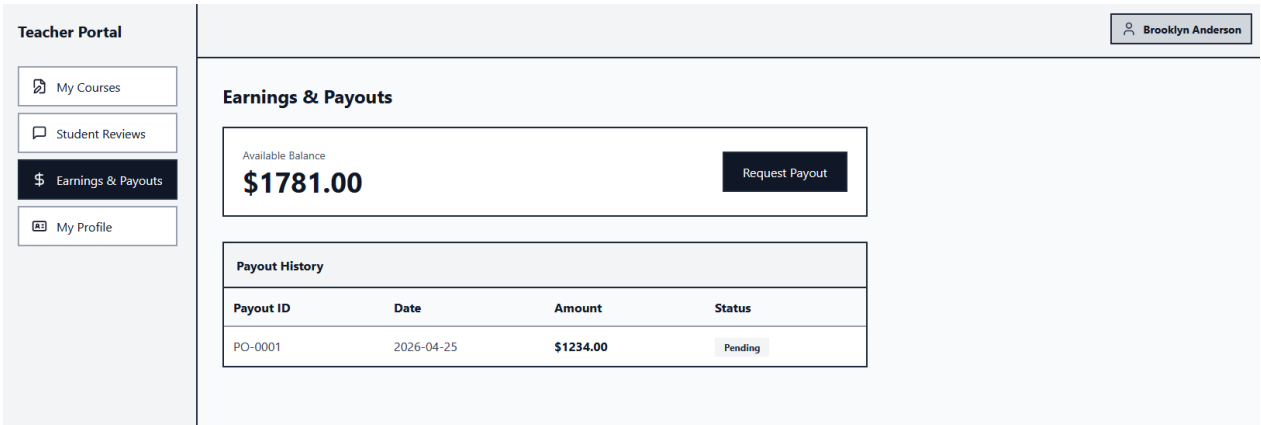


Рис. Г.12. Сторінка виплат (Викладач)

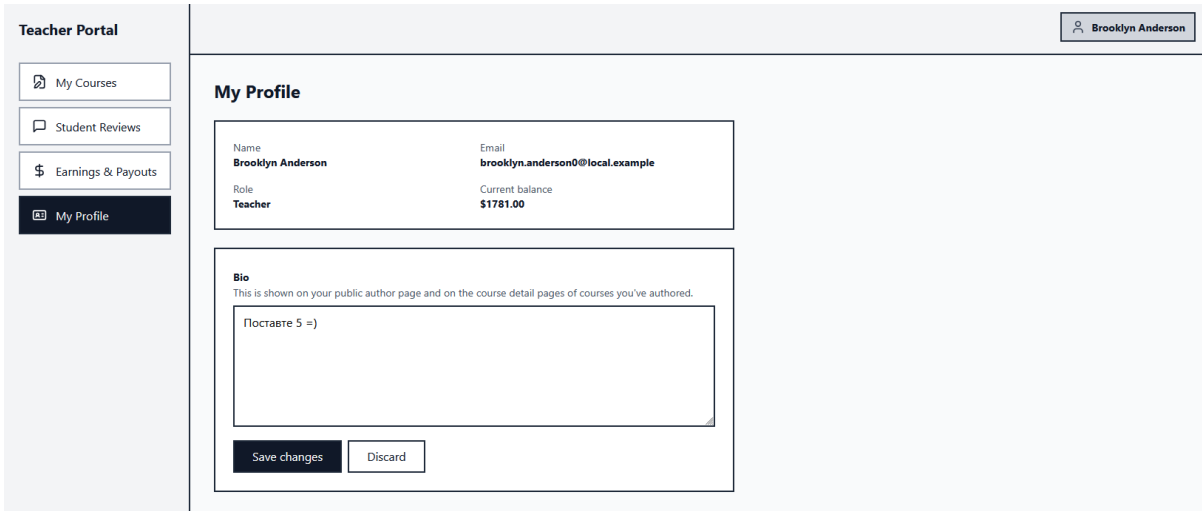
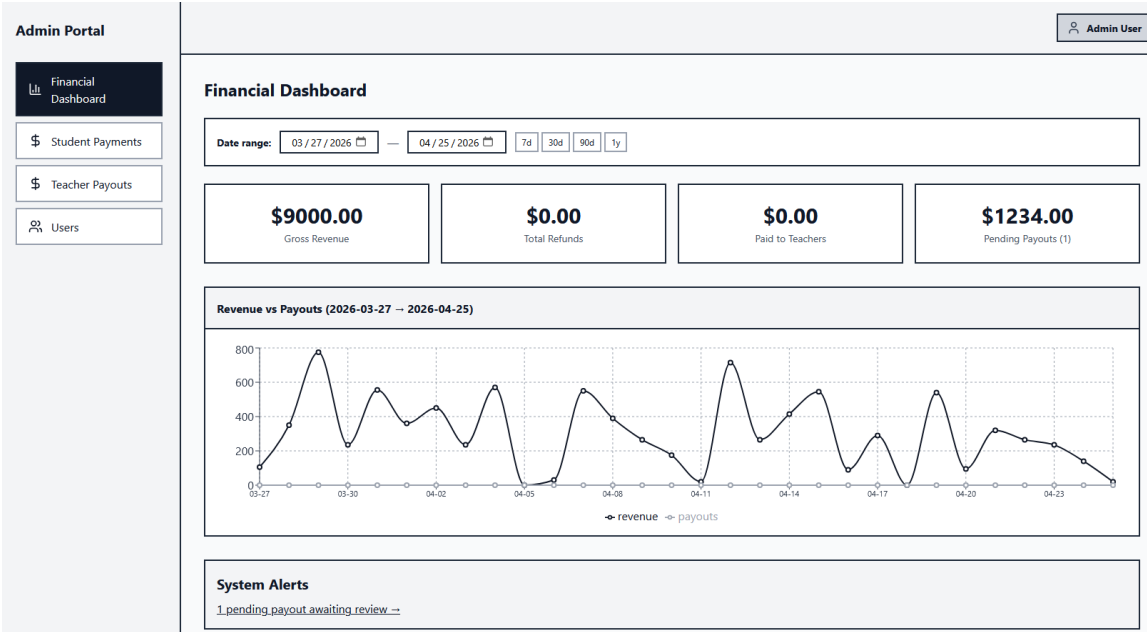


Рис. Г.13. Сторінка профілю (Викладач)



Admin Portal

Financial Dashboard

Student Payments

Teacher Payouts

Users

Admin User

User Management

Role: AllStatus: All

Search by name or email...

User ID	Name	Email	Role	Status	Actions
U-0001	Admin User	admin@local	Admin	Active	Suspend
U-0002	Brooklyn Anderson	brooklyn.anderson0@local.example	Teacher	Active	Suspend
U-0003	Ava Turner	ava.turner0@local.example	Teacher	Active	Suspend
U-0004	Charlotte Ramirez	charlotte.ramirez0@local.example	Teacher	Active	Suspend
U-0005	Stella Evans	stella.evans0@local.example	Teacher	Active	Suspend
U-0006	Lucas Ward	lucas.ward0@local.example	Teacher	Active	Suspend

Рис. Г.17. Сторінка користувачів (Адмін)